

Copy-Protection from UPO, Revisited

Or: How to use Unclonable Puncturable Obfuscation (UPO)

Prabhanjan Ananth*

Amit Behera[†]

Zikuan Huang[‡]

Abstract

Quantum copy-protection is a foundational notion in quantum cryptography that leverages the governing principles of quantum mechanics to tackle the problem of software anti-piracy. Despite progress in recent years, precisely characterizing the class of functionalities that can be copy-protected is still not well understood.

Two recent works, by [Coladangelo and Gunn, STOC 2024] and [Ananth and Behera, CRYPTO 2024] showed that puncturable functionalities can be copy-protected. Both works have significant caveats with regards to the underlying cryptographic assumptions and additionally restrict the output length of the functionalities to be copy-protected. In this work, we make progress towards simultaneously addressing both the caveats. We show the following:

- **Revisiting Unclonable Puncturable Obfuscation (UPO):** We revisit the notion of UPO introduced by [Ananth and Behera, CRYPTO 2024]. We present a new approach to construct UPO and a variant of UPO, called independent-secure UPO. Unlike UPO, we show how to base the latter notion on well-studied assumptions.
- **Copy-Protection from Independent-secure UPO:** Assuming independent-secure UPO, we show that any m -bit, for $m \geq 2$, puncturable functionality can be copy-protected.
- **Copy-Protection from UPO:** Assuming UPO, we show that any 1-bit puncturable functionality can be copy-protected. The security of copy-protection holds against *identical* challenge distributions.

*University of California, Santa Barbara

[†]Ben-Gurion University

[‡]Tsinghua University

Contents

1	Introduction	4
1.1	Our Results in a Nutshell	5
2	Puncturing Security, Copy-Protection and UPO: Recap	5
3	Results: In More Detail	8
4	Technical Overview	10
4.1	New Observations	11
4.2	Copy-Protection from UPO	12
4.3	New Variant of UPO: Independent-Secure UPO	13
4.4	Copy-Protection from Independent-Secure UPO	13
4.5	Independent-Secure UPO: Construction	17
4.5.1	Copy protection for generalized puncturable signature	17
4.5.2	Security Proof	18
4.5.3	Instantiating copy-protection for generalized puncturable signature	22
5	Preliminaries	23
5.1	Keyed Circuit Class	23
6	Generalized Puncturing Security	24
7	Unclonable Puncturable Obfuscation (UPO)	26
7.1	Generalized Security	27
7.2	Independent-secure UPO	28
7.2.1	Composition Theorem	28
8	Copy-Protection	30
8.1	Copy-Protection for Generalized Puncturable Signatures	32
9	Notation and Useful Lemmas	34
9.1	Puncture-Hiding Lemma	36
9.2	Output-Hiding Lemma	37
9.3	Non-Local Puncture-Hiding Lemma	39
10	Copy-Protecting 1-Bit Output Average-Case Puncturable Circuit Classes	44
11	Copy-Protection from Independent-Secure UPO	46
11.1	1-out-of-2 Copy-protection for generalized puncturable functionalities from independent secure UPO	50
12	Constructing UPO: A Modular Approach	52
12.1	UPO from Copy-Protection for Generalized Puncturable Signatures	53
12.2	Copy-Protecting Generalized Puncturable Signatures	62
12.2.1	Copy-protection for generalized puncturable signature construction.	64

12.3 Proof of antipiracy for the construction of Copy-protection for Signatures given in	
Section 12.2.1	67
12.3.1 Proof of Theorem 12.6	69
A Proof of lemma 12.3	98
B Proof of lemma 12.9	100

1 Introduction

The no-cloning theorem of quantum mechanics has inspired the design of many novel quantum cryptographic primitives (i.e. unclonable primitives) that solve tasks that otherwise cannot be solved solely using classical technology. One such primitive of interest is quantum copy-protection. Informally speaking, copy-protection, introduced by Aaronson [Aar09], allows for compiling a (classical) program f into a quantum state such that this quantum state ρ_f still retains the functionality of the original program. Additionally, it is required that given just one copy of ρ_f it is computationally infeasible to produce two or more copies such that all the copies approximately compute f . Copy-protection, having been conceived in 2009, is one of the oldest unclonable primitives. Investigating its (in)feasibility is an active and ongoing research direction. Indeed, in recent years, there has been significant progress made on understanding the possibility of quantum copy-protection [AL21, ALL⁺21, CMP24, CLLZ21, AK22, AKL⁺22, LLQZ22, CHV23, ÇG23, CG24, AB24, MM24] and its variants [AL21, KNY21, KN22, KN23, BGK⁺24]. Since learnable functionalities cannot be copy-protected, the main focus has been to identify interesting classes of functionalities that can or cannot be copy-protected. Currently, we know the following:

- **Impossibility:** It is established [AL21] that not all unlearnable functionalities can be copy-protected. However, the class of unlearnable functionalities ruled out by [AL21] is highly contrived.
- **Concrete cryptographic functionalities:** Two notable works [CLLZ21, LLQZ22] showed that specific cryptographic functionalities such as decryption, signing and pseudorandom functionalities can be copy-protected. Some of the results proposed in these works have limitations in the following sense: they focus on copy-protecting specific instantiations of these primitives rather than being oblivious of its implementation. For instance, [LLQZ22] show that the signing functionality proposed by [SW21] can be copy-protected and in particular, they do not prove that *any* signing functionality can be copy-protected. A few recent works [CMP24, CHV23, AKL⁺22] proposed copy-protecting point functions in different settings.
- **Broad class of cryptographic functionalities:** Two subsequent works [CG24, AB24] gave a characterization for a class of functionalities to be copy-protected: they showed that any class of functionalities, with long output, that satisfy puncturing security can be copy-protected. Roughly speaking, puncturing security, associated with a cryptographic functionality, states that it should be computationally infeasible to predict the output on a point x even given the description of the functionality punctured at x . The indistinguishability obfuscation (iO) revolution led to the development of several cryptographic functionalities satisfying puncturing security, including pseudorandom functions [BGI14, BW13, KPTZ13], signatures [KLW15], encryption [CHJV14] and many more. Interestingly, the functionalities considered in the works in the second bullet can be appropriately cast as special cases of puncturable functionalities. We will explore this direction in this work.

Puncturable Functionalities: Towards a Full Characterization. The goal of our work is to investigate the feasibility of copy-protecting *all* puncturable functionalities. There are two significant caveats in both [CG24, AB24]. While both [CG24, AB24] explore copy-protecting puncturable functionalities, they have two major weaknesses:

- **Weakness 1: Computational Assumptions.** [CG24] uses quantum state obfuscation and [AB24] uses unclonable puncturable obfuscation as intermediate tools to achieve copy-protection. Currently, the existence of quantum state obfuscation is only known in the classical oracle model [BBV24] and the existence of unclonable puncturable obfuscation relies upon new and unproven conjectures. Basing the existence of copy-protecting puncturable functionalities upon well-studied assumptions is an important open problem.
- **Weakness 2: Restriction on the Output Length.** The second weakness is that they necessarily require that the output of puncturable functionality, denoted by m , has to be super-logarithmic in the security parameter¹. This means that we currently do not know how to copy-protect even 1-bit output puncturable functionalities, under any assumption!

The possibility of copy-protecting many cryptographic functionalities from advanced encryption systems (such as homomorphic or functional encryption) or advanced signatures (such as homomorphic signatures) is yet to be fully explored. Establishing the result that *any* puncturable functionality can be copy-protected could invariably aid this process. Such a result would reduce the task of copy-protecting the cryptographic functionality of interest to demonstrating that this functionality can be suitably instantiated to satisfy puncturing security; with the latter being a purely classical cryptographic problem in nature.

1.1 Our Results in a Nutshell

We discuss our results in a nutshell. Our main results are as follows:

1. We define a novel variant of UPO, that we refer to as independent-secure UPO. Unlike UPO, independent-secure UPO can be based on well-studied assumptions.
2. Assuming independent-secure UPO and indistinguishability obfuscation, we show that any m -bit puncturable functionality can be copy-protected as long as $m \geq 2$. A caveat of our result is that the adversarial advantage is not negligible. When $m = \omega(\log(\lambda))$, where λ is the security parameter, we show that we can achieve a weaker notion of copy-protection called **one-out-of-two** copy-protection, which is a special case of the one-out-of-many copy-protection, introduced by [KN23].
3. Assuming UPO and indistinguishability obfuscation, we show that any 1-bit puncturable functionality can be copy-protected. In this case, we show that the adversarial advantage is negligible.

To discuss the results in more detail, we first recall the existing notions in Section 2. We then state the detailed results in Section 3.

2 Puncturing Security, Copy-Protection and UPO: Recap

We first recall the following: (a) the framework of generalized puncturable keyed circuit classes and puncturing security, (b) the definition of unclonable puncturable obfuscation and, (c) the definition of copy-protection.

¹We note that [CG24] also consider a different notion of puncturing (referred to as *decision* puncturing). However, the class of functionalities that can satisfy this definition is limited – we will discuss this more later.

Generalized puncturable keyed circuit classes and puncturing security We are interested in keyed circuit classes. These are circuit classes where every circuit in this class is of the form C_k , where k belongs to a keyspace. We say that a keyed circuit class satisfies generalized puncturing security if it is equipped with an algorithm **GenPuncture** such that it takes as input a key $k \in \mathcal{K}_\lambda$, inputs $x_1, x_2 \in \{0, 1\}^{n(\lambda)}$, circuits μ_1, μ_2 ² and outputs $G_k^{(x_1, x_2, \mu_1, \mu_2)}$ such that the output circuit has the same functionality as C_k on all points except x_1 and x_2 , but on x_i , $G_k^{(x_1, x_2, \mu_1, \mu_2)}$ outputs $\mu_i(x_i)$ for both $i = 1$ and 2 . Note that we can equip any circuit class with a canonical **GenPuncture** algorithm that satisfies the correctness property mentioned above, and hence we can view any circuit class as a generalized puncturable keyed circuit class, without loss of generality. A generalized puncturable keyed circuit class is an *m-bit keyed circuit class* if every circuit in this class has output length m .

Generalized puncturing security. For a generalized puncturable keyed circuit class, we can define generalized puncturing security, associated with distributions $\mathcal{D}_\mathcal{K}$, $\mathcal{D}_{\mathcal{X}^2}$, as follows: consider the following experiment in which a QPT adversary **Adv** participates in.

- Sample $k \leftarrow \mathcal{D}_\mathcal{K}$ and sample (x_1, x_2) from $\mathcal{D}_{\mathcal{X}^2}$.
- **Adv** submits two functions (μ_1, μ_2) .
- Run **GenPuncture** $(k, x_1, x_2, \mu_1, \mu_2)$ and send the output $G_k^{(x_1, x_2, \mu_1, \mu_2)}$, along with (x_1, x_2) , to **Adv**.
- We say that **Adv** wins if, given $(G_k^{(x_1, x_2, \mu_1, \mu_2)}, x_1, x_2)$, can predict either $C_k(x_1)$ or $C_k(x_2)$.

We say that the scheme is secure if the probability that the adversary wins is negligibly close to p_{triv} , where p_{triv} is the maximum probability with which an efficient algorithm would guess the true output given just the input and the knowledge of the distributions $(\mathcal{D}_\mathcal{K}, \mathcal{D}_{\mathcal{X}^2})$ (in particular, without any information about the key k). In this case, we say that the generalized puncturable keyed circuit class **satisfies p_{triv} -general puncturing security**.

Unclonable Puncturable Obfuscation (UPO): Definition. UPO is a compiler that transforms a classical circuit into a quantum state that preserves the functionality of the original state. In more detail, it consists of the following efficient algorithms:

- Obfuscation, **Obf**: takes as input a keyed circuit³ C and outputs a quantum state ρ_C .
- Evaluation, **Eval**: takes as input a quantum state ρ_C with input length ℓ , an input $x \in \{0, 1\}^\ell$ and outputs $C(x)$.

Informally speaking, the security of UPO is defined as follows: consider a cloning adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$.

- $\mathcal{A}$ submits a keyed circuit C to the challenger. It also additionally submits two circuits $(\mu_\mathcal{B}, \mu_\mathcal{C})$; the role of $\mu_\mathcal{B}, \mu_\mathcal{C}$ will be clear below.

²such that if $x_1 = x_2$, $\mu_1(x_1) = \mu_2(x_2)$

³For UPO, we consider only obfuscating keyed circuits. A keyed circuit family $\mathfrak{C}$ is one wherein there is a circuit C associated with $\mathfrak{C}$ and every circuit in $\mathfrak{C}$ is of the form $C(k, \cdot)$, for some string k . We refer the reader to Section 5.1 for a detailed description of keyed circuit classes.

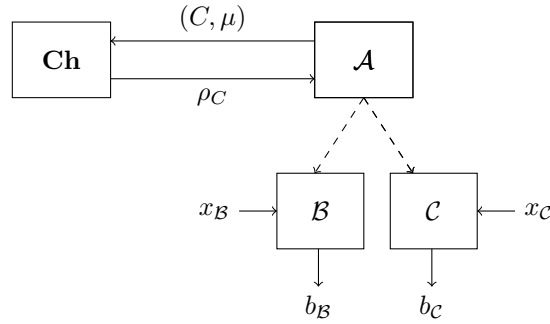
- The challenger then samples the inputs (x_B, x_C) from a distribution $\mathcal{D}_{\mathcal{X}^2}$. It also samples a random bit $b \leftarrow \{0, 1\}$. If $b = 0$, it obfuscates C to obtain ρ_C . If $b = 1$, it first computes the following circuit $G_k^{(x_B, x_C, \mu_B, \mu_C)}$ using **GenPuncture**:

$$G_k^{(x_B, x_C, \mu_B, \mu_C)}(x) = \begin{cases} C_k(x), & x \neq x_B, x \neq x_C \\ \mu_B(x_B), & x = x_B \\ \mu_C(x_C), & x = x_C, \end{cases}$$

It then obfuscates the circuit $G_k^{(x_B, x_C, \mu_B, \mu_C)}$ to obtain ρ_C . It sends ρ_C to $\mathcal{A}$.

- $\mathcal{A}$ then computes a bipartite state and shares one part with $\mathcal{B}$ and the other part with $\mathcal{C}$.
- $\mathcal{B}$ then gets x_B and $\mathcal{C}$ gets x_C . The output of $\mathcal{B}$ and $\mathcal{C}$ is (b_B, b_C) .

We say that $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ wins if $b_B = b$ and $b_C = b$. The security holds if $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ win with probability at most negligibly more than $\frac{1}{2}$.



Some common input distributions of interest for $\mathcal{D}_{\mathcal{X}^2}$ are product uniform distribution (i.e. both x_B and x_C are sampled independently from the uniform distribution) and diagonal distribution (i.e. when x_B is sampled uniformly at random and x_C is set to be x_B).

Copy-Protection: Definition. Similar to UPO, copy-protection is also a functionality preserving compiler. It consists of the following efficient algorithms:

- Copy-Protection, **CP.CopyProtect**: takes as input a circuit C and outputs a quantum state ρ_C .
- Evaluation, **CP.Eval**: takes as input a quantum state ρ_C with input length ℓ , an input $x \in \{0, 1\}^\ell$, and outputs $C(x)$.

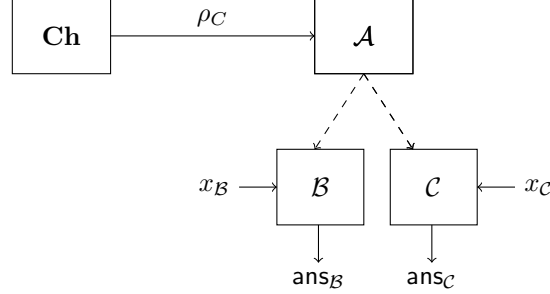
The security definition states that, given ρ_C , it should be infeasible for an efficient adversary to produce a bipartite state such that both the parts can be used to implement C .

Denote the cloning adversary to be $(\mathcal{A}, \mathcal{B}, \mathcal{C})$. Consider the following experiment:

- The challenger samples a circuit C from a distribution $\mathcal{D}_{\mathcal{C}}$. It then copy-protects C to obtain the quantum state ρ_C . It sends ρ_C to $\mathcal{A}$.
- $\mathcal{A}$ then computes a bipartite state and shares one part with $\mathcal{B}$ and the other part with $\mathcal{C}$.
- The challenger then samples (x_B, x_C) from a distribution $\mathcal{D}_{\mathcal{X}^2}$.

- $\mathcal{B}$ then gets $x_{\mathcal{B}}$ and $\mathcal{C}$ gets $x_{\mathcal{C}}$. The output of $\mathcal{B}$ and $\mathcal{C}$ is $(y_{\mathcal{B}}, y_{\mathcal{C}})$.

We say that $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ wins if $\text{ans}_{\mathcal{B}} = C(x_{\mathcal{B}})$ and $\text{ans}_{\mathcal{C}} = C(x_{\mathcal{C}})$. The security holds if $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ win with probability at most negligibly more than the trivial success probability p_{triv} , which is defined to be the success probability (maximized over all cloning adversaries $(\mathcal{A}, \mathcal{B}, \mathcal{C})$) wherein $\mathcal{A}$ hands over the entire state ρ_C to one of $\mathcal{B}, \mathcal{C}$ and gives nothing to the other. More generally, we define the winning probability to be at most $p_{\text{triv}} + \varepsilon$, where ε is defined to be the **adversarial advantage**. In a secure scheme, ε is negligible but in our feasibility results, we also consider cases when ε is not negligible.



The trivial success probability depends on $\mathcal{D}_{\mathcal{C}}, \mathcal{D}_{\chi^2}$ and the output length of C . Specifically, the trivial success probability is at least $\frac{1}{2^k}$, where k is the output length of C .

one-out-of-two Copy-Protection. [KN23] conceived a weaker notion of copy-protection called one-out-of-two copy-protection wherein the success probability of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ is determined to be the sum of the probability that $\mathcal{B}$ succeeds and the probability that $\mathcal{C}$ succeeds. For the scheme to be secure, we require that the success probability is at most $1 + p_{\text{triv}} + \varepsilon$, where ε is negligible. As before, we define ε to be **adversarial advantage**.

3 Results: In More Detail

Our goal is to address both the above mentioned weaknesses in our work. Towards this, we investigate unclonable puncturable obfuscation further and explore its variants. Specifically, we will show how to achieve a variant of unclonable puncturable obfuscation from well-studied assumptions. We will then show how to build copy-protection for puncturable functionalities from UPO and its new variant.

Novel Variant of Unclonable Puncturable Obfuscation (UPO). Recall that in the notion of UPO [AB24], there are only two (equally probable) cases that the non-local adversaries, i.e., $\mathcal{B}$ and $\mathcal{C}$ need to guess: either the obfuscated circuit is punctured for both $\mathcal{B}$'s and $\mathcal{C}$'s inputs, or the circuit is unpunctured. In the new variant, referred to as *independent-secure UPO*, there are four cases where the possibility of puncturing the circuit at $\mathcal{B}$'s input is uniformly random and independent of puncturing the circuit at $\mathcal{C}$'s input. The new variant also satisfies desirable properties such as the composition theorem, as shown for UPO [AB24]. For a more detailed description, refer to the technical overview (Section 4.3).

Interestingly, unlike UPO, independent-secure UPO can be based on well-studied assumptions.

Theorem 3.1 (Informal). *Assuming sub-exponentially secure post-quantum indistinguishability obfuscation and sub-exponentially secure learning with errors, independent-secure unclonable puncturable obfuscation exists.*

Towards proving the above theorem, we follow a modular approach. We first show that copy-protecting a special class of digital signature schemes implies the existence of independent-secure UPO with the same challenge distribution. Next, we show how to copy-protect this special class of digital signature schemes with the uniform challenge distribution.

Copy-Protection from Independent-Secure UPO. We show how to copy-protect m -bit output, for $m \geq 2$, puncturable functionalities using independent-secure UPO. We show the following.

Theorem 3.2 (Informal). *Assuming independent-secure UPO and post-quantum secure indistinguishability obfuscation, there exists a copy-protection scheme:*

- *for m -bit output keyed circuit classes satisfying p_{triv} -generalized puncturing security,*
- *with adversarial advantage being $\frac{1}{3} + \frac{p_{\text{triv}}}{3}$.*

The prior work [AB24] presented a copy-protection scheme for m -bit output functionalities, from UPO, where the adversarial advantage was at most $2p_{\text{triv}}$. In the above result (Theorem 3.2), we show that by slightly worsening this bound we can significantly weaken the assumption from UPO (which relies upon unproven conjectures) to independent-secure UPO (that can be based on well-studied assumptions).

Theorem 3.3 (Informal). *Assuming independent-secure UPO and post-quantum secure indistinguishability obfuscation, there exists a copy-protection satisfying one-out-of-two security:*

- *for m -bit output keyed circuit class, with $m = \omega(\log(\lambda))$, satisfying p_{triv} -generalized puncturing security, where p_{triv} is negligible,*
- *with adversarial advantage being negligible.*

Prior work by [KN23] has constructed one-out-of-two copy-protection schemes only for point functions.

Copy-Protection from UPO. In Theorem 3.2, we showed how to copy-protect any m -bit output puncturable functionality as long as $m \geq 2$. Unfortunately, the bound we obtain for the case when $m = 1$ is useless. This leaves the problem of showing the feasibility of copy-protecting 1-bit output puncturable functionalities based on any assumption⁴. We address this question by showing copy-protection for 1-bit puncturable functionalities based on UPO, as follows.

Theorem 3.4 (Informal). *Assuming UPO and post-quantum secure indistinguishability obfuscation, there exists a copy-protection scheme:*

- *for 1-bit keyed circuit class satisfying p_{triv} -generalized puncturing security,*
- *with the adversarial advantage being negligible.*

m -Keyed Circuit Class (p_{triv} - puncturing security)	Assumptions	Adversarial advantage	CP Definition
$m = \omega(\log(\lambda))$ [CG24]	Quantum state iO	negl	Standard
$m \geq 2$ [AB24]	UPO	p_{triv}	Standard
$m = 1$ [This work]	UPO	negl	Standard
$m \geq 2$ [This work]	Ind. secure UPO	$\frac{1}{3} + \frac{p_{\text{triv}}}{3}$	Standard
$m = \omega(\log(\lambda))$ [This work]	Ind. secure UPO	negl*	1-out-2 Security

Figure 1: State of the art of copy-protecting puncturable functionalities. Note that p_{triv} for an m -bit keyed circuit class is always at least $\frac{1}{2^m}$. *In the last row, we consider only the case when p_{triv} is negligible.

Results Summary. We summarise the results in the table below.

4 Technical Overview

We first recall the prior work [AB24] on designing copy-protection from UPO.

Notation. Before we recall the construction of copy-protection for long output functionalities from UPO, we will first establish some notations that would be useful.

- $\mathbf{P}_{y'_B, y'_C} [y_B, y_C]$: is the probability that $\mathcal{B}, \mathcal{C}$ outputs y_B, y_C respectively on inputs x_B, x_C when $k, (x_B, x_C)$ are drawn from $\mathcal{D}_K, \mathcal{D}_{\mathcal{X}^2}$ respectively condition on $C_k(x_B) = y'_B$ and $C_k(x_C) = y'_C$.
- $\mathbf{P}_{y'_B \rightarrow y_B^*, y'_C \rightarrow y_C^*} [y_B, y_C]$: is the probability that $\mathcal{B}, \mathcal{C}$ outputs y_B, y_C respectively on inputs x_B, x_C , when the challenger uses the punctured circuit G_{k^*} at the beginning and when $k, (x_B, x_C)$ are drawn from $\mathcal{D}_K, \mathcal{D}_{\mathcal{X}^2}$ respectively condition on $C_k(x_B) = y'_B$ and $C_k(x_C) = y'_C$. On top of that, G_{k^*} outputs y_B^*, y_C^* on input x_B, x_C , respectively.

Prior Work: Copy-Protection for Long-Output Functionalities from UPO. [AB24] showed that long-output functionalities can be copy-protected assuming UPO. The construction of copy-protection is simple: to copy protect C , simply obfuscate C to obtain a quantum state ρ_C . To evaluate ρ_C on an input x , run the evaluation algorithm of UPO.

⁴We remark that the notion of *decision copy-protection* considered in [CG24] also requires the output of the functionalities to be large and hence their result has no implications for 1-bit output functionalities.

Suppose $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ wins the copy-protection experiment with probability ε . We construct a cloning adversary $(\mathcal{R}_\mathcal{A}, \mathcal{R}_\mathcal{B}, \mathcal{R}_\mathcal{C})$ that breaks the security experiment of UPO.

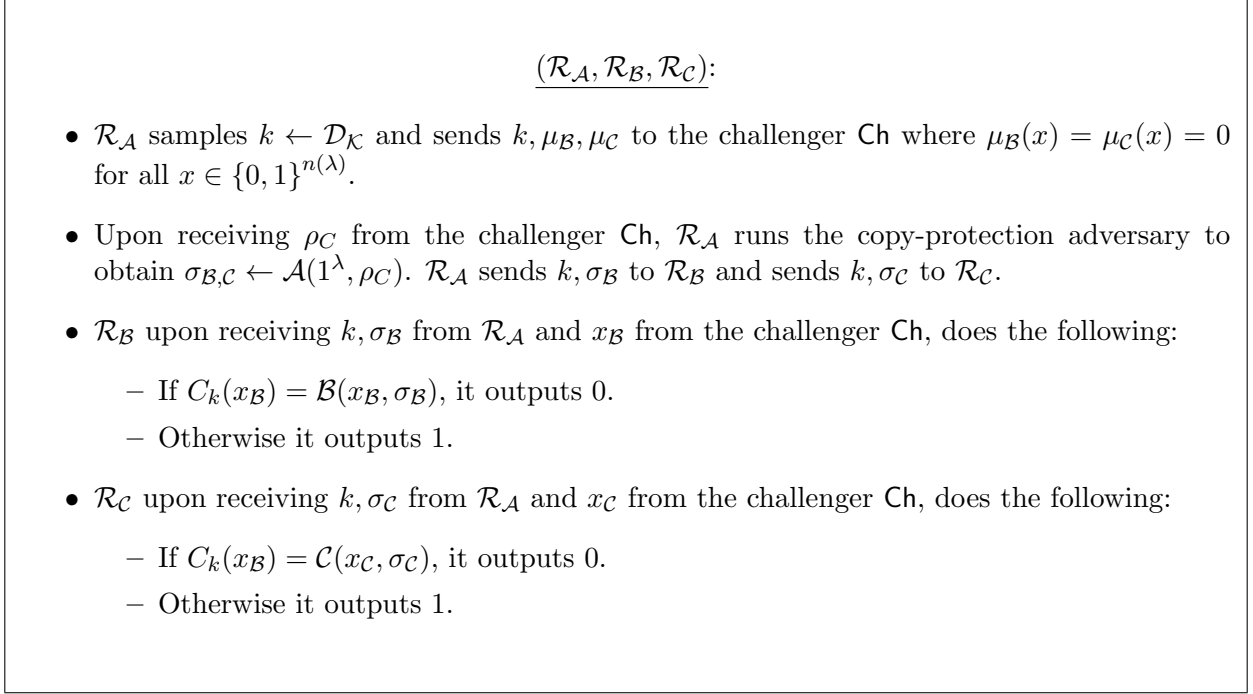


Figure 2: The UPO security adversary $(\mathcal{R}_\mathcal{A}, \mathcal{R}_\mathcal{B}, \mathcal{R}_\mathcal{C})$ from the prior work [AB24]

4.1 New Observations

Now we consider the following instantiation of copy-protection.

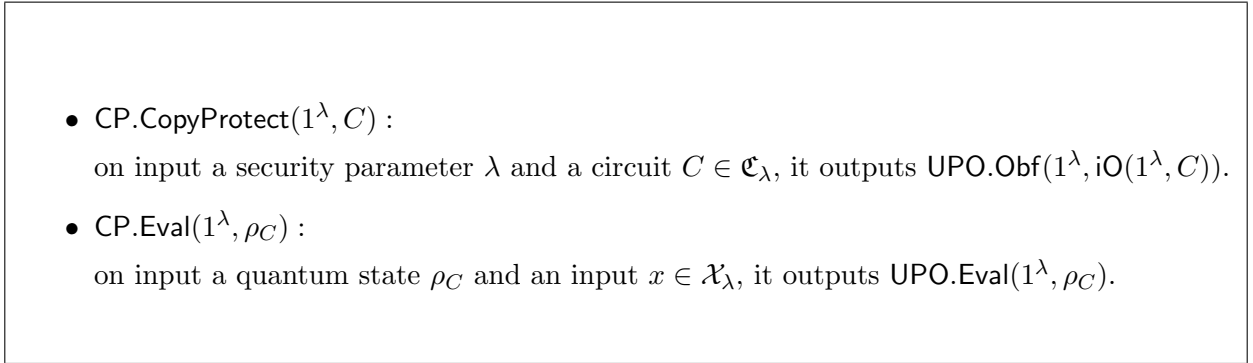


Figure 3: Copy-protection construction from iO and UPO

There are three crucial observations. We will present one of them, the Non-local Puncture Hiding property in Section 4.3 and the other two below.

Puncture-Hiding The puncture Hiding Lemma states that for all $y_B, y_C, y_B^*, y_C^* \in \{0, 1\}^{m(\lambda)}$,

$$\left| \mathbf{P}_{y_B^*, y_C^*} [y_B, y_C] - \mathbf{P}_{y_B^* \rightarrow y_B^*, y_C^* \rightarrow y_C^*} [y_B, y_C] \right| < \text{negl}(\lambda).$$

This lemma roughly says that if we puncture a point to its original value, it is indistinguishable from not puncturing at all. Since we have iO in the construction, this indistinguishability can be reduced to the iO indistinguishability.

Output-Hiding The output hiding Lemma states that if the circuit class satisfies generalized puncturing security with respect to some distribution $(\mathcal{D}_K, \mathcal{D}_{\mathcal{X}^2})$ then for all $y'_0, y'_1, y_B, y_C, y^* \in \{0, 1\}^m$, we have

$$\left| \mathbf{P}_{y'_0 \rightarrow y^*, y'_1 \rightarrow y^*} [y_B, y_C] - \mathbf{P}_{y'_1 \rightarrow y^*, y'_0 \rightarrow y^*} [y_B, y_C] \right| < \text{negl}(\lambda).$$

Intuitively, this means that if we puncture at point x then the original value of $C_k(x)$ is hidden computationally. The reason for this is that if the circuit class satisfies generalized puncturing security then it means that puncturing erases all information at a point computationally and thus we have this output-hiding lemma.

4.2 Copy-Protection from UPO

Suppose that $\mathcal{D}_{\mathcal{X}^2}$ is a diagonal distribution, meaning that we always have $x_B = x_C$. Consider the following reduction from a copy-protection adversary $\mathcal{A}, \mathcal{B}, \mathcal{C}$ to an UPO adversary $\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C$.

$(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$:

- $\mathcal{R}_A$ samples $k \leftarrow \mathcal{D}_K$ and sends k, μ_B, μ_C to the challenger Ch where $\mu_B(x) = \mu_C(x) = 0$ for all $x \in \{0, 1\}^{n(\lambda)}$.
- Upon receiving ρ_C from the challenger Ch, $\mathcal{R}_A$ runs the copy-protection adversary to obtain $\sigma_{B,C} \leftarrow \mathcal{A}(1^\lambda, \rho_C)$. $\mathcal{R}_A$ sends k, σ_B to $\mathcal{R}_B$ and sends k, σ_C to $\mathcal{R}_C$.
- $\mathcal{R}_B$ upon receiving k, σ_B from $\mathcal{R}_A$ and x_B from the challenger Ch, does the following:
 - If $C_k(x_B) = 0$, it outputs 0.
 - Otherwise it computes $y_B \leftarrow \mathcal{B}(\sigma_B, x_B)$ and outputs $1 - y_B$.
- $\mathcal{R}_C$ upon receiving k, σ_C from $\mathcal{R}_A$ and x_C from the challenger Ch, does the following:
 - If $C_k(x_C) = 0$, it outputs 0.
 - Otherwise it computes $y_C \leftarrow \mathcal{C}(\sigma_C, x_C)$ and outputs $1 - y_C$.

Figure 4: The UPO security adversary $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$

Now we analyze the winning probability of this adversary. With probability $\frac{1}{2}$ we have $C_k(x) = 0$ and the winning probability is exactly $\frac{1}{2}$. With the other $\frac{1}{2}$ probability we have $C_k(x) = 1$ and the

winning probability is

$$\begin{aligned}
& \frac{1}{2} (\mathbf{p}_{1,1} [1, 1] + \mathbf{p}_{1 \rightarrow 0, 1 \rightarrow 0} [0, 0]) \\
& \approx \frac{1}{2} (\mathbf{p}_{1,1} [1, 1] + \mathbf{p}_{0 \rightarrow 0, 0 \rightarrow 0} [0, 0]) \\
& \approx \frac{1}{2} (\mathbf{p}_{1,1} [1, 1] + \mathbf{p}_{0,0} [0, 0]) .
\end{aligned}$$

The first approximation is due to the output-hiding lemma and the second approximation is by the puncture-hiding lemma. The last line is exactly the success probability of the copy-protection adversary which is $\frac{1}{2} + \frac{1}{\text{poly}(\lambda)}$.

4.3 New Variant of UPO: Independent-Secure UPO

An Independent-Secure UPO captures the fact that any adversary cannot win the following game with probability significantly greater than the trivial probability. Different the original UPO security, in the following game, bits indicating whether inputs for both parts are punctured or not are sampled independently.

- $\mathcal{A}$ submits a keyed circuit C to the challenger. It also additionally submits a circuit μ ; the role of μ will be clear below.
- The challenger then samples the inputs $(x_{\mathcal{B}}, x_{\mathcal{C}})$ from a distribution $\mathcal{D}_{\mathcal{X}^2}$. It also samples two random bits $b_{\mathcal{B}}, b_{\mathcal{C}} \leftarrow \{0, 1\}$. Computes (a) $G_{k^*}^{(0,0)} = C_k$, (b) $G_{k^*}^{(0,1)} \leftarrow \text{GenPuncture}(k, x_{\mathcal{C}}, \mu_{\mathcal{C}})$, (c) $G_{k^*}^{(1,0)} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, \mu_{\mathcal{B}})$, (d) $G_{k^*}^{(1,1)} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, x_{\mathcal{C}}, \mu_{\mathcal{B}}, \mu_{\mathcal{C}})$. It sends ρ_C to $\mathcal{A}$ where ρ_C is the obfuscation of $G_{k^*}^{(b_{\mathcal{B}}, b_{\mathcal{C}})}$.
- $\mathcal{A}$ then computes a bipartite state and shares one part with $\mathcal{B}$ and the other part with $\mathcal{C}$.
- $\mathcal{B}$ then gets $x_{\mathcal{B}}$ and $\mathcal{C}$ gets $x_{\mathcal{C}}$. The output of $\mathcal{B}$ and $\mathcal{C}$ is $(b_{\mathcal{B}}, b_{\mathcal{C}})$. We say that $\mathcal{A}, \mathcal{B}, \mathcal{C}$ wins if $b_{\mathcal{B}} = b_{\mathcal{B}}$ and $b_{\mathcal{C}} = b_{\mathcal{C}}$.

Non-local Puncture Hiding Before stating the non-local puncture hiding lemma, we first define a property called puncture-hiding (not the same as the puncture hiding lemma) for a pair of distributions $\mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}}$. A circuit class is said to be puncture-hiding with respect to $\mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}}$ if any adversary cannot distinguish between an unpunctured circuit and a circuit that is punctured at a random point according to $\mathcal{D}_{\mathcal{X}}$. Now we first assume that in the copy-protection game, the challenge distribution $\mathcal{D}_{\mathcal{X}^2}$ can be decomposed into the direct product of its marginal distribution $\mathcal{D}_{\mathcal{X}^2} = \mathcal{D}_{\mathcal{X}}^{\mathcal{B}} \times \mathcal{D}_{\mathcal{X}}^{\mathcal{C}}$. The non-local puncture hiding lemma states that if the circuit class is puncture hiding with respect to $\mathcal{D}_{\mathcal{K}}$ and $\mathcal{D}_{\mathcal{X}}^{\mathcal{C}}$ then for any copy-protection adversary $\mathcal{A}, \mathcal{B}, \mathcal{C}$, Bob cannot distinguish between the case where Charlie's input is punctured ($b_{\mathcal{C}} = 1$) and the case where Charlie's input is not-punctured ($b_{\mathcal{C}} = 0$). All these holds vice versa. Intuitively, this is because if Bob can do so, then $\mathcal{A}, \mathcal{B}$ forms an adversary that breaks the puncture hiding property of the circuit class with respect to $\mathcal{D}_{\mathcal{K}}$ and $\mathcal{D}_{\mathcal{X}}^{\mathcal{C}}$.

4.4 Copy-Protection from Independent-Secure UPO

Given a unclonable puncturable obfuscation for P/poly satisfying independent-UPO security, we use the same construction of copy-protection as the one given in Figure 3 given in Section 4.1.

Proving Theorem 3.2 First, we sketch the proof of Theorem 3.2. We show that for the construction given in Figure 3, the maximum joint success probability for $\mathcal{B}$ and $\mathcal{C}$ in the anti-piracy game with respect to uniform challenge distribution is at most $\frac{1}{3} + \frac{4p_{\text{triv}}}{3}$ (hence the advantage is $\frac{1}{3} + \frac{p_{\text{triv}}}{3}$) up to negligible factors, for any non-local adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in the anti-piracy game with respect to some distribution $\mathcal{D}_K$ on the circuit keys and the uniform distribution on the challenge inputs.

Let $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ be any adversary in the copy-protection anti-piracy game with respect to distribution $\mathcal{D}_K$ on the circuit keys and the uniform distribution on the challenge inputs. We reduce the security of copy-protection to the security of UPO, by constructing a reduction adversary $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$, as defined in Figure 5. The reduction is the same as in [AB24].

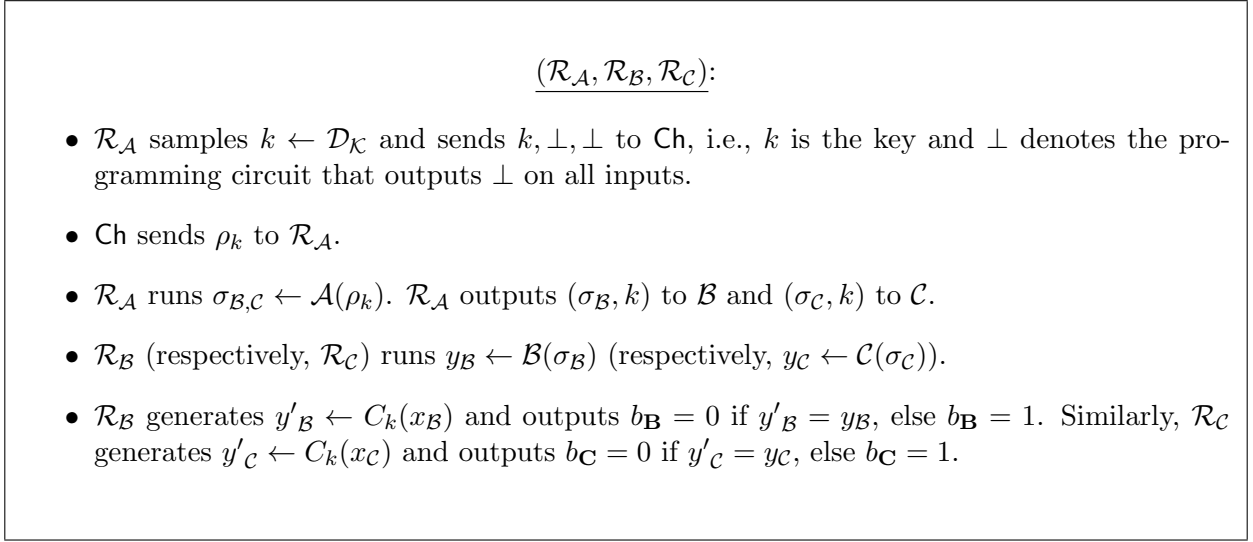


Figure 5: The UPO security adversary $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$

Define the following 4 games $\mathcal{G}_{00}, \mathcal{G}_{01}, \mathcal{G}_{10}, \mathcal{G}_{11}$ as follows: $\mathcal{G}_{00}$ is the copy-protection experiment where no input is punctured, in $\mathcal{G}_{01}$ Charlie's input is punctured, in $\mathcal{G}_{10}$ Bob's input is punctured and in $\mathcal{G}_{11}$ both inputs are punctured. Define the following:

- $q_{00}^{b_B b_C}$ to be the probability that $\mathcal{R}_B$ and $\mathcal{R}_C$ outputs b_B and b_C in $\mathcal{G}_{00}$.
- $q_{10}^{b_B b_C}$ to be the probability that $\mathcal{R}_B$ and $\mathcal{R}_C$ outputs b_B and b_C in $\mathcal{G}_{10}$.
- $q_{01}^{b_B b_C}$ to be the probability that $\mathcal{R}_B$ and $\mathcal{R}_C$ outputs b_B and b_C in $\mathcal{G}_{01}$.
- $q_{11}^{b_B b_C}$ to be the probability that $\mathcal{R}_B$ and $\mathcal{R}_C$ outputs b_B and b_C in $\mathcal{G}_{11}$.

The success probability of the adversary $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ in the independent UPO game can be written as:

$$\frac{q_{00}^{00}}{4} + \frac{q_{10}^{10}}{4} + \frac{q_{01}^{01}}{4} + \frac{q_{11}^{11}}{4}.$$

Hence, by the independent-UPO security of UPO',

$$(\star) \quad \frac{q_{00}^{00}}{4} + \frac{q_{10}^{10}}{4} + \frac{q_{01}^{01}}{4} + \frac{q_{11}^{11}}{4} \leq \frac{1}{2} + \text{negl.}$$

Consider the following lemmas. In the following lemmas and equations, when we say $\alpha \approx \beta$ then we mean that $|\alpha - \beta| \leq \text{negl}$, and $\alpha \leq_{\text{negl}} \beta$ to denote that α is at most $\beta + \text{negl}$, and $\alpha \geq_{\text{negl}} \beta$ to denote that α is at least $\beta - \text{negl}$, where negl is a place holder for a negligible function in λ .

Note that for any generalized puncturable keyed circuit class $\mathfrak{C}$, the fact that we generalized puncture a circuit at a uniformly random point, and then obfuscate it under $i\mathcal{O}$ and never reveal any information about the random point, then assuming the existence of length-doubling PRG and by standard $i\mathcal{O}$ arguments, we get that the obfuscated circuit is indistinguishable from the obfuscation of the unpunctured circuit. Therefore, since we start with an unclonable puncturable obfuscation $\text{UPO}' = \text{UPO}'.\text{Obf}, \text{UPO}', \text{Eval}$ where $\text{UPO}'.\text{Obf}(1^\lambda, C) = \text{UPO}(1^\lambda, i\mathcal{O}(1^\lambda, C))$ and $\text{UPO}'.\text{Eval} = \text{UPO}.\text{Eval}$ for some unclonable puncturable obfuscation scheme $\text{UPO} = (\text{UPO}.\text{Obf}, \text{UPO}.\text{Eval})$,⁵ then by the security of $i\mathcal{O}$, we get the following non-local puncture hiding lemma for $\mathcal{B}$, which roughly states that the statistics of $\mathcal{B}$'s output remains unchanged independent of whether or not the circuit was punctured at $\mathcal{C}$'s challenge input, see Section 4.3 for more discussion. The non-local puncture hiding lemmas are formally shown in Lemma 9.7.

Lemma 4.1. $(q_{00}^{00} + q_{00}^{10}) \approx (q_{10}^{10} + q_{10}^{00})$.

Similarly, changing the roles of $\mathcal{B}$ and $\mathcal{C}$ in the above argument, we get a non-local puncture hiding lemma for $\mathcal{C}$.

Lemma 4.2. $(q_{00}^{00} + q_{00}^{01}) \approx (q_{01}^{01} + q_{01}^{00})$.

Next, by the generalized puncturing security of $\mathfrak{C}$ equipped with the generalized puncturing algorithm GenPuncture , we know that if a circuit is punctured for $\mathcal{B}$, then both in the case when the circuit is punctured at $\mathcal{C}$'s point, and when it is not punctured at $\mathcal{C}$'s point, we conclude that $\mathcal{B}$ cannot guess the true value of the punctured point with probability much higher than the trivial guessing probability p_{triv} , which gives us the following bounds,

$$q_{11}^{00} + q_{11}^{01} \leq_{\text{negl}} p_{\text{triv}}, \quad q_{10}^{00} + q_{10}^{01} \leq_{\text{negl}} p_{\text{triv}}. \quad (1)$$

Similarly, by repeating the generalized puncturing security for $\mathcal{C}$, we conclude that

$$q_{11}^{00} + q_{11}^{10} \leq_{\text{negl}} p_{\text{triv}}, \quad q_{01}^{00} + q_{01}^{10} \leq_{\text{negl}} p_{\text{triv}}. \quad (2)$$

Combining Equation (1) and Equation (2), we get

Lemma 4.3. $q_{11}^{11} \geq 1 - (q_{11}^{00} + q_{11}^{10}) - (q_{11}^{00} + q_{11}^{01}) \geq_{\text{negl}} 1 - 2p_{\text{triv}}$.

Also by direct consequences of Equation (1), and Equation (2) respectively, we get

Lemma 4.4. $q_{10}^{00} \leq_{\text{negl}} p_{\text{triv}}, \quad q_{01}^{00} \leq_{\text{negl}} p_{\text{triv}}$.

Combining Lemma 4.4 with Lemmas 4.1 and 4.2, we conclude that

$$q_{10}^{10} \geq_{\text{negl}} (q_{00}^{00} + q_{00}^{10}) - p_{\text{triv}}, \quad q_{01}^{10} \geq_{\text{negl}} (q_{00}^{00} + q_{00}^{10}) - p_{\text{triv}}. \quad (3)$$

⁵Note that if UPO is any arbitrary unclonable puncturable obfuscation scheme then UPO' which is the composition of UPO and $i\mathcal{O}$ is also an unclonable puncturable obfuscation scheme by a composition result for independent-UPO security (see Corollary 7.5). The proof of this composition result is the same as the proof of the analogous composition result for generalized UPO security, shown in [AB24, Corollary 13].

Thus, by the UPO security, we have the following:

$$\begin{aligned}
& (\star) \frac{1}{2} + \text{negl} \\
& \geq \frac{q_{00}^{00}}{4} + \frac{q_{10}^{10}}{4} + \frac{q_{01}^{01}}{4} + \frac{q_{11}^{11}}{4} \\
& \geq_{\text{negl}} \frac{q_{00}^{00}}{4} + \frac{q_{00}^{00} + q_{00}^{10} - p_{\text{triv}}}{4} + \frac{q_{01}^{01} + q_{00}^{00} - p_{\text{triv}}}{4} + \frac{1 - 2p_{\text{triv}}}{4} \\
& \text{(By Equation (3), and Lemma 4.3)} \\
& \geq \frac{3q_{00}^{00} + q_{10}^{00} + q_{01}^{00}}{4} + \frac{1 - 4p_{\text{triv}}}{4},
\end{aligned}$$

which implies,

$$3q_{00}^{00} + q_{10}^{00} + q_{01}^{00} \leq 1 + 4p_{\text{triv}} + \text{negl}. \quad (4)$$

Hence, we conclude that

$$q_{00}^{00} \leq \frac{1}{3} + \frac{4p_{\text{triv}}}{3} + \text{negl}.$$

Proving Theorem 3.3 We next show that the construction given in Figure 3 also satisfies **one-out-of-two** copy-protection security with respect to distribution $\mathcal{D}_{\mathcal{K}}$ on the circuit keys and the uniform distribution on the challenge inputs, provided the trivial success probability p_{triv} (given by $\mathcal{D}_{\mathcal{K}}$) is negligible. Note that **one-out-of-two** copy-protection is a restriction of the one-out-of-many copy-protection definition of [KN23], where we consider exactly two freeloaders instead of arbitrary polynomially many freeloaders. Let $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ be any adversary in the **one-out-of-two** copy-protection anti-piracy game with respect to distribution $\mathcal{D}_{\mathcal{K}}$ on the circuit keys and the uniform distribution on the challenge inputs. We need to show that the sum of the individual success probability of $\mathcal{B}$ and $\mathcal{C}$ is at most $1 + p_{\text{triv}}$ up to negligible factors. Since p_{triv} is negligible, it is enough to show that the sum of the individual success probabilities is at most $1 + \text{negl}$.

Note that, $2q_{00}^{00} + q_{10}^{00} + q_{01}^{00} = (q_{00}^{00} + q_{10}^{00}) + (q_{00}^{00} + q_{01}^{00})$ is precisely the sum of the individual success probabilities of $\mathcal{B}$ and $\mathcal{C}$ in the copy-protection anti-piracy game. By the same argument as in the (informal) proof of Theorem 3.2, we deduce Equation (4), which says,

$$3q_{00}^{00} + q_{10}^{00} + q_{01}^{00} \leq 1 + 4p_{\text{triv}} + \text{negl}.$$

Hence, from Equation (4), we get the following bound,

$$2q_{00}^{00} + q_{10}^{00} + q_{01}^{00} \leq 1 + 4p_{\text{triv}} + \text{negl}, \quad (5)$$

which is at most negligible since p_{triv} is negligible. Hence, we conclude if the trivial success probability p_{triv} associated with $\mathcal{D}_{\mathcal{K}}$ is negligible, then the above construction also satisfies **one-out-of-two** copy-protection security with respect to $\mathcal{D}_{\mathcal{K}}$ and the uniform challenge distribution.

4.5 Independent-Secure UPO: Construction

4.5.1 Copy protection for generalized puncturable signature

En route to constructing UPO with independent-UPO security, we consider a new notion of copy protection for signatures in the context of (generalized) puncturable signatures, i.e., signatures where the verification key can be punctured in a generalized way. A copy-protection scheme for generalized puncturable signature is a tuple of QPT algorithms, $(\text{KeyGen}, \text{CopyProtect}, \text{Sign}, \text{GenPuncture}, \text{Verify})$:

- Key-generation, **KeyGen**: samples a secret signing key and public verification key pair (sk, vk) .
- Copy-protection, **CopyProtect**: takes a secret key sk and outputs a signing program ρ_{sk} .
- Signing, **Sign**: takes a signing program ρ_{sk} , a message $x \in \{0, 1\}^n$, and outputs a signature $sig \in \{0, 1\}^m$.
- Verification, **Verify**: takes a verification key (both punctured and unpunctured), and a message signature pair (x, sig) and outputs a bit to denote accept or reject. If the verification key is punctured, then **Verify** may output values other than $\{0, 1\}$.
- Generalized puncturing, **GenPuncture**: takes a secret key sk , a pair of inputs $x_1, x_2 \in \{0, 1\}^n$, and two programming circuits μ_1, μ_2 and outputs a punctured verification key $vk_{x_1 \rightarrow \mu_1(x_1), x_2 \rightarrow \mu_2(x_2)}$.

The scheme must satisfy the signing correctness, i.e., sampling a key pair (sk, vk) from the key-generation algorithm, then signing any message x using a signing program ρ_{sk} generated by copy-protecting sk , to get a signature sig , and then running verification on vk, x, sig , must accept. Additionally, it must satisfy puncturing correctness, i.e., a punctured key $vk_{x_1 \rightarrow \mu_1(x_1), x_2 \rightarrow \mu_2(x_2)}$ derived by running **GenPuncture** on some secret key sk , inputs x_1, x_2 and circuits μ_1, μ_2 where $sk, vk \leftarrow \text{KeyGen}$, must have satisfy that the $\text{Verify}(vk_{x_1 \rightarrow \mu_1(x_1), x_2 \rightarrow \mu_2(x_2)}, \cdot, \cdot)$ has the same functionality as $\text{Verify}(vk, \cdot, \cdot)$ on all inputs except x_1, x_2 , and on x_1 it should indicate successful verification by outputting $\mu_1(x_1)$ and if $x = x_2$ (and $x_2 \neq x_1$), then it should indicate successful verification by $\mu_2(x_2)$.

Informally, for any cloning adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$, the security game with respect to a distribution $\mathcal{D}_K$ on circuit keys, and a distribution $\mathcal{D}_{\mathcal{X}^2}$ on the pair of challenge inputs, looks as given below. The security holds if $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ win with probability at most negligibly more than $\frac{1}{2}$.

- $\mathcal{A}$ sends circuits μ_B, μ_C to the challenger Ch.
- Ch samples $(sk, vk) \leftarrow \text{KeyGen}$, generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and then samples the inputs (x_B, x_C) from $\mathcal{D}_{\mathcal{X}^2}$. It also samples two random bits $b_B, b_C \leftarrow \{0, 1\}$. Computes (a) $vk_{k^*}^{(0,0)} = vk$, (b) $vk_{k^*}^{(0,1)} \leftarrow \text{GenPuncture}(sk, x_C, \mu_C)$, (c) $vk_{k^*}^{(1,0)} \leftarrow \text{GenPuncture}(sk, x_B, \mu_B)$, (d) $vk_{k^*}^{(1,1)} \leftarrow \text{GenPuncture}(sk, x_B, x_C, \mu_B, \mu_C)$. It sends $\rho_{sk}, vk_{k^*}^{b_B, b_C}$ to $\mathcal{A}$.
- $\mathcal{A}$ then computes a bipartite state and shares one part with $\mathcal{B}$ and the other part with $\mathcal{C}$.
- $\mathcal{B}$ then gets x_B and $\mathcal{C}$ gets x_C . The output of $\mathcal{B}$ and $\mathcal{C}$ is (b_B, b_C) . We say that $\mathcal{A}, \mathcal{B}, \mathcal{C}$ wins if $b_B = b_B$ and $b_C = b_C$.

Next, we show that given any copy-protection scheme for generalized puncturable signatures scheme CP.GPS satisfying $\mathcal{D}_{\mathcal{X}^2}$, we can construct a UPO scheme satisfying $\mathcal{D}_{\mathcal{X}^2}$ -independent-UPO security, as follows.

UPO construction. The obfuscation algorithm, on input any D , does the following:

1. Sample $sk, vk \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
2. Generate $\rho_{sk} \leftarrow \text{CP.GPS.CopyProtect}(sk)$.
3. Generate $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,
 - It first runs $\text{Verify}(vk, x, sig)$. If the verification does not pass, output $\perp$.
 - Otherwise, it computes $D(x)$.

The evaluation algorithm first computes the signature $\sigma \leftarrow \text{CP.GPS.Sign}(\rho_{sk}, x)$, and then runs $\hat{C}(x, \sigma)$ and then measures the outcome to get a string y , and outputs it.

4.5.2 Security Proof

Let $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ be an adversary in the generalized UPO security game for the construction mentioned above. Let $\mathfrak{C}_\lambda = \{W_k\}_{k \in \mathcal{K}_\lambda}$, and $\mathfrak{C} = \{\mathfrak{C}_\lambda\}$ be the circuit class, equipped with the generalized puncturing algorithm GenPuncture . In the first step of the UPO independent-UPO security game, $\mathcal{A}$ sends k, μ_B, μ_C to the challenger Ch.

Next we analyze each of the four cases corresponding to the four choices of the bits b_B, b_C sampled by the challenger Ch.

Unpunctured ($b_B = 0, b_C = 0$). In this case, the adversary $\mathcal{A}$ receives as input $\text{UPO.Obf}(W_k)$, and $\mathcal{B}$ and $\mathcal{C}$ receive x_B and x_C respectively in the challenge round.

Note that $\text{UPO.Obf}(W_k)$ looks like this:

- $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,
 - It first runs $\text{Verify}(vk, x, sig)$. If the verification does not pass, output $\perp$.
 - Otherwise, it computes $W_k(x)$.
- It computes copy-protection of $\sigma_{sk} \leftarrow \text{CopyProtect}(sk, \cdot)$ where sk is sampled from the keygen algorithm of the copy-protection for (generalized) puncturable signatures scheme.
- Outputs $(\hat{C}, \sigma_{sk})$.

Consider the following hybrids. We mark the changes across consecutive hybrids in [blue](#).

H₁: the above experiment.

H₂: we generate $\hat{C}$ differently. We generate $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,

- It first runs $\text{Verify}(vk, x, sig)$. If the verification does not pass, output $\perp$.
- If the Verify algorithm outputs y_B^* , then compute $z \leftarrow \mu_B(x)$, and output z , and if the Verify algorithm outputs y_C^* , then compute $z \leftarrow \mu_C(x)$, and output z .
- Otherwise, it computes $W_k(x)$.

Since Verify algorithm never outputs $\perp$, adding the check if Verify algorithm outputs y_B^* or y_C^* did not change the functionality of C , and hence by the security of $i\mathcal{O}$, indistinguishability between H₁ and H₂ holds.

Punctured for both $\mathcal{B}$ and $\mathcal{C}$ ($b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 1$) In this case, the adversary receives as input $\text{UPO.Obf}(G_{k^*}^{11})$, where $G_{k^*}^{11} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, x_{\mathcal{C}}, \mu_{\mathcal{B}}, \mu_{\mathcal{C}})$. In this case, the adversary $\mathcal{A}$ receives as input $\text{UPO.Obf}(G_{k^*}^{11})$, and $\mathcal{B}$ and $\mathcal{C}$ receive $x_{\mathcal{B}}$ and $x_{\mathcal{C}}$ respectively in the challenge round.

Note that $\text{UPO.Obf}(G_{k^*}^{11})$ looks like this. We mark the changes from the original experiment in **red**.

- $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,
 - It first runs $\text{Verify}(vk, x, sig)$. If the verification does not pass, output $\perp$.
 - Otherwise, it computes $G_{k^*}^{11}(x)$. **Note that if $x \notin \{x_{\mathcal{B}}, x_{\mathcal{C}}\}$ $G_{k^*}^{11}(x) = W_k(x)$, and $G_{k^*}^{11}(x_{\mathcal{B}}) = \mu_{\mathcal{B}}(x_{\mathcal{B}})$ and $G_{k^*}^{11}(x_{\mathcal{C}}) = \mu_{\mathcal{C}}(x_{\mathcal{C}})$.**
- It computes copy-protection of $\sigma_{sk} \leftarrow \text{CopyProtect}(sk, \cdot)$.
- Outputs $(\hat{C}, \sigma_{sk})$.

Consider the following hybrids.

H_1 : the above experiment.

H_2 : we generate $\hat{C}$ differently. We generate $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,

- It first runs $\text{Verify}(vk_{x_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, x_{\mathcal{C}} \rightarrow y_{\mathcal{C}}^*}, x, sig)$, **where $vk_{x_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, x_{\mathcal{C}} \rightarrow y_{\mathcal{C}}^*} \leftarrow \text{GenPuncture}(x_{\mathcal{B}}, x_{\mathcal{C}}, 1_{y_{\mathcal{B}}^*}, 1_{y_{\mathcal{C}}^*})$.**⁶
If the verification does not pass, output $\perp$.
- **If the Verify algorithm outputs $y_{\mathcal{B}}^*$, then compute $z \leftarrow \mu_{\mathcal{B}}(x)$, and output z , and if the Verify algorithm outputs $y_{\mathcal{C}}^*$, then compute $z \leftarrow \mu_{\mathcal{C}}(x)$, and output z .**
- Otherwise, it computes $G_{k^*}^{11}(x)$.

Note that in H_1 , on input $x_{\mathcal{B}}$ and $x_{\mathcal{C}}$, the output of the circuit $G_{k^*}^{11}$ and hence that of C is $\mu_{\mathcal{B}}(x_{\mathcal{B}})$ and $\mu_{\mathcal{C}}(x_{\mathcal{C}})$. Moreover, by the correctness of GenPuncture , the generalized puncturing algorithm associated with the underlying copy-protection scheme for (generalized) puncturable signatures, $\text{Verify}(vk_{x_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, x_{\mathcal{C}} \rightarrow y_{\mathcal{C}}^*}, x, sig)$ outputs $y_{\mathcal{B}}^*$ or $y_{\mathcal{C}}^*$ only when $x = x_{\mathcal{B}}$ or $x = x_{\mathcal{C}}$ respectively, and the signature sig is valid for x . Hence, adding an explicit instruction to output $\mu_{\mathcal{B}}(x_{\mathcal{B}})$ when Verify outputs $y_{\mathcal{B}}^*$ and to output $\mu_{\mathcal{C}}(x_{\mathcal{C}})$ when Verify outputs $y_{\mathcal{C}}^*$, does not change the functionality of C . Therefore the functionality of C did not change across H_1 and H_2 , and hence by security of $i\mathcal{O}$, H_1 and H_2 are computationally indistinguishable.

H_3 : we generate $\hat{C}$ differently. We generate $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,

- It first runs $\text{Verify}(vk_{x_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, x_{\mathcal{C}} \rightarrow y_{\mathcal{C}}^*}, x, sig)$, where $vk_{x_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, x_{\mathcal{C}} \rightarrow y_{\mathcal{C}}^*} \leftarrow \text{GenPuncture}(sk, x_{\mathcal{B}}, x_{\mathcal{C}}, 1_{y_{\mathcal{B}}^*}, 1_{y_{\mathcal{C}}^*})$.
If the verification does not pass, output $\perp$.
- If the Verify algorithm outputs $y_{\mathcal{B}}^*$, then compute $z \leftarrow \mu_{\mathcal{B}}(x)$, and output z , and if the Verify algorithm outputs $y_{\mathcal{C}}^*$, then compute $z \leftarrow \mu_{\mathcal{C}}(x)$, and output z .

⁶Here GenPuncture refers to the puncturing algorithm of the copy protection of signature scheme.

- Otherwise, it computes $W_k(x)$.

Since anyway in H_2 , on input x_B and x_C , the output of the circuit is $\mu_B(x_B)$ and $\mu_C(x_C)$, hardcoding f instead of $G_{k^*}^{11}$ did not change the functionality of C . Hence, the functionality of $\hat{C}$ did not change across H_2 and H_3 . Therefore, by security of $i\mathcal{O}$ H_2 and H_3 are computationally indistinguishable.

Punctured only for $\mathcal{B}$ ($b_B = 1, b_C = 0$) In this case, the adversary receives as input $\text{UPO.Obf}(G_{k^*}^{10})$, where $G_{k^*}^{10} \leftarrow \text{GenPuncture}(k, x_B, \mu_B)$. In this case, the adversary $\mathcal{A}$ receives as input $\text{UPO.Obf}(G_{k^*}^{10})$, and $\mathcal{B}$ and $\mathcal{C}$ receive x_B and x_C respectively in the challenge round.

Note that $\text{UPO.Obf}(G_{k^*}^{11})$ looks like this. We mark the changes from the original experiment in the unpunctured case in **red**.

- $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,
 - It first runs $\text{Verify}(vk, x, sig)$. If the verification does not pass, output $\perp$.
 - Otherwise, it computes $G_{k^*}^{10}(x)$. **Note that if $x \neq x_B$ $G_{k^*}^{10}(x) = W_k(x)$, and $G_{k^*}^{10}(x_B) = \mu_B(x_B)$.**
- It computes copy-protection of $\sigma_{sk} \leftarrow \text{CopyProtect}(sk, \cdot)$.
- Outputs $(\hat{C}, \sigma_{sk})$.

Consider the following hybrids.

H_1 : the above experiment.

H_2 : we generate $\hat{C}$ differently. We generate $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,

- It first runs $\text{Verify}(vk_{x_B \rightarrow y_B^*}, x, sig)$, **where $vk_{x_B \rightarrow y_B^*} \leftarrow \text{GenPuncture}(sk, x_B, 1_{y_B^*})$** . If the verification does not pass, output $\perp$.
- **If the Verify algorithm outputs y_B^* , then compute $z \leftarrow \mu_B(x)$, and output z .**
- Otherwise, it computes $G_{k^*}^{10}(x)$.

The indistinguishability holds by the same arguments as in the proof of indistinguishability of H_1 and H_2 in the both punctured case ($b_B = b_C = 1$).

H_3 : we generate $\hat{C}$ differently. We generate $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,

- It first runs $\text{Verify}(vk_{x_B \rightarrow y_B^*}, x, sig)$, where $vk_{x_B \rightarrow y_B^*} \leftarrow \text{GenPuncture}(sk, x_B, 1_{y_B^*})$. If the verification does not pass, output $\perp$.
- If the Verify algorithm outputs y_B^* , then compute $z \leftarrow \mu_B(x)$, and output z .
- Otherwise, it computes $W_k(x)$.

The indistinguishability holds by the same arguments as in the proof of indistinguishability of H_2 and H_3 in the both punctured case ($b_B = b_C = 1$).

Punctured only for $\mathcal{C}$ ($b_{\mathcal{C}} = 1, b_{\mathcal{B}} = 0$) In this case, the adversary receives as input $\text{UPO.Obf}(G_{k^*}^{01})$, where $G_{k^*}^{01} \leftarrow \text{GenPuncture}(k, x_{\mathcal{C}}, \mu_{\mathcal{C}})$. In this case, the adversary $\mathcal{A}$ receives as input $\text{UPO.Obf}(G_{k^*}^{01})$, and $\mathcal{C}$ and $\mathcal{C}$ receive $x_{\mathcal{C}}$ and $x_{\mathcal{C}}$ respectively in the challenge round.

Note that $\text{UPO.Obf}(G_{k^*}^{11})$ looks like this. We mark the changes from the original experiment in **red**.

- $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,
 - It first runs $\text{Verify}(vk, x, sig)$. If the verification does not pass, output $\perp$.
 - Otherwise, it computes $G_{k^*}^{01}(x)$. **Note that if $x \neq x_{\mathcal{C}}$ $G_{k^*}^{01}(x) = W_k(x)$, and $G_{k^*}^{01}(x_{\mathcal{C}}) = \mu_{\mathcal{C}}(x_{\mathcal{C}})$.**
- It computes copy-protection of $\sigma_{sk} \leftarrow \text{CopyProtect}(sk, \cdot)$.
- Outputs $(\hat{C}, \sigma_{sk})$.

Consider the following hybrids.

H_1 : the above experiment.

H_2 : we generate $\hat{C}$ differently. We generate $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,

- It first runs $\text{Verify}(vk_{x_{\mathcal{C}} \rightarrow y_{\mathcal{C}}^*}, x, sig)$, **where $vk_{x_{\mathcal{C}} \rightarrow y_{\mathcal{C}}^*} \leftarrow \text{GenPuncture}(sk, x_{\mathcal{C}}, 1_{y_{\mathcal{C}}^*})$** . If the verification does not pass, output $\perp$.
- **If the Verify algorithm outputs $y_{\mathcal{C}}^*$, then compute $z \leftarrow \mu_{\mathcal{C}}(x)$, and output z .**
- Otherwise, it computes $G_{k^*}^{01}(x)$.

The indistinguishability holds by the same arguments as in the proof of indistinguishability of H_1 and H_2 in the both punctured case ($b_{\mathcal{B}} = b_{\mathcal{C}} = 1$).

H_3 : we generate $\hat{C}$ differently. We generate $\hat{C} \leftarrow i\mathcal{O}(C)$, where C does the following: it takes as input (x, σ) ,

- It first runs $\text{Verify}(vk_{x_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*}, x, sig)$, where $vk_{x_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*} \leftarrow \text{GenPuncture}(sk, x_{\mathcal{B}}, 1_{y_{\mathcal{B}}^*})$. If the verification does not pass, output $\perp$.
- If the Verify algorithm outputs $y_{\mathcal{B}}^*$, then compute $z \leftarrow \mu_{\mathcal{B}}(x)$, and output z .
- Otherwise, it computes $W_k(x)$.

The indistinguishability holds by the same arguments as in the proof of indistinguishability of H_2 and H_3 in the both punctured case ($b_{\mathcal{B}} = b_{\mathcal{C}} = 1$).

Combining the final hybrids from each of the four cases, corresponding to the four possible values of $\mathcal{B}$ and $\mathcal{C}$ we get the following H^* , which is indistinguishable from the original independent-UPO security game.

H^* :

1. Ch samples $b_{\mathcal{B}} \xleftarrow{\$} \{0, 1\}$, and $b_{\mathcal{C}} \xleftarrow{\$} \{0, 1\}$.

2. Ch generates the copy-protected state $\sigma_{sk} \leftarrow \text{CopyProtect}(sk, \cdot)$ where sk is sampled from the keygen algorithm of the copy-protection for (generalized) puncturable signatures scheme, and also generates the circuit $\hat{C} \leftarrow i\mathcal{O}(C)$, and sends $(\hat{C}, \rho_{sk})$ to $\mathcal{A}$, where C is defined as follows:

- It first runs $\text{Verify}(\widetilde{vk}, x, sig)$, where if $b_B = b_C = 0$, $\widetilde{vk} = vk$ (the unpunctured verification key), if $b_B = 1, b_C = 1$ $\widetilde{vk} = vk_{x_B \rightarrow y_B^*, x_C \rightarrow y_C^*} \leftarrow \text{GenPuncture}(sk, x_B, x_C, 1_{y_B^*}, 1_{y_C^*})$, if $b_B = 1, b_C = 0$, $\widetilde{vk} = vk_{x_B \rightarrow y_B^*} \leftarrow \text{GenPuncture}(sk, x_B, 1_{y_B^*})$, and if $b_C = 1, b_B = 0$, $\widetilde{vk} = vk_{x_C \rightarrow y_C^*} \leftarrow \text{GenPuncture}(sk, x_C, 1_{y_C^*})$.
If the verification does not pass, output $\perp$.
- If the Verify algorithm outputs y_B^* , then compute $z \leftarrow \mu_B(x)$, and output z .
- Otherwise, it computes $W_k(x)$.

Note that H^* can be easily reduced to anti-piracy game for the underlying copy-protection of (generalized) puncturable signature scheme: Given an adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in the independent-UPO security game, we can construct $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ in the anti-piracy of the underlying copy-protection of (generalized) puncturable signature scheme:

1. $\mathcal{R}_A$ runs $\mathcal{A}(1^\lambda)$ to get k, μ_B, μ_C , and sends $1_{y_B^*}, 1_{y_C^*}$ as the programmed circuits to the copy-protection challenger Ch.
2. Ch sends back a state ρ_{sk} and a verification key $\widetilde{vk}$.
3. $\mathcal{R}_A$ uses k, μ_B, μ_C and $\widetilde{vk}$ to construct C as described in H^* and then generates $\hat{C} \leftarrow i\mathcal{O}(C)$, and runs $\mathcal{A}$ on $(\rho_{sk}, \hat{C})$ to get $\sigma_{B,C}$ which $\mathcal{R}_A$ outputs.
4. $\mathcal{R}_B$ and $\mathcal{R}_C$ does the same as $\mathcal{B}$ and $\mathcal{C}$ respectively.

4.5.3 Instantiating copy-protection for generalized puncturable signature

Now that we know how to build independent-secure UPO from copy-protection for generalized puncturable signature, we instantiate from well-studied assumptions, copy-protection for generalized puncturable signature with respect to the uniform challenge distribution, i.e., uniform distribution for the challenge pairs of inputs⁷.

Our construction (formally given in Section 12.2.1) can be viewed as a (generalized) puncturable extension of the construction of copy-protection of signatures proposed in [LLQZ22] and uses similar assumptions as that in [LLQZ22]. Conceptually, our construction of copy-protection for generalized puncturable signature is also similar to the UPO construction in [AB24] as both the constructions have essentially two circuits (obfuscated under $i\mathcal{O}$), an evaluation circuit for a PRF which requires the unclonable quantum state for evaluation, and then a verification circuit that checks the authenticity of the PRF evaluation. The main difference is that in [AB24] the verification circuit is not independent of the unclonable quantum state, whereas in our construction, the verification circuit is independent of the unclonable state, thereby getting a cleaner construction.

⁷Note that the applications of independent-secure UPO for copy-protection (see Section 4.4) also require the uniform distribution for the challenge pairs of inputs, and hence, this instantiation of copy-protection for generalized puncturable signature, along with the instantiation of independent-secure UPO from copy-protection for generalized puncturable signature, discussed in the first part of this section completes the instantiation of these applications from well-studied assumptions.

The proof techniques are based on the techniques used in [AB24, LLQZ22]. However, we had to generalize the core technical lemma used in [LLQZ22, Lemma C.7] involving hidden-triggers [SW21], which maybe of independent interest. The generalized lemma is formally proven in Lemma 12.9 in the Section 12.3.

5 Preliminaries

We use the following notation throughout the paper.

- We use the abbreviation QPT for quantum polynomial time.
- If two distributions $\mathcal{D}_0$ and $\mathcal{D}_1$ are computationally indistinguishable then we denote them by $\mathcal{D}_0 \approx_c \mathcal{D}_1$. Similarly, if they are statistically indistinguishable then we denote them by $\mathcal{D}_0 \approx_s \mathcal{D}_1$.
- We use $\text{negl}(\cdot)$ to denote a negligible function.
- For two probabilities p_0 and p_1 , we say $p_0 \sim p_1$ if $|p_0 - p_1| \leq \text{negl}(\lambda)$.
- We denote λ to be the security parameter.
- We will use $n(\lambda)$ as the input length for circuit classes, with respect to the security parameter λ for most scenarios unless mentioned otherwise. Similarly, we will denote the input space for the circuit class by $\mathcal{X}$ and use the notation $\mathcal{D}_{\mathcal{X}}$ and $\mathcal{D}_{\mathcal{X}^2}$ to denote distributions on inputs and pairs of inputs, respectively. Hence, in most cases, $\mathcal{X}$ would be equal to $\{0, 1\}^{n(\lambda)}$, and $\mathcal{D}_{\mathcal{X}}(1^\lambda)$ and $\mathcal{D}_{\mathcal{X}^2}(1^\lambda)$ would be distributions on $\{0, 1\}^{n(\lambda)}$ and $\{0, 1\}^{n(\lambda)} \times \{0, 1\}^{n(\lambda)}$ respectively, unless specified otherwise.

5.1 Keyed Circuit Class

We copy the background on keyed circuit class verbatim from [AB24].

A class of classical circuits of the form $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ is said to be a keyed circuit class if the following holds: $\mathfrak{C}_\lambda = \{C_k : k \in \mathcal{K}_\lambda\}$, where C_k is a (classical) circuit with input length $n(\lambda)$, output length $m(\lambda)$ and $\mathcal{K} = \{\mathcal{K}_\lambda\}_{\lambda \in \mathbb{N}}$ is the key space. We refer to C_k as a keyed circuit. We note that any circuit class can be represented as a keyed circuit class using universal circuits. We will be interested in the setting when C_k is a polynomial-sized circuit; henceforth, unless specified otherwise, all keyed circuit classes considered in this work will consist only of polynomial-sized circuits. We will also make a simplifying assumption that C_k and $C_{k'}$ have the same size, where $k, k' \in \mathcal{K}_\lambda$.

Puncturable Keyed Circuit Class. Consider a keyed circuit class $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$. We say that $\mathfrak{C}_\lambda$ is said to be puncturable if there exists a deterministic polynomial time puncturing algorithm **Puncture** such that the following holds: on input $k \in \{0, 1\}^\lambda$, strings $x_1 \in \{0, 1\}^{n(\lambda)}$, $x_2 \in \{0, 1\}^{n(\lambda)}$, it outputs a circuit C_{k^*} . Moreover, the following holds: for every $x \in \{0, 1\}^{n(\lambda)}$,

$$C_{k^*}(x) = \begin{cases} C_k(x), & x \neq x_1, x \neq x_2, \\ \perp, & x \in \{x_1, x_2\}. \end{cases}$$

Without loss of generality, we can assume that the size of C_{k^*} is the same as the size of C_k .

Generalized Puncturable Keyed Circuit Class. The notion of puncturing can be generalized further. In the new generalized definition, we allow the adversary to choose the outputs of the circuit generated by **Puncture** on the punctured points. Previously, the circuit generated by the puncturing algorithm was such that on the punctured points, it output $\perp$, but now we add programmability, i.e., instead of outputting $\perp$, we allow the punctured circuits to output special values at the punctured points. Formally, the (generalized) puncturing algorithm **GenPuncture** now takes as input $k \in \mathcal{K}_\lambda$, polynomial-sized circuits $\mu_1 : \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)} \cup \{\perp\}$, $\mu_2 : \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)} \cup \{\perp\}$, strings $x_1 \in \{0, 1\}^{n(\lambda)}$, $x_2 \in \{0, 1\}^{n(\lambda)}$, if $x_1 \neq x_2$, it outputs a circuit G_{k^*} such that for every $x \in \{0, 1\}^{n(\lambda)}$,

$$G_{k^*}(x) = \begin{cases} C_k(x), & x \neq x_1, x \neq x_2 \\ \mu_1(x_1), & x = x_1 \\ \mu_2(x_2), & x = x_2, \end{cases}$$

else it outputs a circuit G_{k^*} such that for every $x \in \{0, 1\}^{n(\lambda)}$,

$$G_{k^*}(x) = \begin{cases} C_k(x), & x \neq x_1 \\ \mu_1(x_1), & x = x_1. \end{cases}$$

We will refer to μ_1, μ_2 as the programmed circuits. Moreover, we use the notation that if $x_1 = x_2 = x$ and $\mu_1 = \mu_2 = \mu$, then we will write $\text{GenPuncture}(k, x, \mu)$ to represent $\text{GenPuncture}(k, x_1, x_2, \mu_1, \mu_2)$. Moreover, when clear from the context, for succinctness we will abuse the notation and use the notation $\perp$ to also denote a programmed circuit that always outputs bot or $\perp$.

A keyed circuit class $\mathfrak{C}$ associated with a generalized puncturing algorithm **GenPuncture** is referred to as a *generalized puncturable keyed circuit class*. Note that for every keyed circuit class $\mathfrak{C} = \{C_k\}_k$, there exists a trivial **GenPuncture** algorithm, which on any input $k, x_1, x_2, \mu_1, \mu_2$, constructs the circuit C_k and then outputs a circuit G_{k^*} ⁸, defined as follows: on input x , G_{k^*} checks if $x = x_i$ for any $i \in \{0, 1\}$. If so, it outputs $\mu_i(x_i)$. Else if $x \notin \{x_1, x_2\}$, it outputs $C_k(x)$.

Keyed Circuit Implementation. A function class $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ is said to have a keyed circuit implementation $\mathfrak{C} = \{\{C_k\}_{k \in \mathcal{K}_\lambda}\}_{\lambda \in \mathbb{N}}$ if for every $f \in \mathcal{F}$, there is a circuit C_k in $\mathfrak{C}$ that implements f . Formally, we can consider a family of maps $\{S_\lambda : \mathfrak{C}_\lambda \rightarrow \mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ defined as follows: for every $k \in \mathcal{K}_\lambda$, C_k is an implementation of the function $S_\lambda(C_k)$. Moreover, S_λ is surjective. In addition, if there exists a distribution $\mathcal{D}_\mathcal{F}$ on $\mathcal{F}$ (i.e. $\mathcal{D}_\mathcal{F}(1^\lambda)$ samples a function from $\mathcal{F}_\lambda$), and an efficiently samplable distribution $\mathcal{D}_\mathcal{K}$ on $\mathcal{K}$ (i.e., $\mathcal{D}_\mathcal{K}(1^\lambda)$ samples a key from $\mathcal{K}_\lambda$) such that

$$\{S_\lambda(C_k)\}_{k \leftarrow \mathcal{D}_\mathcal{K}(1^\lambda)} \approx_s \{f\}_{f \leftarrow \mathcal{D}_\mathcal{F}(1^\lambda)},$$

then $(\mathcal{D}_\mathcal{K}, \mathfrak{C})$ is called a keyed circuit implementation of $(\mathcal{D}_\mathcal{F}, \mathcal{F})$. Since any circuit class can be represented as a keyed circuit class using universal circuits, there is no loss of generality in our definition of keyed circuit implementation.

6 Generalized Puncturing Security

⁸As before, the output circuit G_{k^*} may not have the same size as C_k , but this can be resolved by sufficient padding of the circuit class.

Definition 6.1 ($((n, m, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2})$ -Puncturing Security). Consider a puncturable keyed circuit class $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$, where $\mathfrak{C}_\lambda = \{C_k\}_{k \in \mathcal{K}_\lambda}$, where each circuit in the family $\mathfrak{C}_\lambda$ has input length $n(\lambda)$ and output length $m(\lambda)$ such that $m(\lambda)$ and $n(\lambda)$ are polynomials in λ . Let **Puncture** be the puncturing algorithm associated with $\mathfrak{C}$. We say that $(\mathfrak{C}, \mathbf{Puncture})$ satisfies $(\mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2})$ -puncturing security, where $\mathcal{D}_{\mathcal{K}}$ is a distribution over $\mathcal{K} = \{\mathcal{K}_\lambda\}_{\lambda \in \mathbb{N}}$ and $\mathcal{D}_{\mathcal{X}^2}$ is a distribution over $\{0, 1\}^{n(\lambda)} \times \{0, 1\}^{n(\lambda)}$, if the following holds: for any QPT adversary Adv in Puncturing.Expt defined in Figure 6, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[1 \leftarrow \text{Puncturing.Expt}^{\text{Adv}, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda) \right] \leq p_{\text{triv}}(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}) + \text{negl}(\lambda),$$

where

$$p_{\text{triv}}(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}) = \max \{ p_{\text{triv}}^1(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}), p_{\text{triv}}^2(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}), \},$$

and for every $i \in [2]$, $p_{\text{triv}}^i(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}) := \sup_{\mathcal{B}} \left\{ \Pr \left[y = C_k(x_i) : \begin{array}{l} k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda) \\ (x_1, x_2) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda) \\ y \leftarrow \mathcal{B}(1^\lambda, x_i) \end{array} \right] \right\}$, and the supremum is over all QPT algorithms $\mathcal{B}$.

$\text{Puncturing.Expt}^{\text{Adv}, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda)$:

- Ch samples $k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda)$, $(x_0, x_1) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda)$ and generates $C_{k^*} \leftarrow \mathbf{Puncture}(k, x_0, x_1)$.
- Ch sends C_{k^*} to the adversary Adv .
- Adv upon receiving C_{k^*} sends a bit b to the challenger Ch.
- Ch sends x_b to Adv .
- Adv upon receiving C_{k^*} , sends y_b to Ch.
- Output 1 if $C_k(x_b) = y_b$.

Figure 6: Puncturing Security Experiment

Definition 6.2 (Generalized Puncturing Security). We say that a generalized keyed circuit class $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ equipped with a generalized puncturing algorithm GenPuncture , satisfies **generalized puncturing security** with respect to two distributions, $\mathcal{D}_{\mathcal{K}}$ on $\mathcal{K}_\lambda$ and $\mathcal{D}_{\mathcal{X}^2}$ on $\{0, 1\}^{n(\lambda)} \times \{0, 1\}^{n(\lambda)}$ if the following holds: for every QPT adversary Adv in $\text{GenPuncturing.Expt}$ (defined in Figure 7) there exists a negligible function $\text{negl}(\cdot)$ such that

$$\Pr \left[1 \leftarrow \text{GenPuncturing.Expt}^{\text{Adv}, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda) \right] \leq p_{\text{triv}}(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}) + \text{negl}(\lambda),$$

where $p_{\text{triv}}(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2})$ is as defined in Definition 6.1.

$\text{GenPuncturing.Expt}^{\text{Adv}, \mathcal{D}_K, \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda)$:

- Adv sends two circuits (μ_0, μ_1) , where μ_0, μ_1 mapping $\{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$ are circuit that the adversary wishes to use on the punctured points, to the challenger Ch.
- Ch samples $k \leftarrow \mathcal{D}_K(1^\lambda), (x_0, x_1) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda)$ and generates $G_{k^*} \leftarrow \text{GenPuncture}(k, x_0, x_1, \mu_0, \mu_1)$.
- Ch sends G_{k^*} to the adversary Adv.
- Adv upon receiving G_{k^*} sends a bit b to the challenger Ch.
- Ch sends x_b to Adv.
- Adv upon receiving G_{k^*} , sends y_b to Ch.
- Output 1 if $C_k(x_b) = y_b$.

Figure 7: Generalized Puncturing Security Experiment

Remark 6.3 (Equivalence of puncturing security (Definition 6.1) and generalized puncturing security (Definition 6.2)). *We claim that puncturing security can be generically upgraded to generalized puncturing security. Suppose $(\mathfrak{C}, \text{Puncture})$ satisfies puncturing security (with respect to the distributions $(\mathcal{D}_K, \mathcal{D}_{\mathcal{X}^2})$). Then we define a canonical algorithm GenPuncture such that $(\mathfrak{C}, \text{GenPuncture})$ satisfies generalized puncturing security with respect to the same distributions.*

The description of GenPuncture is as follows: on input $(k, x_1, x_2, \mu_1, \mu_2)$, GenPuncture first computes $C_{k^} \leftarrow \text{Puncture}(k, x_1, x_2)$ and then outputs the circuit G_{k^*} , where G_{k^*} on an input x , outputs $C_{k^*}(x)$ if $x \notin \{x_1, x_2\}$, otherwise it outputs $\mu_i(x_i)$ if $x = x_i$. From the description of GenPuncture , it is clear that the puncturing security of $(\mathfrak{C}, \text{Puncture})$ implies the generalized puncturing security of $(\mathfrak{C}, \text{GenPuncture})$.*

7 Unclonable Puncturable Obfuscation (UPO)

We recall the definition of unclonable puncturable obfuscation verbatim from [AB24].

Syntax. An unclonable puncturable obfuscation (UPO) scheme $(\text{Obf}, \text{Eval})$ for a keyed circuit class $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$, consists of the following QPT algorithms:

- $\text{Obf}(1^\lambda, C)$: on input a security parameter λ and a keyed circuit $C \in \mathfrak{C}_\lambda$ with input length $n(\lambda)$, it outputs a quantum state ρ_C .
- $\text{Eval}(\rho_C, x)$: on input a quantum state ρ_C and an input $x \in \{0, 1\}^{n(\lambda)}$, it outputs (ρ'_C, y) .

Correctness. An unclonable puncturable obfuscation scheme $(\text{Obf}, \text{Eval})$ for a keyed circuit class $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ is δ -correct, if for every $C \in \mathfrak{C}_\lambda$ with input length $n(\lambda)$, and for every $x \in \{0, 1\}^{n(\lambda)}$,

$$\Pr \left[C(x) = y \mid \begin{array}{l} \rho_C \leftarrow \text{Obf}(1^\lambda, C) \\ (\rho'_C, y) \leftarrow \text{Eval}(\rho_C, x) \end{array} \right] \geq \delta$$

If δ is negligibly close to 1 then we say that the scheme is correct (i.e., we omit mentioning δ).

7.1 Generalized Security

Definition 7.1 (Generalized UPO security). *We say that a pair of QPT algorithms $(\text{Obf}, \text{Eval})$ for a generalized keyed circuit class $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ equipped with a puncturing algorithm GenPuncture , satisfies **generalized UPO security** with respect to a distribution $\mathcal{D}_{\mathcal{X}^2}$ on $\{0, 1\}^{n(\lambda)} \times \{0, 1\}^{n(\lambda)}$ if the following holds for every QPT $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in GenUPO.Expt (defined in Figure 8):*

$$\Pr \left[1 \leftarrow \text{GenUPO.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda, b) : b \xleftarrow{\$} \{0, 1\} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

$\text{GenUPO.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda, b):$

- $\mathcal{A}$ sends $(k, \mu_{\mathcal{B}}, \mu_{\mathcal{C}})$, where $k \in \mathcal{K}_\lambda, \mu_{\mathcal{B}} : \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}, \mu_{\mathcal{C}} : \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$, to the challenger Ch .
- Ch samples $(x_{\mathcal{B}}, x_{\mathcal{C}}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda)$ and generates $G_{k^*} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, x_{\mathcal{C}}, \mu_{\mathcal{B}}, \mu_{\mathcal{C}})$.
- Ch generates ρ_b as follows:
 - $\rho_0 \leftarrow \text{Obf}(1^\lambda, C_k)$,
 - $\rho_1 \leftarrow \text{Obf}(1^\lambda, G_{k^*})$

It sends ρ_b to $\mathcal{A}$.

- Apply $(\mathcal{B}(x_{\mathcal{B}}, \cdot) \otimes \mathcal{C}(x_{\mathcal{C}}, \cdot))(\sigma_{\mathcal{B}, \mathcal{C}})$ to obtain $(b_{\mathcal{B}}, b_{\mathcal{C}})$.
- Output 1 if $b = b_{\mathcal{B}} = b_{\mathcal{C}}$.

Figure 8: Generalized UPO Security Experiment

We recall the following composition result from [AB24]. Roughly speaking, the composition result states that applying an indistinguishability obfuscation compiler to a circuit first before applying UPO preserves the security of UPO. At first, this might seem straightforward. However, there is a subtlety: it is possible that the compiler destroys the puncturing property. The result below shows that this subtlety can be circumvented.

Corollary 7.2 (Restatement of [AB24, Corollary 13]). *Consider a generalized puncturable keyed circuit class $\mathfrak{C}$. Suppose $i\mathcal{O}$ be an indistinguishability obfuscation scheme for $\mathfrak{C}$. Suppose UPO is an unclonable puncturable obfuscation scheme for $\mathfrak{G}$ (as defined above) satisfying $\mathcal{D}_{\mathcal{X}^2}$ -generalized UPO security. Then UPO' is a secure unclonable puncturable obfuscation scheme for $\mathfrak{C}$ satisfying $\mathcal{D}_{\mathcal{X}^2}$ -generalized UPO security where UPO' is defined as follows:*

- $\text{UPO}'.\text{Obf}(1^\lambda, C) = \text{UPO}.\text{Obf}(1^\lambda, i\mathcal{O}(1^\lambda, C))$ for every circuit C .
- $\text{UPO}'.\text{Eval} = \text{UPO}.\text{Eval}$.

Thus we conclude that assuming UPO is an unclonable puncturable obfuscation scheme that satisfies $\mathcal{D}_{\mathcal{X}^2}$ -independent-UPO security for any $\mathcal{D}_{\mathcal{X}^2}$ -generalized puncturable keyed circuit class in P/poly , then the same holds for the unclonable puncturable obfuscation scheme $\text{UPO}' = (\text{UPO}'.\text{Obf}, \text{UPO}'.\text{Eval})$.

7.2 Independent-secure UPO

In this work, we extend the definition of UPO security proposed in [AB24], to an independent version as defined below.

Definition 7.3 (Independent Challenge UPO security). *We say that a pair of QPT algorithms $(\text{Obf}, \text{Eval})$ for a generalized keyed circuit class $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ equipped with a generalized puncturing algorithm GenPuncture , satisfies $\mathcal{D}_{\mathcal{X}^2}$ -independent-UPO security for any distribution $\mathcal{D}_{\mathcal{X}^2}$ on $\{0, 1\}^{n(\lambda)} \times \{0, 1\}^{n(\lambda)}$ where $\{0, 1\}^{n(\lambda)}$ is the input space, if the following holds for every QPT (A, B, C) in $\text{GenUPO}.\text{Expt}$ defined in Figure 8:*

$$\Pr \left[1 \leftarrow \text{IndUPO}.\text{Expt}^{(A, B, C), \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda, b_B, b_C) : \begin{matrix} b_B \xleftarrow{\$} \{0, 1\} \\ b_C \xleftarrow{\$} \{0, 1\} \end{matrix} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

7.2.1 Composition Theorem

We note that the composition theorem proven in [AB24, Theorem 12] and its corollary that we recalled in Corollary 7.2 for UPO with generalized unclonable puncturable obfuscation security, also extends to UPO with independent-UPO security. The proof of the composition theorem in the case of independent-UPO security, is the same as that in the case of generalized unclonable puncturable obfuscation security, up to minor adaptations.

Let Compile be a functionality-preserving circuit compiler, i.e., Compile is a probabilistic algorithm that takes as input a security parameter λ , classical circuit C and outputs another classical circuit $\tilde{C}$ such that C and $\tilde{C}$ have the same functionality. For instance, program obfuscation [BGI⁺01] is an example of a circuit compiler.

Let $\mathfrak{C}$ be a generalized puncturable keyed circuit class associated with keyspace $\mathcal{K}$ defined as follows: $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$, where every circuit in $\mathfrak{C}_\lambda$ is of the form C_k , where $k \in \mathcal{K}_\lambda$, with input length $n(\lambda)$ and the output length $m(\lambda)$. We denote GenPuncture to be a generalized puncturing algorithm associated with $\mathfrak{C}$.

Let $\text{UPO} = (\text{UPO}.\text{Obf}, \text{UPO}.\text{Eval})$ be an unclonable puncturable obfuscation scheme for every generalized puncturable keyed circuit class in P/poly .

$\text{IndUPO.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda, b_{\mathcal{B}}, b_{\mathcal{C}}):$

- $\mathcal{A}$ sends $(k, \mu_{\mathcal{B}}, \mu_{\mathcal{C}})$, where $k \in \mathcal{K}_\lambda, \mu_{\mathcal{B}} : \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}, \mu_{\mathcal{C}} : \{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$, to the challenger Ch .
- Ch samples $(x_{\mathcal{B}}, x_{\mathcal{C}}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda)$ and generates the following: (a) $G_{k^*}^{(0,0)} = C_k$, (b) $G_{k^*}^{(0,1)} \leftarrow \text{GenPuncture}(k, x_{\mathcal{C}}, \mu_{\mathcal{C}})$, (c) $G_{k^*}^{(1,0)} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, \mu_{\mathcal{B}})$, (d) $G_{k^*}^{(1,1)} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, x_{\mathcal{C}}, \mu_{\mathcal{B}}, \mu_{\mathcal{C}})$.
- Ch generates $\rho_{(b_0, b_1)}$, for every $b_0, b_1 \in \{0, 1\}$, as follows: $\rho_{(b_0, b_1)} \leftarrow \text{Obf}(1^\lambda, G_{k^*}^{(b_0, b_1)})$. It sends $\rho_{(b_{\mathcal{B}}, b_{\mathcal{C}})}$ to $\mathcal{A}$.
- $\mathcal{A}$ on input $\rho_{(b_{\mathcal{B}}, b_{\mathcal{C}})}$ outputs a bipartite state $\sigma_{\mathcal{B}, \mathcal{C}}$.
- Apply $(\mathcal{B}(x_{\mathcal{B}}, \cdot) \otimes \mathcal{C}(x_{\mathcal{C}}, \cdot))(\sigma_{\mathcal{B}, \mathcal{C}})$ to obtain $(b_{\mathcal{B}}, b_{\mathcal{C}})$.
- Output 1 if $b_{\mathcal{B}} = b_{\mathbf{B}}$ and $b_{\mathcal{C}} = b_{\mathbf{C}}$.

Figure 9: Independent Challenge Security

Given the circuit class $\mathfrak{C}$, we define a generalized puncturable keyed circuit class $\mathfrak{G}$ $\mathfrak{G} = \{\mathfrak{G}_\lambda\}_{\lambda \in \mathbb{N}}$, where every circuit in $\mathfrak{G}_\lambda$ is of the form $G_{k||r}(\cdot)$, with input length $n(\lambda)$, output length $m(\lambda)$, $k \in \mathcal{K}_\lambda$, and $r \in \{0, 1\}^{t(\lambda)}$. Here, $t(\lambda)$ denotes the number of bits of randomness consumed by $\text{Compile}(1^\lambda, C_k; \cdot)$. Moreover, the circuit $G_{k||r}$ takes as input $x \in \{0, 1\}^n$, applies $\text{Compile}(1^\lambda, C_k; r)$ to obtain $\widetilde{C}_k$ and then it outputs $\widetilde{C}_k(x)$. The puncturing algorithm associated with $\mathfrak{G}$ is $\text{GenPuncture}'$ which on input $k||r$ and the set of inputs x_1, x_2 and circuits μ_1, μ_2 , generates $W_{k^*} \leftarrow \text{GenPuncture}(k, x_1, x_2, \mu_1, \mu_2)$, and then outputs the circuit $G_{k^*, r}$, where $G_{k^*, r}$ is defined as follows: it takes as input $x \in \{0, 1\}^n$, applies $\text{Compile}(1^\lambda, W_{k^*}; r)$ to obtain $\widetilde{W}_{k^*}$ and then it outputs $\widetilde{W}_{k^*}$. The keyspace associated with $\mathfrak{G}$ is $\mathcal{K}' = \{\mathcal{K}'_\lambda\}_{\lambda \in \mathbb{N}}$, where $\mathcal{K}'_\lambda = \mathcal{K}_\lambda \times \{0, 1\}^{t(\lambda)}$.

We define $\text{UPO}' = (\text{UPO}'.\text{Obf}, \text{UPO}'.\text{Eval})$ for every generalized puncturable keyed circuit class as follows:

- $\text{UPO}'.\text{Obf}(1^\lambda, C)$: output $\rho \leftarrow \text{UPO}.\text{Obf}(1^\lambda, \widetilde{C})$, where $\widetilde{C} \leftarrow \text{Compile}(1^\lambda, C)$.
- $\text{UPO}'.\text{Eval}(\rho, x)$: on input a quantum state ρ , string x , output $y \leftarrow \text{UPO}.\text{Eval}(\rho, x)$.

Theorem 7.4 (Composition theorem). *Let Compile be a functionality-preserving circuit compiler. Let $\text{UPO} = (\text{UPO}.\text{Obf}, \text{UPO}.\text{Eval})$ be an unclonable puncturable obfuscation scheme that satisfies $\mathcal{D}_{\mathcal{X}^2}$ -independent-UPO security for any generalized puncturable keyed circuit classes in P/poly , then the same holds for the unclonable puncturable obfuscation scheme $\text{UPO}' = (\text{UPO}'.\text{Obf}, \text{UPO}'.\text{Eval})$ defined above on Page 28.*

Proof. Let $\mathfrak{C}$ be any arbitrary generalized puncturable keyed circuit class in $\mathsf{P/poly}$, with the notation mentioned above. Let $\mathfrak{G}$ be the corresponding generalized puncturable keyed circuit class in $\mathsf{P/poly}$ derived from $\mathfrak{C}$ and `Compile` as mentioned above on Page 28. Suppose, there is an adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ that violates the $\mathcal{D}_{\chi^2}$ -independent-UPO security of UPO' for $\mathfrak{C}$ with probability p . We construct a QPT reduction $(\mathcal{R}_{\mathcal{A}}, \mathcal{R}_{\mathcal{B}}, \mathcal{R}_{\mathcal{C}})$ that violates the $\mathcal{D}_{\chi^2}$ -independent-UPO security of UPO for $\mathfrak{G}$, also with probability p . From the $\mathcal{D}_{\chi^2}$ -independent-UPO security of UPO' it then follows that p is at most $\frac{1}{2} + \varepsilon$, for some negligible function ε , which proves the theorem. The reduction is the same as the one used in the proof of [AB24, Proposition 13].

$\mathcal{R}_{\mathcal{A}}(1^\lambda)$ first runs $\mathcal{A}(1^\lambda)$ to obtain $k \in \mathcal{K}_\lambda$. It then samples $r \xleftarrow{\$} \{0, 1\}^{t(\lambda)}$. Then, $\mathcal{R}_{\mathcal{A}}$ forwards $k||r$ to the external challenger of UPO . Then, $\mathcal{R}_{\mathcal{A}}$ receives ρ^* which it then duly forwards to $\mathcal{A}$. Similarly, even in the challenge phase, $\mathcal{R}_{\mathcal{B}}$ (resp., $\mathcal{R}_{\mathcal{C}}$) forwards the challenge from the challenger to $\mathcal{B}$ (resp., $\mathcal{C}$).

It can be seen that the probability that $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ breaks the security of UPO' is the same as the probability that $(\mathcal{R}_{\mathcal{A}}, \mathcal{R}_{\mathcal{B}}, \mathcal{R}_{\mathcal{C}})$ breaks the security of UPO . Since $\mathfrak{C}$ is an arbitrary generalized puncturable keyed circuit class in $\mathsf{P/poly}$, by the $\mathcal{D}_{\chi^2}$ -independent-UPO security of UPO for every generalized puncturable keyed circuit class in $\mathsf{P/poly}$, we conclude that the same holds for UPO' . $\square$

Instantiating `Compile` with an indistinguishability obfuscation $i\mathcal{O}$ in theorem 7.4, the following corollary is immediate.

Corollary 7.5. *Consider a generalized puncturable keyed circuit class $\mathfrak{C}$. Suppose $i\mathcal{O}$ be an indistinguishability obfuscation scheme for $\mathfrak{C}$. Let $\text{UPO} = (\text{UPO}.\text{Obf}, \text{UPO}.\text{Eval})$ be an unclonable puncturable obfuscation scheme that satisfies $\mathcal{D}_{\chi^2}$ -independent-UPO security for any generalized puncturable keyed circuit classes in $\mathsf{P/poly}$, then the same holds for the unclonable puncturable obfuscation scheme UPO' , where UPO' is defined as follows:*

- $\text{UPO}'.\text{Obf}(1^\lambda, C)$: output $\rho \leftarrow \text{UPO}.\text{Obf}(1^\lambda, \tilde{C})$, where $\tilde{C} \leftarrow i\mathcal{O}(1^\lambda, C)$.
- $\text{UPO}'.\text{Eval}(\rho, x)$: on input a quantum state ρ , string x , output $y \leftarrow \text{UPO}.\text{Eval}(\rho, x)$.

In the corollary above, we assume that the indistinguishability scheme does not have an explicit evaluation algorithm. In other words, the obfuscation algorithm on input a circuit C outputs another circuit $\tilde{C}$ that is functionally equivalent to C . This is without loss of generality since we can combine any indistinguishability obfuscation scheme (that has an evaluation algorithm) with universal circuits to obtain an obfuscation scheme with the desired format.

Notation We use the convention that whenever we use an unclonable puncturable obfuscation scheme satisfying some form of UPO security for any application to copy-protections, we mean an unclonable puncturable obfuscation scheme for $\mathsf{P/poly}$ satisfying the respective form of UPO security.

8 Copy-Protection

Consider a function class $\mathcal{F}$ with keyed circuit implementation $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$, where $\mathcal{F}_\lambda$ (respectively, $\mathfrak{C}_\lambda$) consists of functions (respectively, circuits) with input length $n(\lambda)$ and output length $m(\lambda)$.

A copy-protection scheme for $\mathcal{F}$ is a pair of QPT algorithms $(\text{CP.CopyProtect}, \text{CP.Eval})$ defined as follows:

- $\text{CP.CopyProtect}(1^\lambda, C)$: on input a security parameter λ and a circuit $C \in \mathfrak{C}_\lambda$, it outputs a quantum state ρ_C .
- $\text{CP.Eval}(\rho_C, x)$: on input a quantum state ρ_C and an input $x \in \mathcal{X} = \{0, 1\}^{n(\lambda)}$, it outputs (ρ'_C, y) .

Correctness. A copy-protection scheme $(\text{CP.CopyProtect}, \text{CP.Eval})$ for a function class $\mathcal{F}$ with keyed circuit implementation $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ is δ -correct, if for every $C \in \mathfrak{C}_\lambda$, for every $x \in \{0, 1\}^{n(\lambda)}$,

$$\Pr \left[C(x) = y \mid \begin{array}{l} \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, C) \\ (\rho'_C, y) \leftarrow \text{CP.Eval}(\rho_C, x) \end{array} \right] \geq \delta(\lambda).$$

If δ is negligibly close to 1 then we say that the scheme is correct (i.e., we omit mentioning δ).

$\text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_\mathcal{K}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda)$:

- Ch samples $k \leftarrow \mathcal{D}_\mathcal{K}(1^\lambda)$ and generates $\rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, C_k)$. It sends ρ_C to $\mathcal{A}$.
- $\mathcal{A}$ produces a bipartite state $\sigma_{\mathcal{B}, \mathcal{C}}$.
- Ch samples $(x_\mathcal{B}, x_\mathcal{C}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda)$.
- Apply $(\mathcal{B}(x_\mathcal{B}, \cdot) \otimes \mathcal{C}(x_\mathcal{C}, \cdot))(\sigma_{\mathcal{B}, \mathcal{C}})$ to obtain $(y_\mathcal{B}, y_\mathcal{C})$.
- Output (b, c) where $b = 1$ if and only if $y_\mathcal{B} = C(x_\mathcal{B})$ and $c = 1$ if and only if $y_\mathcal{C} = C(x_\mathcal{C})$.

Figure 10: $(\mathcal{D}_\mathcal{K}, \mathcal{D}_{\mathcal{X}^2})$ experiment of copy-protection.

Suppose $\mathcal{D}_{\mathcal{X}^2}$ is a distribution on $\{0, 1\}^{n(\lambda)} \times \{0, 1\}^{n(\lambda)}$, and $\mathcal{D}_\mathcal{F}$ is a distribution on $\mathcal{F}$.

$(\mathcal{D}_\mathcal{F}, \mathcal{D}_{\mathcal{X}^2})$ -anti-piracy.

Definition 8.1. Consider the experiment in Figure 10. We define $p_{\text{triv}} = \max\{p_{\text{triv}_\mathcal{B}}, p_{\text{triv}_\mathcal{C}}\}$, where $p_{\text{triv}_\mathcal{B}}$ is defined as, $p_{\text{triv}_\mathcal{B}}(\lambda, \mathcal{D}_\mathcal{K}, \mathcal{D}_{\mathcal{X}^2}) := \sup_{\mathcal{B}} \left\{ \Pr \left[y = C_k(x_i) : \begin{array}{l} k \leftarrow \mathcal{D}_\mathcal{K}(1^\lambda) \\ (x_\mathcal{B}, x_\mathcal{C}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda) \\ y \leftarrow \mathcal{B}(1^\lambda, x_\mathcal{B}) \end{array} \right] \right\}$, and the supremum is over all QPT algorithms $\mathcal{B}$.

We define $p_{\text{triv}_\mathcal{C}}$ symmetrically.⁹

For any $\epsilon = \epsilon(\lambda)$, we say that a copy-protection scheme $(\text{CP.CopyProtect}, \text{CP.Eval})$ for $\mathcal{F}$ satisfies ϵ -($\mathcal{D}_\mathcal{F}, \mathcal{D}_{\mathcal{X}^2}$)-anti-piracy if there exists a keyed circuit implementation of the form $(\mathcal{D}_\mathcal{K}, \mathfrak{C})$ for

⁹Clearly, the definition of p_{triv} is the same as p_{triv} defined in Definitions 6.1 and 6.2.

$(\mathcal{D}_{\mathcal{F}}, \mathcal{F})$ such that for every tuple of QPT adversaries $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ there exists a negligible function $\text{negl}(\lambda)$ such that:

$$\Pr \left[(1, 1) \leftarrow \text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda) \right] \leq p_{\text{triv}}(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}) + \epsilon(\lambda) + \text{negl}(\lambda).$$

Definition 8.2. We say that a copy-protection scheme $(\text{CP.CopyProtect}, \text{CP.Eval})$ for $\mathcal{F}$ satisfies $(\mathcal{D}_{\mathcal{F}}, \mathcal{D}_{\mathcal{X}^2})$ -one-out-of-two-anti-piracy if there exists a keyed circuit implementation of the form $(\mathcal{D}_{\mathcal{K}}, \mathcal{C})$ for $(\mathcal{D}_{\mathcal{F}}, \mathcal{F})$ such that for every tuple of QPT adversaries $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ there exists a negligible function $\text{negl}(\lambda)$ such that:

$$\begin{aligned} & \Pr \left[(1, \star) \leftarrow \text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda) \right] + \Pr \left[(\star, 1) \leftarrow \text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda) \right] \\ & \leq 1 + p_{\text{triv}} + \text{negl}(\lambda). \end{aligned}$$

Remark 8.3. We note that Definition 8.2 is a special case of the one-out-of-many copy-protection security defined in [KN23, Definition 5.4]. In particular, we restrict the definition of one-out-of-many copy-protection security defined in [KN23, Definition 5.4], to have exactly two freeloaders instead of arbitrary polynomially many freeloaders.

8.1 Copy-Protection for Generalized Puncturable Signatures

We consider copy-protection schemes for signing functionalities. [LLQZ22] presented the first feasibility result on copy-protecting signatures. In this work, we consider a new definition of copy-protecting signatures that is incomparable with [LLQZ22, Definition 4.1]. In this definition, we consider signature schemes in which the verification key can be punctured on two messages x_1 and x_2 along with corresponding outputs $\mu_1(x_1)$ and $\mu_2(x_2)$. The correctness property states that the verification algorithm on input any valid signature on x_1 (resp., x_2 if $x_1 \neq x_2$) using this punctured verification key outputs $\mu(x_1)$ (resp., $\mu_2(x_2)$ if $x_1 \neq x_2$). To implement this, we add an additional algorithm called **GenPuncture** to the syntax¹⁰ as [LLQZ22, Definition 4.1]. Moreover, we use a different notion of anti-piracy based on generalized puncturing.

Definition 8.4 (Adapted from [LLQZ22, Definition 4.1]). A copy-protection scheme for generalized puncturable signature is a tuple of QPT algorithms, $(\text{KeyGen}, \text{CopyProtect}, \text{Sign}, \text{GenPuncture}, \text{Verify})$:

1. $(sk, vk) \leftarrow \text{KeyGen}(1^\lambda)$: Takes as input the security parameter and outputs a secret key sk and a public verification key vk .
2. $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$: Takes as input a secret key sk , and outputs a quantum state ρ_{sk} as a signing program.
3. $\sigma \leftarrow \text{Sign}(\rho_{sk}, x)$: Takes as input a signing program ρ_{sk} , a message x and outputs a classical signature σ .
4. $vk_{x_1 \rightarrow \mu_1(x_1), x_2 \rightarrow \mu_2(x_2)} \leftarrow \text{GenPuncture}(sk, x_1, x_2, \mu_1, \mu_2)$: Takes as input a secret key sk , a pair of messages x_1, x_2 , and a pair of circuits μ_1, μ_2 and outputs a generalized punctured key $vk_{x_1 \rightarrow \mu_1(x_1), x_2 \rightarrow \mu_2(x_2)}$.
5. $b \leftarrow \text{Verify}(vk, x, \sigma)$: Takes as input a verification key vk , a message m , a signature σ , and accepts (i.e., outputs 1) or rejects (i.e., outputs 0).

¹⁰We slightly change the syntax, as we use the algorithm **CopyProtect** instead of **QKeygen**.

Correctness: A copy-protection scheme for a generalized puncturable signature scheme (KeyGen , CopyProtect , Sign , Verify) is $\delta(\lambda)$ -correct if

1. For any message x ,

$$\Pr \left[b = 1 \mid \begin{array}{l} (sk, vk) \leftarrow \text{KeyGen}(1^\lambda, C) \\ (\rho_{sk} \leftarrow \text{CopyProtect}(sk) \\ sig \leftarrow \text{Sign}(\rho_{sk}, x) \\ b \leftarrow \text{Verify}(vk, x, sig) \end{array} \right] \geq \delta(\lambda).$$

If δ is negligibly close to 1, then we say that the scheme is correct (i.e., we omit mentioning δ).

2. For any pair of messages x_1, x_2 and circuits μ_1, μ_2 , if $(sk, vk) \leftarrow \text{KeyGen}(1^\lambda)$, and $vk_{x_1 \rightarrow \mu_1(x_1), x_2 \rightarrow \mu_2(x_2)} \leftarrow \text{GenPuncture}(sk, x_1, x_2, \mu_1, \mu_2)$, then it must hold that if $x_1 = x_2$:

$$\text{Verify}(vk_{x_1 \rightarrow \mu_1(x_1), x_2 \rightarrow \mu_2(x_2)}, x, \sigma) = \begin{cases} \text{Verify}(vk, x, \sigma) & \text{if } x \neq x_1 \text{ and } x \neq x_2 \\ \mu_1(x_1) & \text{if } x = x_1 \text{ and } \text{Verify}(vk, x, \sigma) = 1 \\ 0 & \text{if } x = x_1 \text{ and } \text{Verify}(vk, x, \sigma) = 0 \end{cases},$$

and if $x_1 \neq x_2$,

$$\text{Verify}(vk_{x_1 \rightarrow \mu_1(x_1), x_2 \rightarrow \mu_2(x_2)}, x, \sigma) = \begin{cases} \text{Verify}(vk, x, \sigma) & \text{if } x \neq x_1 \text{ and } x \neq x_2 \\ \mu_1(x_1) & \text{if } x = x_1 \text{ and } \text{Verify}(vk, x, \sigma) = 1 \\ 0 & \text{if } x = x_1 \text{ and } \text{Verify}(vk, x, \sigma) = 0 \\ \mu_2(x_2) & \text{if } x = x_2 \text{ and } \text{Verify}(vk, x, \sigma) = 1 \\ 0 & \text{if } x = x_2 \text{ and } \text{Verify}(vk, x, \sigma) = 0 \end{cases}.$$

$\text{CPSign.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda, b_{\mathcal{B}}, b_{\mathcal{C}})$:

- $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
- Ch does the following: it runs $\text{KeyGen}(1^\lambda)$ to obtain (sk, vk) . It generates $\rho_{\mathcal{C}} \leftarrow \text{CP.CopyProtect}(1^\lambda, \text{Sign}(\text{sk}, \cdot))$.
- Ch samples $(x_{\mathcal{B}}, x_{\mathcal{C}}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda)$.
- $\mathcal{A}$ sends $\rho_{\mathcal{C}}$ to $\mathcal{A}$, along with the the following:
 - If $b_{\mathcal{B}} = 0 = b_{\mathcal{C}}$: it sends vk .
 - If $b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0$: it generates $\text{vk}_{x_{\mathcal{B}} \rightarrow \mu_{\mathcal{B}}(x_{\mathcal{B}})} \leftarrow \text{GenPuncture}(\text{sk}, x_{\mathcal{B}}, \mu_{\mathcal{B}})$.
 - If $b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 1$: it generates $\text{vk}_{x_{\mathcal{C}} \rightarrow \mu_{\mathcal{C}}(x_{\mathcal{C}})} \leftarrow \text{GenPuncture}(\text{sk}, x_{\mathcal{C}}, \mu_{\mathcal{C}})$.
 - If $b_{\mathcal{B}} = 1 = b_{\mathcal{C}}$: it generates $\text{vk}_{x_{\mathcal{B}} \rightarrow \mu_{\mathcal{B}}(x_{\mathcal{B}}), x_{\mathcal{C}} \rightarrow \mu_{\mathcal{C}}(x_{\mathcal{C}})} \leftarrow \text{GenPuncture}(\text{sk}, \{x_{\mathcal{B}}, x_{\mathcal{C}}\}, \{\mu_{\mathcal{B}}, \mu_{\mathcal{C}}\})$.
- $\mathcal{A}$ produces a bipartite state $\sigma_{\mathcal{B}, \mathcal{C}}$.
- Apply $(\mathcal{B}(x_{\mathcal{B}}, \cdot) \otimes \mathcal{C}(x_{\mathcal{C}}, \cdot))(\sigma_{\mathcal{B}, \mathcal{C}})$ to obtain $(b_{\mathcal{B}}, b_{\mathcal{C}})$.
- Output 1 if $b_{\mathcal{B}} = b_{\mathcal{B}}$ and $b_{\mathcal{C}} = b_{\mathcal{C}}$ else 0.

Figure 11: $(\mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2})$ experiment of copy-protection.

$\mathcal{D}_{\mathcal{X}^2}$ -anti-piracy. Suppose $\mathcal{D}_{\mathcal{X}^2}$ is a distribution on $\{0, 1\}^{n(\lambda)} \times \{0, 1\}^{n(\lambda)}$.

We say that a copy-protection scheme for a generalized puncturable signature scheme (KeyGen , CopyProtect , Sign , Verify) satisfies $\mathcal{D}_{\mathcal{X}^2}$ -anti-piracy if for every tuple of QPT adversaries $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ there exists a negligible function $\text{negl}(\lambda)$ such that:

$$\Pr \left[1 \leftarrow \text{CPSign.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda, b_{\mathcal{B}}, b_{\mathcal{C}}) : b_{\mathcal{B}}, b_{\mathcal{C}} \overset{\$}{\leftarrow} \{0, 1\} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Remark 8.5. We note that any copy-protection scheme for a generalized puncturable signature provides a canonical signature scheme by combining Sign and CopyProtect algorithms of the copy-protection scheme to get a signing algorithm for a signature scheme. Note that we do not require any additional unforgeability property from the above scheme; the above notion is sufficient for building UPO.

9 Notation and Useful Lemmas

Let $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ be a function class. Each function in the class maps from $\{0, 1\}^{n(\lambda)}$ to $\{0, 1\}^{m(\lambda)}$. Let $(\mathcal{D}_{\mathcal{K}}, \mathfrak{C})$ be keyed circuit implementation of $(\mathcal{D}_{\mathcal{F}}, \mathcal{F})$ where $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ is also a generalized keyed circuit class equipped with a puncturing algorithm GenPuncture . Let $\text{UPO} = (\text{UPO.Obf}, \text{UPO.Eval})$

be a unclonable puncturable obfuscation for $(\mathfrak{C}, \text{GenPuncture})$ that satisfies **generalized UPO security** with respect to some distribution $\mathcal{D}_{\mathcal{X}^2}$. Let $i\mathcal{O}$ be an indistinguishability obfuscation. In this section, we mainly discuss the following potential construction of a copy-protection scheme for function class $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$.

- $\text{CP.CopyProtect}(1^\lambda, C)$: on input a security parameter λ and a circuit $C \in \mathfrak{C}_\lambda$, it outputs $\text{UPO.Obf}(1^\lambda, i\mathcal{O}(1^\lambda, C))$.
- $\text{CP.Eval}(1^\lambda, \rho_C)$: on input a quantum state ρ_C and an input $x \in \mathcal{X}_\lambda$, it outputs $\text{UPO.Eval}(1^\lambda, \rho_C)$.

We introduce some useful notations and lemmas. For any fixed copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ and distributions $\mathcal{D}_\mathcal{K}$, $\mathcal{D}_{\mathcal{X}^2}$ (which will be clear in later analysis), we define the following:

Unpunctured case. For any $y_\mathcal{B}, y_\mathcal{C}, y'_\mathcal{B}, y'_\mathcal{C} \in \{0, 1\}^{m(\lambda)}$ define

$$\mathbf{P}_{y'_\mathcal{B}, y'_\mathcal{C}}[y_\mathcal{B}, y_\mathcal{C}]$$

to be the probability that $\mathcal{B}, \mathcal{C}$ outputs $y_\mathcal{B}, y_\mathcal{C}$ respectively on inputs $x_\mathcal{B}, x_\mathcal{C}$ when $k, (x_\mathcal{B}, x_\mathcal{C})$ are drawn from $\mathcal{D}_\mathcal{K}, \mathcal{D}_{\mathcal{X}^2}$ respectively condition on $C_k(x_\mathcal{B}) = y'_\mathcal{B}$ and $C_k(x_\mathcal{C}) = y'_\mathcal{C}$. Formally,

$$\mathbf{P}_{y'_\mathcal{B}, y'_\mathcal{C}}[y_\mathcal{B}, y_\mathcal{C}] = \Pr_{\substack{k \leftarrow \mathcal{D}_\mathcal{K}(1^\lambda) \\ (x_\mathcal{B}, x_\mathcal{C}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} \left[\begin{array}{c} \mathcal{B}(x_\mathcal{B}, \sigma_\mathcal{B}) = y_\mathcal{B} \wedge \\ \mathcal{C}(x_\mathcal{C}, \sigma_\mathcal{C}) = y_\mathcal{C} \end{array} \middle| \begin{array}{c} \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, C_k) \\ \sigma_{\mathcal{B}, \mathcal{C}} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ C_k(x_\mathcal{B}) = y'_\mathcal{B} \wedge C_k(x_\mathcal{C}) = y'_\mathcal{C} \end{array} \right].$$

Punctured case. For any $y_\mathcal{B}, y_\mathcal{C}, y'_\mathcal{B}, y'_\mathcal{C}, y_\mathcal{B}^*, y_\mathcal{C}^* \in \{0, 1\}^{m(\lambda)}$ define

$$\begin{aligned} \mathbf{P}_{y'_\mathcal{B}, y'_\mathcal{C} \rightarrow y_\mathcal{C}^*}[y_\mathcal{B}, y_\mathcal{C}] &= \Pr_{\substack{k \leftarrow \mathcal{D}_\mathcal{K}(1^\lambda) \\ (x_\mathcal{B}, x_\mathcal{C}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} \left[\begin{array}{c} \mathcal{B}(x_\mathcal{B}, \sigma_\mathcal{B}) = y_\mathcal{B} \wedge \\ \mathcal{C}(x_\mathcal{C}, \sigma_\mathcal{C}) = y_\mathcal{C} \end{array} \middle| \begin{array}{c} G_{k^*} \leftarrow \text{GenPuncture}(k, x_\mathcal{C}, \mu_{\mathcal{C}}^*) \\ \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, G_{k^*}) \\ \sigma_{\mathcal{B}, \mathcal{C}} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ C_k(x_\mathcal{B}) = y'_\mathcal{B} \wedge C_k(x_\mathcal{C}) = y'_\mathcal{C} \end{array} \right], \\ \mathbf{P}_{y'_\mathcal{B} \rightarrow y_\mathcal{B}^*, y'_\mathcal{C}}[y_\mathcal{B}, y_\mathcal{C}] &= \Pr_{\substack{k \leftarrow \mathcal{D}_\mathcal{K}(1^\lambda) \\ (x_\mathcal{B}, x_\mathcal{C}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} \left[\begin{array}{c} \mathcal{B}(x_\mathcal{B}, \sigma_\mathcal{B}) = y_\mathcal{B} \wedge \\ \mathcal{C}(x_\mathcal{C}, \sigma_\mathcal{C}) = y_\mathcal{C} \end{array} \middle| \begin{array}{c} G_{k^*} \leftarrow \text{GenPuncture}(k, x_\mathcal{B}, \mu_{\mathcal{B}}^*) \\ \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, G_{k^*}) \\ \sigma_{\mathcal{B}, \mathcal{C}} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ C_k(x_\mathcal{B}) = y'_\mathcal{B} \wedge C_k(x_\mathcal{C}) = y'_\mathcal{C} \end{array} \right] \end{aligned}$$

and

$$\mathbf{P}_{y'_\mathcal{B} \rightarrow y_\mathcal{B}^*, y'_\mathcal{C} \rightarrow y_\mathcal{C}^*}[y_\mathcal{B}, y_\mathcal{C}] = \Pr_{\substack{k \leftarrow \mathcal{D}_\mathcal{K}(1^\lambda) \\ (x_\mathcal{B}, x_\mathcal{C}) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} \left[\begin{array}{c} \mathcal{B}(x_\mathcal{B}, \sigma_\mathcal{B}) = y_\mathcal{B} \wedge \\ \mathcal{C}(x_\mathcal{C}, \sigma_\mathcal{C}) = y_\mathcal{C} \end{array} \middle| \begin{array}{c} G_{k^*} \leftarrow \text{GenPuncture}(k, x_\mathcal{B}, x_\mathcal{C}, \mu_{\mathcal{B}}^*, \mu_{\mathcal{C}}^*) \\ \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, G_{k^*}) \\ \sigma_{\mathcal{B}, \mathcal{C}} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ C_k(x_\mathcal{B}) = y'_\mathcal{B} \wedge C_k(x_\mathcal{C}) = y'_\mathcal{C} \end{array} \right]$$

where $\mu_{\mathcal{B}}^*(x) = y_\mathcal{B}^*$ for all $x \in \{0, 1\}^{n(\lambda)}$ and $\mu_{\mathcal{C}}^*(x) = y_\mathcal{C}^*$ for all $x \in \{0, 1\}^{n(\lambda)}$.

Other notations. For any $y'_\mathcal{B}, y'_\mathcal{C}, y_\mathcal{B}^*, y_\mathcal{C}^* \in \{0, 1\}^{m(\lambda)}$ define

$$\mathbf{r}_{y'_\mathcal{B} \rightarrow y_\mathcal{B}^*, y'_\mathcal{C} \rightarrow y_\mathcal{C}^*}[y_\mathcal{B}, y_\mathcal{C}]$$

to be the probability that $C_k(x_\mathcal{B}) = y'_\mathcal{B}$ and $C_k(x_\mathcal{C}) = y'_\mathcal{C}$ when $k, (x_\mathcal{B}, x_\mathcal{C})$ are drawn from $\mathcal{D}_\mathcal{K}, \mathcal{D}_{\mathcal{X}^2}$ respectively condition on $\mathcal{B}$ outputting $y_\mathcal{B}$ and $\mathcal{C}$ outputting $y_\mathcal{C}$ when the challenger uses

the punctured circuit G_{k^*} at the beginning. On top of that, G_{k^*} outputs y_B^*, y_C^* on input x_B, x_C , respectively. Formally, let μ be a circuit that outputs y_B^* on x_B and outputs y_C^* on x_C .

$$\mathbf{r}_{y_B' \rightarrow y_B^*, y_C' \rightarrow y_C^*}[y_B, y_C] = \Pr_{\substack{k \leftarrow \mathcal{D}_K(1^\lambda) \\ (x_B, x_C) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} \left[\begin{array}{c|c} C_k(x_B) = y_B \wedge \\ C_k(x_C) = y_C \end{array} \middle| \begin{array}{l} G_{k^*} \leftarrow \text{GenPuncture}(k, x_B, x_C, \mu_B^*, \mu_C^*) \\ \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, G_{k^*}) \\ \sigma_{B,C} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ \mathcal{B}(x_B, \sigma_B) = y_B' \wedge \mathcal{C}(x_C, \sigma_C) = y_C' \end{array} \right]$$

where $\mu_B^*(x) = y_B^*$ for all $x \in \{0, 1\}^{n(\lambda)}$ and $\mu_C^*(x) = y_C^*$ for all $x \in \{0, 1\}^{n(\lambda)}$.

Also we define

$$\mathbf{s}_{y_B', y_C'} = \Pr_{\substack{k \leftarrow \mathcal{D}_K(1^\lambda) \\ (x_B, x_C) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} [C_k(x_B) = y_B' \wedge C_k(x_C) = y_C'].$$

We extend above notations by allowing y_B', y_C', y_B, y_C to be $*$ meaning that we remove the constraint on that value and it can be any value in its domain. Also we may write $\neq y_C^{\text{target}}$ to add the constraint that Charlie's output is not y_C^{target} . For example,

$$\begin{aligned} & \mathbf{P}_{y_B' \rightarrow y_B^*, * \rightarrow y_C^*}[y_B, \neq y_C^{\text{target}}] \\ &= \Pr_{\substack{k \leftarrow \mathcal{D}_K(1^\lambda) \\ (x_B, x_C) \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} \left[\begin{array}{c|c} \mathcal{B}(x_B, \sigma_B) = y_B \wedge \\ \mathcal{C}(x_C, \sigma_C) \neq y_C^{\text{target}} \end{array} \middle| \begin{array}{l} G_{k^*} \leftarrow \text{GenPuncture}(k, x_B, x_C, \mu_B^*, \mu_C^*) \\ \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, G_{k^*}) \\ \sigma_{B,C} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ C_k(x_B) = y_B' \end{array} \right] \\ &= \frac{\sum_{y_C'} \mathbf{s}_{y_C', y_C^*}^{\text{target}} \mathbf{P}_{y_B' \rightarrow y_B^*, y_C' \rightarrow y_C^*}[y_B, y_C]}{\sum_{y_C'} \mathbf{s}_{y_B', y_C'}}. \end{aligned}$$

9.1 Puncture-Hiding Lemma

Lemma 9.1. *For any copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ and distributions $\mathcal{D}_K, \mathcal{D}_{\mathcal{X}^2}$, for all $y_B, y_C, y_B^*, y_C^* \in \{0, 1\}^{m(\lambda)}$,*

$$\left| \mathbf{P}_{y_B^*, y_C^*}[y_B, y_C] - \mathbf{P}_{y_B^* \rightarrow y_B^*, y_C^* \rightarrow y_C^*}[y_B, y_C] \right| < \text{negl}(\lambda).$$

Proof. Note that C_k and G_{k^*} are functionally equivalent when $C_k(x_B) = y_B^*$ and $C_k(x_C) = y_C^*$. If there exists a polynomial $p(\cdot)$ such that

$$\left| \mathbf{P}_{y_B^*, y_C^*}[y_B, y_C] - \mathbf{P}_{y_B^* \rightarrow y_B^*, y_C^* \rightarrow y_C^*}[y_B, y_C] \right| \geq \frac{1}{p(\lambda)}.$$

Because the probability is averaging over k, x_B, x_C there must exists $k \in \mathcal{D}_K$ and $x_B, x_C \in \mathcal{X}$ such that $C_k(x_B) = y_B^*, C_k(x_C) = y_C^*$ and

$$\begin{aligned} & \left| \Pr \left[\begin{array}{c|c} \mathcal{B}(x_B, \sigma_B) = y_B \wedge \\ \mathcal{C}(x_C, \sigma_C) = y_C \end{array} \middle| \begin{array}{l} \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, C_k) \\ \sigma_{B,C} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ C_k(x_B) = y_B' \wedge C_k(x_C) = y_C' \end{array} \right] - \right. \\ & \left. \Pr \left[\begin{array}{c|c} \mathcal{B}(x_B, \sigma_B) = y_B \wedge \\ \mathcal{C}(x_C, \sigma_C) = y_C \end{array} \middle| \begin{array}{l} G_{k^*} \leftarrow \text{GenPuncture}(k, x_B, x_C, \mu_B^*, \mu_C^*) \\ \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, G_{k^*}) \\ \sigma_{B,C} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ C_k(x_B) = y_B' \wedge C_k(x_C) = y_C' \end{array} \right] \right| \geq \frac{1}{p(\lambda)}. \end{aligned}$$

Then there must exist the adversary Adv shown in Figure 12 breaks the $i\mathcal{O}$ security by distinguishing obfuscations of C_k and obfuscations of G_{k^*} .

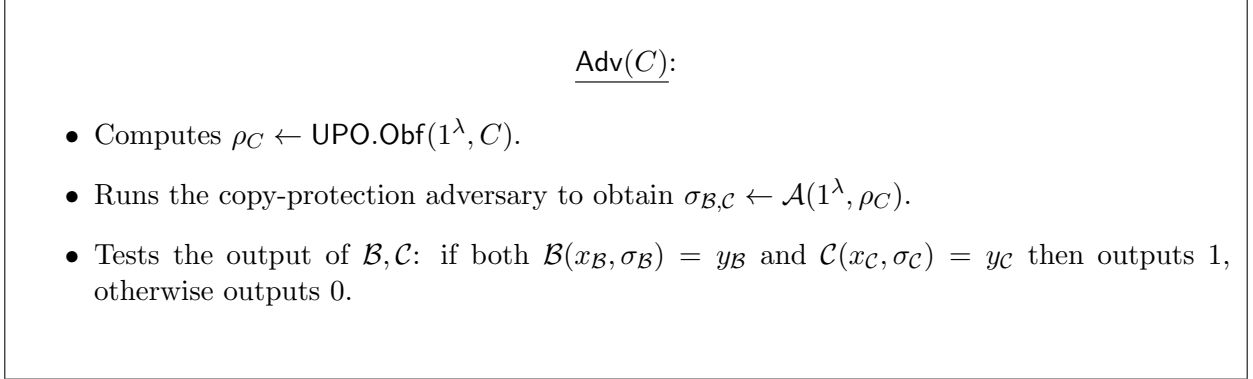


Figure 12: The puncturing security adversary Adv

By contradiction, we complete the proof. □

9.2 Output-Hiding Lemma

We say that a distribution $\mathcal{D}_{\mathcal{X}^2}$ on $\mathcal{X} \times \mathcal{X}$ is a diagonal distribution if all its support $(x_{\mathcal{B}}, x_{\mathcal{C}})$ satisfies $x_{\mathcal{B}} = x_{\mathcal{C}}$.

Lemma 9.2. *Let $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ be a keyed puncturable circuit class with $m(\lambda) = O(\log \lambda)$ that satisfies the generalized puncturing security with respect to $\mathcal{D}_{\mathcal{K}}$ and a diagonal distribution $\mathcal{D}_{\mathcal{X}^2}$ such that $p_{\text{triv}}(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}) = \frac{1}{2^{m(\lambda)}}$. If we construct a copy-protection scheme for this circuit class as above then for any copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ and any $y'_0, y'_1, y_{\mathcal{B}}, y_{\mathcal{C}}, y^* \in \{0, 1\}^m$, we have*

$$\left| \Pr_{y'_0 \rightarrow y^*, y'_0 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] - \Pr_{y'_1 \rightarrow y^*, y'_1 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] \right| < \text{negl}(\lambda).$$

Proof. If there exists a copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$, a polynomial $p(\cdot)$ and $y'_0, y'_1, y_{\mathcal{B}}, y_{\mathcal{C}}, y^* \in \{0, 1\}^m$ such that

$$\Pr_{y'_0 \rightarrow y^*, y'_0 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] - \Pr_{y'_1 \rightarrow y^*, y'_1 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] \geq \frac{1}{p(\lambda)}.$$

We turn such adversary to a generalized puncturing security adversary Adv that satisfies

$$\begin{aligned} & \Pr \left[1 \leftarrow \text{GenPuncturing.Expt}^{\text{Adv}, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda) \right] \\ & \geq p_{\text{triv}}(\lambda, \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}) + \frac{1}{2^{2m(\lambda)} p(\lambda)} = \frac{1}{2^{m(\lambda)}} + \frac{1}{2^{2m(\lambda)} p(\lambda)}. \end{aligned}$$

Define

$$\mathbf{t}[y_{\mathcal{B}}, y_{\mathcal{C}}] = \Pr_{\substack{k \leftarrow \mathcal{D}_{\mathcal{K}} \\ (x_{\mathcal{B}}, x_{\mathcal{C}}) \leftarrow \mathcal{D}_{\mathcal{X}^2}}} \left[\begin{array}{l} G_{k^*} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, x_{\mathcal{C}}, \mu, \mu) \\ \rho_C \leftarrow \text{CP.CopyProtect}(1^\lambda, G_{k^*}) \\ \sigma_{\mathcal{B}, \mathcal{C}} \leftarrow \mathcal{A}(1^\lambda, \rho_C) \\ \mathcal{B}(x_{\mathcal{B}}, \sigma_{\mathcal{B}}) = y_{\mathcal{B}} \wedge \\ \mathcal{C}(x_{\mathcal{C}}, \sigma_{\mathcal{C}}) = y_{\mathcal{C}} \end{array} \right]$$

be the overall probability that $\mathcal{B}$ outputs $y_{\mathcal{B}}$ and $\mathcal{C}$ outputs $y_{\mathcal{C}}$ where μ is a circuit that outputs y^* on every input. Firstly, we have

$$\begin{aligned}
& \mathbf{r}_{y'_0 \rightarrow y^*, y'_0 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] - \mathbf{r}_{y'_1 \rightarrow y^*, y'_1 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] \\
&= \frac{\mathbf{s}_{y'_0, y'_0}}{\mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]} \mathbf{p}_{y'_0 \rightarrow y^*, y'_0 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] - \frac{\mathbf{s}_{y'_1, y'_1}}{\mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]} \mathbf{p}_{y'_1 \rightarrow y^*, y'_1 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] \\
&= \frac{1}{2^{m(\lambda)} \mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]} \left(\mathbf{p}_{y'_0 \rightarrow y^*, y'_0 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] - \mathbf{p}_{y'_1 \rightarrow y^*, y'_1 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] \right) \\
&\geq \frac{1}{2^{m(\lambda)} p(\lambda) \mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]} \cdot (\star)
\end{aligned}$$

From this, we have the following claim.

Claim 9.3. *There exists a $y' \in \{0, 1\}^{m(\lambda)}$ that satisfies*

$$\mathbf{r}_{y' \rightarrow y^*, y' \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] \geq \frac{1}{2^{m(\lambda)}} + \frac{1}{2^{2m(\lambda)} p(\lambda) \mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]}.$$

Proof. To see this, first note that:

$$\sum_y \mathbf{r}_{y \rightarrow y^*, y \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] = 1 \quad (\star\star).$$

Suppose for every y' (and in particular, for $y' = y'_0$),

$$\mathbf{r}_{y' \rightarrow y^*, y' \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] < \frac{1}{2^{m(\lambda)}} + \frac{1}{2^{2m(\lambda)} p(\lambda) \mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]}.$$

From $(\star)$, it follows that:

$$\mathbf{r}_{y'_1 \rightarrow y^*, y'_1 \rightarrow y^*} [y_{\mathcal{B}}, y_{\mathcal{C}}] < \frac{1}{2^{m(\lambda)}} - \frac{1}{2^{2m(\lambda)} p(\lambda) \mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]}$$

Plugging this into $(\star\star)$, the following holds

$$\begin{aligned}
1 &< (2^{m(\lambda)} - 1) \left(\frac{1}{2^{m(\lambda)}} + \frac{1}{2^{2m(\lambda)} p(\lambda) \mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]} \right) + \left(\frac{1}{2^{m(\lambda)}} - \frac{1}{2^{2m(\lambda)} p(\lambda) \mathbf{t} [y_{\mathcal{B}}, y_{\mathcal{C}}]} \right) \\
&< 1,
\end{aligned}$$

which is a contradiction. □

Now we can construct Adv as shown in Figure 13.

Adv(1^λ):

- The adversary Adv sends two identical circuits (μ, μ) that outputs y^* on every input to the challenger. It receives a circuit G , sends $b = 0$ and receives a point x .
- Computes $\rho_G \leftarrow \text{CP.CopyProtect}(1^\lambda, G)$.
- Runs the copy-protection adversary to obtain $\sigma_{\mathcal{B}, \mathcal{C}} \leftarrow \mathcal{A}(1^\lambda, \rho_G)$.
- Tests the output of $\mathcal{B}, \mathcal{C}$: if both $\mathcal{B}(x, \sigma_{\mathcal{B}}) = y_{\mathcal{B}}$ and $\mathcal{C}(x, \sigma_{\mathcal{C}}) = y_{\mathcal{C}}$ then outputs y' , otherwise outputs an uniform random guess $y' \xleftarrow{\$} \{0, 1\}^{m(\lambda)}$.

Figure 13: The puncturing security adversary Adv

The probability that Adv wins the security game is

$$\begin{aligned} & \mathbf{t}[y_{\mathcal{B}}, y_{\mathcal{C}}] \mathbf{r}_{y' \rightarrow y^*, y' \rightarrow y^*}[y_{\mathcal{B}}, y_{\mathcal{C}}] + (1 - \mathbf{t}[y_{\mathcal{B}}, y_{\mathcal{C}}]) \frac{1}{2^{m(\lambda)}} \\ & \geq \frac{1}{2^{m(\lambda)}} + \frac{1}{2^{2m(\lambda)} p(\lambda)}. \end{aligned}$$

When $m(\lambda) = O(\log \lambda)$, this probability non-negligible. $\square$

9.3 Non-Local Puncture-Hiding Lemma

Definition 9.4 (Puncture-Hiding). Let $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ be a (generalized) puncturable keyed circuit class with key space $\mathcal{K}$. We say $\mathfrak{C}$ is puncture-hiding with respect to $\mathcal{D}_{\mathcal{K}}$ and $\mathcal{D}_{\mathcal{X}}$ on $\mathcal{X}$ if for any adversary $\mathcal{A}$ and any circuit μ :

$$\left| \Pr_{k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda)} [\text{Adv}(i\mathcal{O}(1^\lambda, (C_k))) = 1] - \Pr_{\substack{k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda) \\ x \leftarrow \mathcal{D}_{\mathcal{X}}(1^\lambda, k)}} [\text{Adv}(i\mathcal{O}(1^\lambda, G_{k^*})) = 1 \mid G_{k^*} \leftarrow \text{GenPuncture}(k, x, \mu)] \right| < \text{negl}(\lambda),$$

for some negligible function $\text{negl}(\cdot)$.

Lemma 9.5. Let $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ be a keyed puncturable circuit class with key space $\mathcal{K}$. Let $\mathcal{D}_{\mathcal{K}}$ be a distribution on $\mathcal{K}$ and let $\mathcal{D}_{\mathcal{X}^2} = \mathcal{D}_{\mathcal{X}}^{\mathcal{B}} \times \mathcal{D}_{\mathcal{X}}^{\mathcal{C}}$ be a distribution on $\mathcal{X} \times \mathcal{X}$ such that the two elements are independent. If $\mathfrak{C}$ is puncture-hiding with respect to $\mathcal{D}_{\mathcal{K}}$ and $\mathcal{D}_{\mathcal{X}}^{\mathcal{C}}$ then for any copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ we have

- When Bob's input is not punctured, he cannot distinguish whether Charlie's input is punctured or not

$$s_{y'_B, *} \left| \mathbf{P}_{y'_B, *} [y_{\mathcal{B}}, *] - \mathbf{P}_{y'_B, * \rightarrow y_C^*} [y_{\mathcal{B}}, *] \right| < \text{negl}(\lambda).$$

- When Bob's input is punctured, he cannot distinguish whether Charlie's input is punctured or not

$$s_{y'_B, *} \left| \mathbf{P}_{y'_B \rightarrow y_B^*, *} [y_B, *] - \mathbf{P}_{y'_B \rightarrow y_B^*, * \rightarrow y_C^*} [y_B, *] \right| < \text{negl}(\lambda).$$

The statement also holds when we swap Bob and Charlie.

Proof. In the case where Bob's input not punctured. Suppose there exists a copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ and a polynomial $p(\cdot)$ such that

$$s_{y'_B, *} \left| \mathbf{P}_{y'_B, *} [y_B, *] - \mathbf{P}_{y'_B, * \rightarrow y_C^*} [y_B, *] \right| < \text{negl}(\lambda).$$

Then we construct an adversary $\text{Adv}_{\text{unpunc}}$ and a circuit μ that breaks the puncture-hiding property of $\mathfrak{C}$. μ is the circuit that outputs y_C^* on any inputs and $\text{Adv}_{\text{unpunc}}$ is shown in Figure 14.

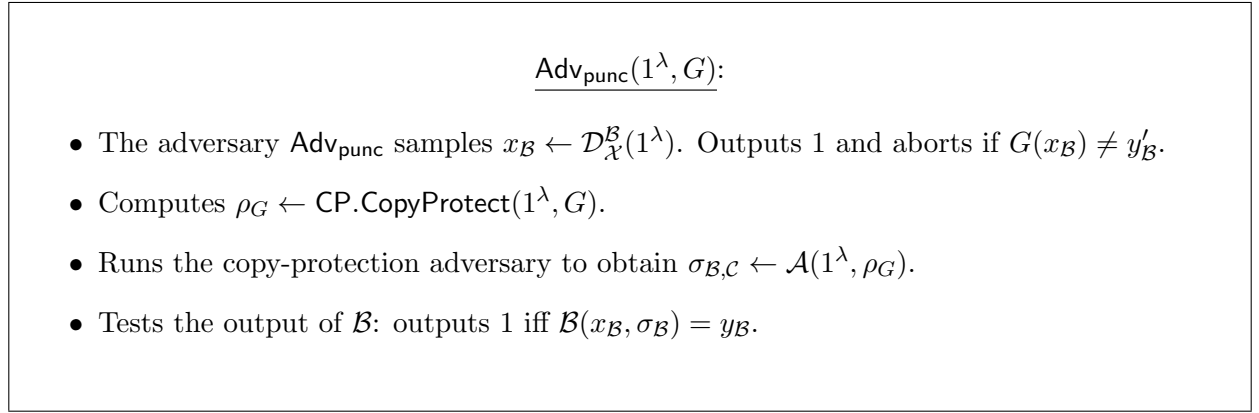


Figure 14: The puncturing security adversary Adv_{punc}

When G is the punctured circuit that outputs y_C^* at point x_C , the distribution of $i\mathcal{O}(G)$ is exactly the distribution of $i\mathcal{O}(\text{GenPuncture}(C_k, x_C, \mu_C))$ where μ_C outputs y_C^* on every input. Thus if $G(x_B) = y'_B$ then the probability of $\mathcal{B}(x_B, \sigma_B) = y_B$ is exactly $\mathbf{P}_{y'_B, * \rightarrow y_C^*} [y_B, *]$ over the randomness of x_C and the randomness of $\mathcal{C}$. On the other hand, when G is the original circuit, the distribution of $i\mathcal{O}(G)$ is exactly the distribution of $i\mathcal{O}(C_k)$. Thus if $G(x_B) = y'_B$ then the probability of $\mathcal{B}(x_B, \sigma_B) = y_B$ is exactly $\mathbf{P}_{y'_B, *} [y_B, *]$ over the randomness of x_C and the randomness of $\mathcal{C}$. We have

$$\begin{aligned} & \Pr_{k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda)} [\text{Adv}_{\text{unpunc}}(C_k) = 1] \\ & - \Pr_{\substack{k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda) \\ x_C \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} [\text{Adv}_{\text{unpunc}}(G_{k^*}) = 1 | G_{k^*} \leftarrow \text{GenPuncture}(C_k, x_C, \mu_C)] \\ & = s_{y'_B, *} \left(\mathbf{P}_{y'_B, *} [y_B, *] - \mathbf{P}_{y'_B, * \rightarrow y_C^*} [y_B, *] \right) \geq \frac{1}{p(\lambda)}. \end{aligned}$$

By contradiction we prove the unpunctured case. Now we consider the case where Bob's input is punctured and it is similar to the unpunctured case. Suppose there exists a copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ and a polynomial $p(\cdot)$ such that

$$s_{y'_B, *} \left(\mathbf{P}_{y'_B \rightarrow y_B^*, *} [y_B, *] - \mathbf{P}_{y'_B \rightarrow y_B^*, * \rightarrow y_C^*} [y_B, *] \right) \geq \frac{1}{p(\lambda)}. \quad (6)$$

Then we construct an adversary Adv_{punc} and a circuit μ that breaks the puncture-hiding property of $\mathfrak{C}$. μ is the circuit that outputs $y_{\mathcal{C}}^*$ on any inputs and Adv_{punc} is shown in Figure 15.

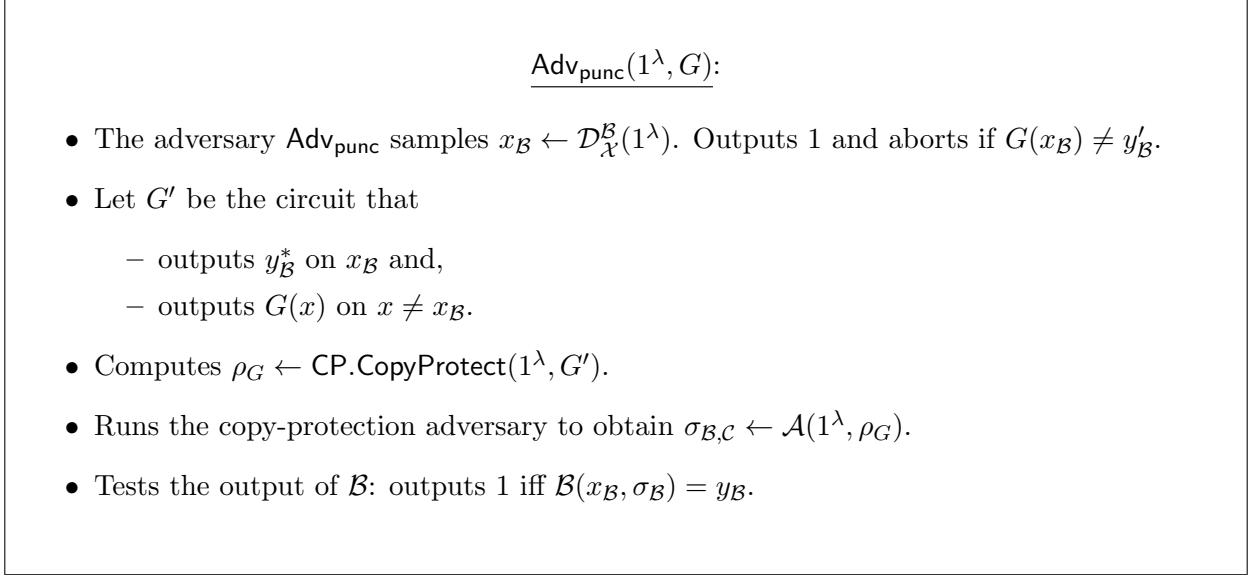


Figure 15: The puncturing security adversary Adv_{punc}

When G is the punctured circuit that outputs $y_{\mathcal{C}}^*$ at point $x_{\mathcal{C}}$, the distribution of $i\mathcal{O}(G')$ is computationally indistinguishable from the distribution of $i\mathcal{O}(\text{GenPuncture}(C_k, x_{\mathcal{B}}, x_{\mathcal{C}}, \mu_{\mathcal{B}}, \mu_{\mathcal{C}}))$ where $\mu_{\mathcal{B}}/\mu_{\mathcal{C}}$ outputs $y_{\mathcal{B}}^*/y_{\mathcal{C}}^*$ on every input, respectively. Thus if $G(x_{\mathcal{B}}) = y'_{\mathcal{B}}$ then the probability of $\mathcal{B}(x_{\mathcal{B}}, \sigma_{\mathcal{B}}) = y_{\mathcal{B}}$ is exactly $\mathbf{P}_{y'_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, * \rightarrow y_{\mathcal{C}}^*}[y_{\mathcal{B}}, *]$ over the randomness of $x_{\mathcal{C}}$ and the randomness of $\mathcal{C}$. On the other hand, when G is the original circuit, the distribution of $i\mathcal{O}(G')$ is computationally indistinguishable from the distribution of $i\mathcal{O}(\text{GenPuncture}(C_k, x_{\mathcal{B}}, x_{\mathcal{B}}, \mu_{\mathcal{B}}, \mu_{\mathcal{B}}))$. Thus if $G(x_{\mathcal{B}}) = y'_{\mathcal{B}}$ then the probability of $\mathcal{B}(x_{\mathcal{B}}, \sigma_{\mathcal{B}}) = y_{\mathcal{B}}$ is exactly $\mathbf{P}_{y'_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, *}[y_{\mathcal{B}}, *]$ over the randomness of $x_{\mathcal{C}}$ and the randomness of $\mathcal{C}$. We have

$$\begin{aligned}
& \Pr_{k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda)} [\text{Adv}_{\text{punc}}(C_k) = 1] \\
& - \Pr_{\substack{k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda) \\ x_{\mathcal{C}} \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda, k)}} [\text{Adv}_{\text{punc}}(G_{k^*}) = 1 | G_{k^*} \leftarrow \text{GenPuncture}(C_k, x_{\mathcal{C}}, \mu_{\mathcal{C}})] \\
& = \mathbf{P}_{y'_{\mathcal{B}}, *}(\mathbf{P}_{y'_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, *}[y_{\mathcal{B}}, *] - \mathbf{P}_{y'_{\mathcal{B}} \rightarrow y_{\mathcal{B}}^*, * \rightarrow y_{\mathcal{C}}^*}[y_{\mathcal{B}}, *]) - \text{negl}(\lambda) \geq \frac{1}{p(\lambda)} - \text{negl}(\lambda).
\end{aligned}$$

By contradiction, we prove the theorem. □

Lemma 9.6. *Let $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ be a (generalized) puncturable keyed circuit class with key space $\mathcal{K}$. Then assuming length-doubling post-quantum PRG mapping $\{0, 1\}^n \rightarrow \{0, 1\}^{2n}$, and for any efficiently sampleable distribution $\mathcal{D}_{\mathcal{K}}$ on $\mathcal{K}$, $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ satisfies puncture hiding (as defined in Definition 9.4) with respect to $\mathcal{D}_{\mathcal{K}}$ and $\mathcal{U}_n$, the uniform challenge input distribution on $\{0, 1\}^n$.*

Proof. We need to show that

$$\{\widetilde{C}_k\}_{k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda), \widetilde{C}_k \leftarrow i\mathcal{O}(C_k)} \approx_c \{\widetilde{G}_{k^*}\}_{k \leftarrow \mathcal{D}_{\mathcal{K}}(1^\lambda), x \xleftarrow{\$} \{0, 1\}^{n(\lambda)}, G_{k^*} \leftarrow \text{GenPuncture}(k, x, \mu), \widetilde{G}_{k^*} \leftarrow i\mathcal{O}(G_{k^*})}, \quad (7)$$

where $\approx_c$ means computationally indistinguishable. Let Adv be any efficient distinguisher. Consider the following set of hybrids. We indicate the changes across consecutive hybrids in blue.

H_0 : This is the same as the LHS distribution in Equation (7).

1. Ch samples $k \leftarrow \mathcal{D}_K(1^\lambda)$ and generates $\widetilde{C}_k \leftarrow i\mathcal{O}(C_k)$, and sends $\widetilde{C}_k$ to Adv.
2. Adv on input 1^λ and $\widetilde{C}_k$, outputs a bit b .

H_1 :

1. Ch samples $k \leftarrow \mathcal{D}_K(1^\lambda)$ and samples $z^* \xleftarrow{\$} \{0,1\}^{2n}$ generates $\widetilde{C}_k \leftarrow i\mathcal{O}(G)$ where G is the circuit given in Figure 16, and sends $\widetilde{C}_k$ to Adv.

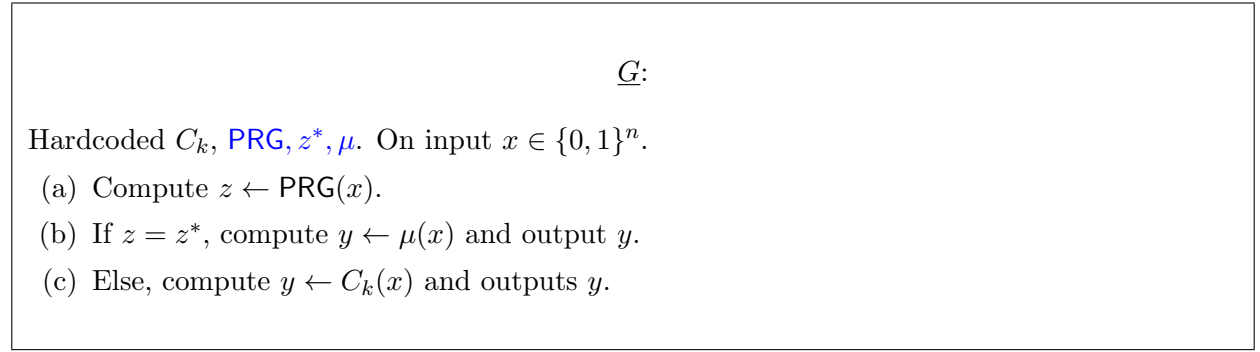


Figure 16: Circuit G in the H_1 of the LHS.

2. Adv on input 1^λ and $\widetilde{C}_k$, outputs a bit b .

Note that the only difference across the hybrids is how we generate $\widetilde{C}_k$. Since the image of PRG is a negligible (in n and hence λ) fraction of $\{0,1\}^{2n}$ and since z^* was sampled uniformly at random from $\{0,1\}^{2n}$, we conclude that except with negligible probability over the random choice of z^* , z^* lies outside the image of PRG , and hence the *if* check in line 1b in the description of G in H_1 is not satisfied for any input, and hence the circuit G and C_k has the same functionality. Hence, by the security of $i\mathcal{O}$, we conclude that except with negligible probability in λ , H_0 and H_1 are indistinguishable for the Adv.

H_2 :

1. Ch samples $k \leftarrow \mathcal{D}_K(1^\lambda)$ and samples $x^* \xleftarrow{\$} \{0,1\}^n$ and generates $z^* \leftarrow \text{PRG}(x^*)$ and generates $\widetilde{C}_k \leftarrow i\mathcal{O}(G)$, where G is the circuit given in Figure 17, and sends $\widetilde{C}_k$ to Adv.
2. Adv on input 1^λ and $\widetilde{C}_k$, outputs a bit b .

The indistinguishability between H_1 and H_2 holds by the pseudorandomness of PRG .

H_3 :

1. Ch samples $k \leftarrow \mathcal{D}_K(1^\lambda)$ and samples $x^* \xleftarrow{\$} \{0,1\}^n$ and generates $z^* \leftarrow \text{PRG}(x^*)$ and generates $\widetilde{C}_k \leftarrow i\mathcal{O}(G)$, where G is the circuit given in Figure 17, and sends $\widetilde{C}_k$ to Adv.

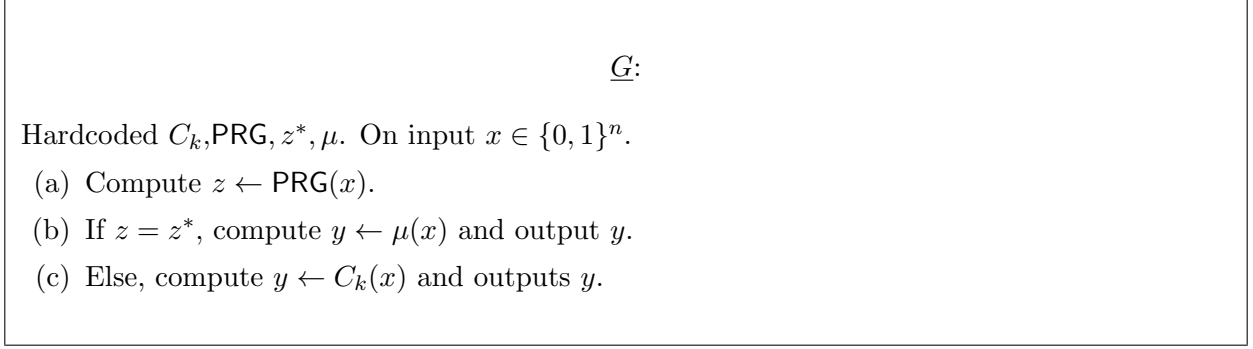


Figure 17: Circuit G in the H_2 of the LHS.

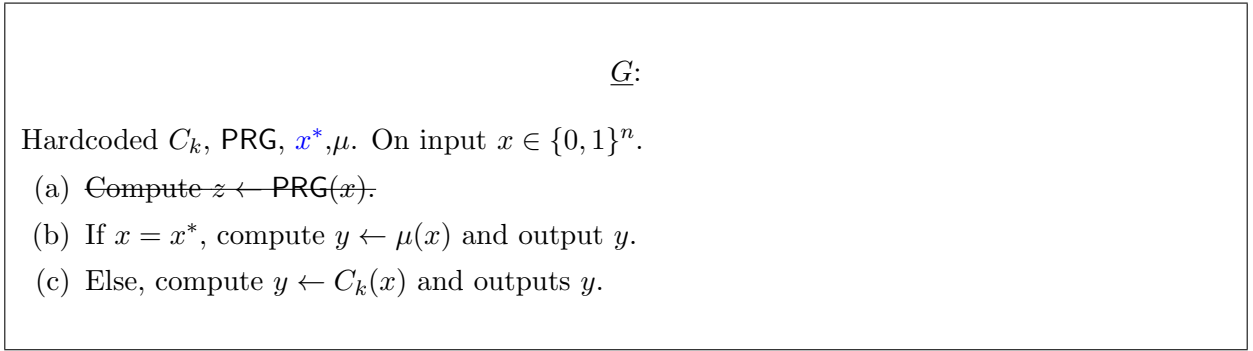


Figure 18: Circuit G in the H_3 of the LHS.

2. Adv on input 1^λ and $\widetilde{C}_k$, outputs a bit b .

By the injectivity of PRG , the functionality of G in H_2 and H_3 is the same and hence by the security of $i\mathcal{O}$, $\widetilde{C}_k$ as generated in H_2 and as generated in H_3 are indistinguishable, and hence the indistinguishability of H_2 and H_3 holds.

H_4 :

1. Ch samples $k \leftarrow \mathcal{D}_K(1^\lambda)$ and samples $x^* \xleftarrow{\$} \{0, 1\}^n$ and generates $\widetilde{C}_k \leftarrow i\mathcal{O}(G_{k^*})$, where $G_{k^*} \leftarrow \text{GenPuncture}(k, x^*, \mu)$, and sends $\widetilde{C}_k$ to Adv.
2. Adv on input 1^λ and $\widetilde{C}_k$, outputs a bit b .

Note that the only difference between H_3 and H_4 , is that in the generation of $\widetilde{C}_k$, we replace G in H_3 with G_{k^*} in H_4 . Note that the functionality of G and G_{k^*} is the same by the correctness of GenPuncture , and hence by the security of $i\mathcal{O}$, the indistinguishability between H_3 and H_4 holds.

Note that H_4 exactly captures the RHS distribution of Equation (7). Since H_0 captures the LHS distribution of Equation (7) (and H_4 captures the RHS distribution of Equation (7)), by the indistinguishability of the above-mentioned hybrids, we conclude that Equation (7) holds, which completes the proof of the lemma. $\square$

Lemma 9.7 (Non-local puncture hiding lemma for generalized puncturable keyed circuit classes with respect to the uniform challenge input distribution). *Let $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ be a (generalized) puncturable keyed circuit class with key space $\mathcal{K}$. Let $\mathcal{D}_\mathcal{K}$ be a distribution on $\mathcal{K}$ and let $\mathcal{D}_{\mathcal{X}^2} = \mathcal{D}_\mathcal{X}^\mathcal{B} \times \mathcal{D}_\mathcal{X}^\mathcal{C}$ be a distribution on $\mathcal{X} \times \mathcal{X}$ such that the two elements are independent. If $\mathfrak{C}$ satisfies generalized puncturing security (see Definition 6.2) with respect to $\mathcal{D}_\mathcal{K}$ and $\mathcal{D}_\mathcal{X}$ then for any copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ we have*

- When Bob's input is not punctured, he cannot distinguish whether Charlie's input is punctured or not

$$\mathbf{s}_{y'_B, *} \left| \mathbf{P}_{y'_B, *} [y_B, *] - \mathbf{P}_{y'_B, * \rightarrow y_C^*} [y_B, *] \right| < \text{negl}(\lambda).$$

- When Bob's input is punctured, he cannot distinguish whether Charlie's input is punctured or not

$$\mathbf{s}_{y'_B, *} \left| \mathbf{P}_{y'_B \rightarrow y_B^*, *} [y_B, *] - \mathbf{P}_{y'_B \rightarrow y_B^*, * \rightarrow y_C^*} [y_B, *] \right| < \text{negl}(\lambda).$$

The statement also holds when we swap Bob and Charlie.

Proof. The proof is immediate by combining Lemma 9.5 and Lemma 9.6. $\square$

10 Copy-Protecting 1-Bit Output Average-Case Puncturable Circuit Classes

In this section, we show how to copy-protect ‘balance’ function class with one-bit output. Let $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ be the function class that we want to copy-protect such that $m(\lambda) = 1$. Let $\mathcal{D}_{\mathcal{X}^2}$ be a diagonal distribution such that $p_{\text{triv}}(\lambda, \mathcal{D}_\mathcal{K}, \mathcal{D}_\mathcal{X}^\mathcal{B}) = \frac{1}{2}$.

Construction 10.1. *Let $(\mathcal{D}_\mathcal{K}, \mathfrak{C})$ be a keyed circuit implementation of $(\mathcal{D}_\mathcal{F}, \mathcal{F})$ that satisfies **generalized puncturing security** with respect to $\mathcal{D}_\mathcal{K}$ and $\mathcal{D}_{\mathcal{X}^2}$. Let $\text{UPO} = (\text{Obf.UPO}, \text{UPO.Eval})$ be an generalized secure UPO for $\mathfrak{C}$ and GenPuncture with respect to $\mathcal{D}_{\mathcal{X}^2}$. Let $i\mathcal{O}$ be an indistinguishability obfuscation. The pair of QPT algorithm $(\text{CP.CopyProtect}, \text{CP.Eval})$ is defined as follows:*

- $\text{CP.CopyProtect}(1^\lambda, C)$: on input a security parameter λ and a circuit $C \in \mathfrak{C}_\lambda$, it outputs $\text{Obf.UPO}(1^\lambda, i\mathcal{O}(1^\lambda, C))$.
- $\text{CP.Eval}(1^\lambda, \rho_C)$: on input a quantum state ρ_C and an input $x \in \mathcal{X}_\lambda$, it outputs $\text{UPO.Eval}(1^\lambda, \rho_C)$.

Theorem 10.2. *Let $\mathfrak{C} = \{\mathfrak{C}_{n(\lambda)}\}_{\lambda \in \mathbb{N}}$ be a keyed circuit class equipped with a puncturing algorithm GenPuncture . Let $\mathcal{D}_\mathcal{F}$ be a distribution on $\mathcal{F}$, $\mathcal{D}_\mathcal{K}$ be a distribution on $\mathcal{K}$ and $\mathcal{D}_{\mathcal{X}^2}$ be a distribution on $\mathcal{X}^2$. The copy-protection scheme Construction 10.1 satisfies correctness and $(\mathcal{D}_\mathcal{F}, \mathcal{D}_{\mathcal{X}^2})$ -**anti-priacy**.*

Proof. The correctness of the above construction follows from the correctness of the unclonable puncturable obfuscation scheme and the correctness of the indistinguishability obfuscation. Now we prove that it satisfies $(\mathcal{D}_\mathcal{F}, \mathcal{D}_{\mathcal{X}^2})$ -**anti-priacy**. Suppose there exists a copy-protection adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ and a polynomial $p(\cdot)$ such that

$$\Pr \left[1 \leftarrow \text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_\mathcal{K}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda) \right] \geq \frac{1}{2} + \frac{1}{p(\lambda)}$$

where $\text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_\mathcal{K}, \mathcal{D}_{\mathcal{X}^2}}$ is the experiment shown in Figure 10. We show some claims below.

Claim 10.3. For any $y_B, y_C \in \{0, 1\}$,

$$|\mathbf{p}_{0,0}[y_B, y_C] - \mathbf{p}_{0 \rightarrow 0, 0 \rightarrow 0}[y_B, y_C]| < \text{negl}(\lambda).$$

Proof. Let $y_B^* = y_C^* = 0$ and this claim follows from Lemma 9.1. $\square$

Claim 10.4. For any $y_B, y_C \in \{0, 1\}$,

$$|\mathbf{p}_{0 \rightarrow 0, 0 \rightarrow 0}[y_B, y_C] - \mathbf{p}_{1 \rightarrow 0, 1 \rightarrow 0}[y_B, y_C]| < \text{negl}(\lambda).$$

Proof. Follows from Lemma 9.2. $\square$

Claim 10.5.

$$\mathbf{p}_{0,0}[0, 0] + \mathbf{p}_{1,1}[1, 1] \geq 1 + \frac{2}{p(\lambda)}.$$

Proof. The probability that $\text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_K, \mathcal{D}_{X^2}}(1^\lambda)$ outputs 1 is exactly $\frac{1}{2} (\mathbf{p}_{0,0}[0, 0] + \mathbf{p}_{1,1}[1, 1])$. $\square$

Now we construct an adversary $\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C$ that breaks the generalized UPO security of the underlying unclonable puncturable obfuscation scheme $(\text{Obf.UPO}(1^\lambda, i\mathcal{O}(1^\lambda, \cdot)), \text{UPO.Eval})$. By Theorem 7.4, $(\text{Obf.UPO}(1^\lambda, i\mathcal{O}(1^\lambda, \cdot)), \text{UPO.Eval})$ is a generalized secure UPO.

$(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$:

- $\mathcal{R}_A$ samples $k \leftarrow \mathcal{D}_K$ and sends k, μ_B, μ_C to the challenger Ch where $\mu_B(x) = \mu_C(x) = 0$ for all $x \in \{0, 1\}^{n(\lambda)}$.
- Upon receiving ρ_C from the challenger Ch , $\mathcal{R}_A$ runs the copy-protection adversary to obtain $\sigma_{B,C} \leftarrow \mathcal{A}(1^\lambda, \rho_C)$. $\mathcal{R}_A$ sends k, σ_B to $\mathcal{R}_B$ and sends k, σ_C to $\mathcal{R}_C$.
- $\mathcal{R}_B$ upon receiving k, σ_B from $\mathcal{R}_A$ and x_B from the challenger Ch , does the following:
 - If $C_k(x_B) = 0$, it outputs 0.
 - Otherwise it computes $y_B \leftarrow \mathcal{B}(\sigma_B, x_B)$ and outputs $1 - y_B$.
- $\mathcal{R}_C$ upon receiving k, σ_C from $\mathcal{R}_A$ and x_C from the challenger Ch , does the following:
 - If $C_k(x_C) = 0$, it outputs 0.
 - Otherwise it computes $y_C \leftarrow \mathcal{C}(\sigma_C, x_C)$ and outputs $1 - y_C$.

Figure 19: The UPO security adversary $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$

Now we analyze the success probability of $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$:

$$\begin{aligned}
& \Pr_{b \xleftarrow{\$} \{0,1\}} \left[\text{GenUPO.Expt}^{(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C), \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda, b) = 1 \right] \\
&= \frac{1}{2} \left(\Pr \left[\text{GenUPO.Expt}^{(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C), \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda, 0) = 1 \right] \right. \\
&\quad \left. + \Pr \left[\text{GenUPO.Expt}^{(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C), \mathcal{D}_{\mathcal{X}^2}, \mathfrak{C}}(1^\lambda, 1) = 1 \right] \right) \\
&= \frac{1}{2} \left(s_{0,0} \cdot \frac{1}{2} + (1 - s_{0,0}) \cdot \mathbf{p}_{1,1} [1, 1] + s_{0,0} \cdot \frac{1}{2} + (1 - s_{0,0}) \cdot \mathbf{p}_{1 \rightarrow 0, 1 \rightarrow 0} [0, 0] \right) \\
&\geq \frac{1}{4} + \frac{1}{4} \mathbf{p}_{1,1} [1, 1] + \frac{1}{4} \mathbf{p}_{1 \rightarrow 0, 1 \rightarrow 0} [0, 0] - \text{negl}(\lambda) \\
&\geq \frac{1}{4} + \frac{1}{4} \mathbf{p}_{1,1} [1, 1] + \frac{1}{4} \mathbf{p}_{0 \rightarrow 0, 0 \rightarrow 0} [0, 0] - \text{negl}(\lambda) \\
&\geq \frac{1}{4} + \frac{1}{4} \mathbf{p}_{1,1} [1, 1] + \frac{1}{4} \mathbf{p}_{0,0} [0, 0] - \text{negl}(\lambda) \\
&\geq \frac{1}{2} + \frac{1}{2p(\lambda)}.
\end{aligned}$$

Thus, by contradiction, Construction 10.1 satisfies $(\mathcal{D}_{\mathcal{F}}, \mathcal{D}_{\mathcal{X}^2})$ -**anti-piracy**. $\square$

11 Copy-Protection from Independent-Secure UPO

In this section, we will show that given any unclonable puncturable obfuscation scheme satisfying independent UPO security for a generalized keyed circuit class implementing a puncturable function class containing functions with m -bit output functions is itself a $\frac{1}{3} + \frac{p_{\text{triv}}}{3}$ copy-protection scheme for the puncturable function class, where $M = 2^m$.

Let $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ equipped with a distribution $\mathcal{D}_{\mathcal{F}}$ be a puncturable function class, consisting of functions mapping $\{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$. Let $(\mathcal{D}_{\mathcal{K}}, \mathfrak{C})$ be a keyed circuit implementation of $(\mathcal{D}_{\mathcal{F}}, \mathcal{F})$ where $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ is also a generalized keyed circuit class with input length $\{0, 1\}^{n(\lambda)}$ equipped with a puncturing algorithm **GenPuncture**, satisfying generalized puncturing security (see Definition 6.2). Let $\mathcal{U}_{2n(\lambda)}(1^\lambda)$ be the uniform distribution on $\{0, 1\}^{2n(\lambda)}$.

Theorem 11.1. *Given any unclonable puncturable obfuscation $\text{UPO} = (\text{UPO.Obf}, \text{UPO.Eval})$ for $(\mathfrak{C}, \text{GenPuncture})$ that satisfies $\mathcal{U}_{2n(\lambda)}$ -independent-UPO security (as defined Definition 7.3), the copy-protection construction given in Construction 10.1 for $\mathcal{F}$, satisfies $\frac{1}{3} + \frac{p_{\text{triv}}}{3} - (\mathcal{D}_{\mathcal{F}}, \mathcal{U}_{2n(\lambda)})$ -anti-piracy (see Section 8) for $(\mathcal{D}_{\mathcal{F}}, \mathcal{F})$ implemented by $(\mathcal{D}_{\mathcal{K}}, \mathfrak{C})$, where $p_{\text{triv}}(\lambda) = p_{\text{triv}}$ is with respect to the distributions $\mathcal{D}_{\mathcal{K}}, \mathcal{U}_{2n(\lambda)}$, as defined in Definition 8.1.*

Proof. The correctness of the copy protection scheme is immediate from the correctness of UPO. Next we prove security. Let $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ be an adversary in the copy-protection game $(\text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda))$, see Figure 10) against the copy protection scheme given in Construction 10.1. Let $\mathfrak{C}_\lambda$ be represented as $\mathfrak{C}_\lambda = \{C_k\}$.

Suppose q denote the probability that $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ wins the copy-protection anti-piracy game (Figure 10), i.e., the output of the anti-piracy game with respect to $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ is 1. Therefore, we need to show that,

$$q \leq p_{\text{triv}} + \frac{1}{3} + \frac{p_{\text{triv}}}{3} + \delta(\lambda) = \frac{1}{3} + \frac{4p_{\text{triv}}}{3} + \delta(\lambda), \quad (8)$$

Let $\text{UPO}' = (\text{UPO}'.\text{Obf}, \text{UPO}'.\text{Eval})$ be defined as $\text{UPO}'.\text{Obf}(1^\lambda, C) = \text{UPO}(1^\lambda, i\mathcal{O}(1^\lambda, C))$ and $\text{UPO}'.\text{Eval} = \text{UPO}.\text{Eval}$ where $\text{UPO} = (\text{UPO}.\text{Obf}, \text{UPO}.\text{Eval})$ is the unclonable puncturable obfuscation scheme used in Construction 10.1. Since UPO satisfies independent-UPO security, then so does UPO' by the composition result for UPO with independent-UPO security, see Corollary 7.5. We construct a reduction $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ to the independent UPO security game (see Figure 9) of UPO' as given in Figure 20.¹¹

$(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$:

- $\mathcal{R}_A$ samples $k \leftarrow \mathcal{D}_K(1^\lambda)$ and sends $k, \perp, \perp$ to Ch , i.e., k is the key and $\perp$ denotes the programming circuit that outputs $\perp$ on all inputs.
- Ch samples $b_B, b_C \xleftarrow{\$} \{0, 1\}$.
- Ch samples $x_B, x_C \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda)$.
- Ch does the following:
 1. If $b_B = b_C = 0$: generate $\rho_k \leftarrow \text{UPO}'.\text{Obf}(1^\lambda, C_k)$.
 2. Else if $b_B = 1, b_C = 0$: generate $G_{k*}^{10} \leftarrow \text{GenPuncture}(k, x_B, \perp)$, and generate $\rho_k \leftarrow \text{UPO}'.\text{Obf}(1^\lambda, G_{k*}^{10})$.
 3. Else if $b_B = 0, b_C = 1$: generate $G_{k*}^{01} \leftarrow \text{GenPuncture}(k, x_C, \perp)$, and generate $\rho_k \leftarrow \text{UPO}'.\text{Obf}(1^\lambda, G_{k*}^{01})$.
 4. Else if $b_B = b_C = 1$: generate $G_{k*}^{11} \leftarrow \text{GenPuncture}(k, x_B, x_C, \perp, \perp)$, and generate $\rho_k \leftarrow \text{UPO}'.\text{Obf}(1^\lambda, G_{k*}^{11})$.
- Ch sends ρ_k to $\mathcal{R}_A$.
- $\mathcal{R}_A$ runs $\sigma_{B,C} \leftarrow \mathcal{A}(\rho_k)$. $\mathcal{R}_A$ outputs (σ_B, k) to $\mathcal{B}$ and (σ_C, k) to $\mathcal{C}$.
- $\mathcal{R}_B$ (respectively, $\mathcal{R}_C$) runs $y_B \leftarrow \mathcal{B}(\sigma_B)$ (respectively, $y_C \leftarrow \mathcal{C}(\sigma_C)$).
- $\mathcal{R}_B$ generates $y'_B \leftarrow C_k(x_B)$ and outputs $b_B = 0$ if $y'_B = y_B$, else $b_B = 1$. Similarly, $\mathcal{R}_C$ generates $y'_C \leftarrow C_k(x_C)$ and outputs $b_C = 0$ if $y'_C = y_C$, else $b_C = 1$.

Figure 20: Reduction from the anti-piracy experiment of CP to independent-UPO security experiment of UPO' .

For any quadruple of bits, $b_1, b_2, b_3, b_4 \in \{0, 1\}$, let $q_{b_3 b_4}^{b_1 b_2}$ denote the probability that the outputs of $\mathcal{R}_B$ and $\mathcal{R}_C$ are b_1 and b_2 , respectively, i.e., $b_B = b_1$, and $b_C = b_2$, conditioned on $b_B = b_3$ and $b_C = b_4$ in the independent UPO security game.

¹¹This is the same reduction as the reduction used in [AB24, Theorem 56].

By definition of the reduction $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$, the view of the adversary inside the simulation of $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ conditioned on $b_B = 0 = b_C = 0$ in the independent-UPO security game is the same as that in the real copy-protection game, and the event that $\mathcal{B}$ and $\mathcal{C}$ output correct answers in the copy-protection game, corresponds to the event that the reduction $(\mathcal{R}_B, \mathcal{R}_C)$ output $(0, 0)$ in the independent-UPO security game conditioned on $b_B = 0 = b_C = 0$. Hence, we conclude that

$$q_{00}^{00} = q.$$

Therefore in order to show Equation (8), it is enough to show that

$$q_{00}^{00} \leq \frac{1}{3} + \frac{4p_{\text{triv}}}{3} + \delta(\lambda), \quad (9)$$

for some negligible function $\delta(\cdot)$.

We will instead prove a stronger claim, as given below.

Claim 11.2. *There exists a negligible function $\delta'(\lambda)$ such that,*

$$3q_{00}^{00} + q_{00}^{01} + q_{00}^{10} \leq 1 + 4p_{\text{triv}} + \delta'(\lambda).$$

Note that by Claim 11.2 (by dividing both sides by 3), we get,

$$q_{00}^{00} \leq \frac{1}{3} + \frac{4p_{\text{triv}}}{3} + \frac{\delta'(\lambda)}{3},$$

which implies Equation (9), and hence the theorem.

Next we prove Claim 11.2, to complete the proof of the theorem.

Proof of Claim 11.2 Note that the independent-UPO security of UPO implies that

$$\frac{\sum_{b_1, b_2} q_{b_1 b_1}^{b_1 b_1}}{4} \leq \frac{1}{2} + \epsilon(\lambda), \quad (10)$$

for some negligible function $\epsilon(\cdot)$. By the definition of the reduction, the event $b_{\mathbf{B}} = 0$ given $b_B = 0, b_C = 0$ in the independent UPO security experiment happens with the same probability as the event in the true copy-protection game that the output of $\mathcal{B}$ is correct, i.e., $y_B = y'_B$ where $y'_B \leftarrow C_k(x_B)$ and x_B is the challenge message sampled uniformly at random by the challenger. Hence using the notations from Section 9, this is the same as saying:

$$\Pr[b_{\mathbf{B}} = 0 \mid b_B = 0, b_C = 0] = \sum_{y'_B \in \{0,1\}^m} \mathbf{s}_{y'_B, *} \cdot \mathbf{p}_{y'_B, *} [y'_B, *], \quad (11)$$

where the notations are with respect to the distributions $\mathcal{D}_{\mathcal{K}}$ and $\mathcal{D}_{\mathcal{X}^2} = \mathcal{U}_{2n(\lambda)}$. Similarly, the event $b_{\mathbf{B}} = 0$ given $b_B = 0, b_C = 1$ in the independent UPO security experiment happens with the same probability as the event in the copy-protection game with the circuit is punctured for $\mathcal{C}$'s input, where the output of $\mathcal{B}$ is incorrect, i.e., $y_B = y'_B$ where $y'_B \leftarrow C_k(x_B)$ and x_B is the challenge message sampled uniformly at random by the challenger. Hence using the notations from Section 9, this is the same as saying:

$$\Pr[b_{\mathbf{B}} = 0 \mid b_B = 0, b_C = 1] = \sum_{y'_B \in \{0,1\}^m} \mathbf{s}_{y'_B, *} \cdot \mathbf{p}_{y'_B, * \rightarrow \perp} [y_B, *]. \quad (12)$$

Combining Equations (11) and (12) with the non-local puncture hiding lemma for circuits satisfying generalized puncturing security (see Lemma 9.6), we conclude that,

$$\begin{aligned}
& |\Pr[b_{\mathbf{B}} = 0 \mid b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 0] - \Pr[b_{\mathbf{B}} = 0 \mid b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 1]| \\
&= \left| \sum_{y'_{\mathcal{B}} \in \{0,1\}^m} \mathbf{s}_{y'_{\mathcal{B}},*} \cdot \mathbf{p}_{y'_{\mathcal{B}},*} [y'_{\mathcal{B}},*] - \sum_{y'_{\mathcal{B}} \in \{0,1\}^m} \mathbf{s}_{y'_{\mathcal{B}},*} \cdot \mathbf{p}_{y'_{\mathcal{B}},* \rightarrow \perp} [y'_{\mathcal{B}},*] \right| \\
&= \left| \sum_{y'_{\mathcal{B}} \in \{0,1\}^m} \left(\mathbf{s}_{y'_{\mathcal{B}},*} \cdot \mathbf{p}_{y'_{\mathcal{B}},*} [y'_{\mathcal{B}},*] - \mathbf{s}_{y'_{\mathcal{B}},*} \cdot \mathbf{p}_{y'_{\mathcal{B}},* \rightarrow \perp} [y'_{\mathcal{B}},*] \right) \right| \\
&= \left| \sum_{y'_{\mathcal{B}} \in \{0,1\}^m} \mathbf{s}_{y'_{\mathcal{B}},*} \cdot \left(\mathbf{p}_{y'_{\mathcal{B}},*} [y'_{\mathcal{B}},*] - \mathbf{p}_{y'_{\mathcal{B}},* \rightarrow \perp} [y'_{\mathcal{B}},*] \right) \right| \\
&\leq \sum_{y'_{\mathcal{B}} \in \{0,1\}^m} \mathbf{s}_{y'_{\mathcal{B}},*} \cdot \left| \mathbf{p}_{y'_{\mathcal{B}},*} [y'_{\mathcal{B}},*] - \mathbf{p}_{y'_{\mathcal{B}},* \rightarrow \perp} [y'_{\mathcal{B}},*] \right| \quad \text{By triangle inequality.} \\
&\leq \sum_{y'_{\mathcal{B}} \in \{0,1\}^m} \mathbf{s}_{y'_{\mathcal{B}},*} \cdot \text{negl}(\lambda) \quad \text{By Lemma 9.7.} \\
&= \text{negl}(\lambda) \sum_{y'_{\mathcal{B}} \in \{0,1\}^m} \mathbf{s}_{y'_{\mathcal{B}},*} = \text{negl}(\lambda),
\end{aligned}$$

for some negligible function $\text{negl}(\cdot)$.

Note that we could use Lemma 9.7 in the last inequality since we are dealing with the challenge input distribution being $\mathcal{U}_{2(n)}$. Since for any $b_3, b_4 \in \{0,1\}$, $\Pr[b_{\mathbf{B}} = 0 \mid b_{\mathcal{B}} = b_3, b_{\mathcal{C}} = b_4] = \Pr[b_{\mathbf{B}} = 0, b_{\mathcal{C}} = 0 \mid b_{\mathcal{B}} = b_3, b_{\mathcal{C}} = b_4] + \Pr[b_{\mathbf{B}} = 0, b_{\mathcal{C}} = 1, b^{\mathcal{C}} = 0 \mid b_{\mathcal{B}} = b_3, b_{\mathcal{C}} = b_4]$, we conclude that,

$$|q_{00}^{00} + q_{00}^{01} - q_{01}^{00} + q_{01}^{01}| = |\Pr[b_{\mathbf{B}} = 0 \mid b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 0] - \Pr[b_{\mathbf{B}} = 0 \mid b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 1]| \leq \text{negl}(\lambda),$$

which implies that,

$$q_{01}^{01} \geq q_{00}^{00} + q_{00}^{01} - q_{01}^{00} - \text{negl}(\lambda). \quad (13)$$

Similarly, by a symmetric argument obtained by exchanging the roles of $\mathcal{B}$ and $\mathcal{C}$ in the derivation of Equation (13), we conclude that there exists a negligible function $\text{negl}'(\cdot)$ such that

$$q_{10}^{10} \geq q_{00}^{00} + q_{00}^{10} - q_{10}^{00} - \text{negl}'(\lambda). \quad (14)$$

Next, for any $b \in \{0,1\}$, by the generalized puncturing security guarantee (see Definition 6.2) of **GenPuncture** for the circuit class $\mathfrak{C}$, it is clear that in the copy-protection game, conditioned on the case $b_{\mathcal{B}} = 1, b_{\mathcal{C}} = b$, i.e., the circuit is punctured at $\mathcal{B}$'s challenge point, then the probability that $\mathcal{B}$ outputs correctly, i.e., the probability that $\mathcal{R}_{\mathcal{B}}$ outputs 0 conditioned on $b_{\mathcal{B}} = 1, b_{\mathcal{C}} = b$ in the independent UPO security game, is at most $p_{\text{triv}} + ^{12}$ negligible. Hence, there exists a negligible function $\text{negl}''(\cdot)$ such that

$$\Pr[b_{\mathbf{B}} = 0 \mid b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0] = q_{10}^{00} + q_{10}^{01} \leq p_{\text{triv}} + \text{negl}''(\lambda), \quad (15)$$

$$\Pr[b_{\mathbf{B}} = 0 \mid b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 1] = q_{11}^{00} + q_{11}^{01} \leq p_{\text{triv}} + \text{negl}''(\lambda). \quad (16)$$

¹²Note that this argument is implicitly using the fact that the p_{triv} defined in the copy-protection anti-piracy definition (Definition 8.1) is the same as the one defined in the generalized puncturing security definition (Definition 6.2).

By similar arguments we conclude that a similar bound must hold on the probabilities, $\Pr[b_C = 0 \mid b_B = 1, b_C = b]$, for every $b \in \{0, 1\}$. Hence, there exists a negligible function $\text{negl}'''(\cdot)$ such that

$$q_{01}^{00} + q_{01}^{10} \leq p_{\text{triv}} + \text{negl}'''(\lambda), \quad (17)$$

$$q_{11}^{00} + q_{11}^{10} \leq p_{\text{triv}} + \text{negl}'''(\lambda). \quad (18)$$

From Equation (15) and Equation (17), we conclude that,

$$q_{11}^{11} = 1 - q_{11}^{10} - q_{11}^{01} - q_{11}^{00} \geq 1 - (q_{11}^{00} + q_{11}^{10}) - (q_{11}^{00} + q_{11}^{01}) \geq 1 - 2p_{\text{triv}} - \text{negl}''(\lambda) - \text{negl}'''(\lambda). \quad (19)$$

Similarly, by combining Equation (15) with Equation (14), we get

$$q_{10}^{10} \geq q_{00}^{00} + q_{00}^{10} - q_{10}^{00} - \text{negl}'(\lambda) \geq q_{00}^{00} + q_{00}^{10} - (q_{10}^{00} + q_{10}^{01}) - \text{negl}'(\lambda) \geq q_{00}^{00} + q_{00}^{10} - p_{\text{triv}} - \text{negl}''(\lambda) - \text{negl}'(\lambda). \quad (20)$$

By a similar argument, by combining Equation (17) with Equation (13),

$$q_{01}^{01} \geq q_{00}^{00} + q_{00}^{01} - q_{01}^{00} - \text{negl}(\lambda) \geq q_{00}^{00} + q_{00}^{01} - (q_{01}^{00} + q_{01}^{10}) - \text{negl}(\lambda) \geq q_{00}^{00} + q_{00}^{01} - p_{\text{triv}} - \text{negl}'''(\lambda) - \text{negl}(\lambda). \quad (21)$$

Let $\widetilde{\text{negl}}(\lambda) := \min(\text{negl}(\lambda), \text{negl}'(\lambda), \text{negl}''(\lambda), \text{negl}'''(\lambda))$

Combining Equations (19) to (21), we conclude that,

$$\begin{aligned} & \frac{\sum_{b_1, b_2} q_{b_1 b_1}^{b_1 b_1} + q_{01}^{01} + q_{10}^{10} + q_{11}^{11}}{4} \\ & \geq \frac{q_{00}^{00} + 1 - 2p_{\text{triv}} - 2\widetilde{\text{negl}}(\lambda) + q_{00}^{00} + q_{00}^{10} - p_{\text{triv}} - 2\widetilde{\text{negl}}(\lambda) + q_{00}^{00} + q_{00}^{01} - p_{\text{triv}} - 2\widetilde{\text{negl}}(\lambda)}{4} \\ & = \frac{1 + 3q_{00}^{00} + q_{00}^{01} + q_{00}^{10} - 4p_{\text{triv}} - 6\widetilde{\text{negl}}(\lambda)}{4}. \end{aligned}$$

Comparing with Equation (10), we conclude that

$$\frac{1 + 3q_{00}^{00} + q_{00}^{01} + q_{00}^{10} - 4p_{\text{triv}} - 6\widetilde{\text{negl}}(\lambda)}{4} \leq \frac{1}{2} + \epsilon(\lambda),$$

which implies,

$$3q_{00}^{00} + q_{00}^{01} + q_{00}^{10} \leq 1 + 4p_{\text{triv}} + 6\widetilde{\text{negl}}(\lambda) + \epsilon(\lambda).$$

Since, $6\widetilde{\text{negl}}(\lambda) + \epsilon(\lambda)$ is a negligible function of λ , this concludes the proof of Claim 11.2, and hence completes the proof of the theorem. $\square$

11.1 1-out-of-2 Copy-protection for generalized puncturable functionalities from independent secure UPO

We show the following result in this section. Let $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ equipped with a distribution $\mathcal{D}_{\mathcal{F}}$ be a puncturable function class, consisting of functions mapping $\{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$. Let $(\mathcal{D}_{\mathcal{K}}, \mathfrak{C})$ be a keyed circuit implementation of $(\mathcal{D}_{\mathcal{F}}, \mathcal{F})$ where $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ is also a generalized keyed circuit class with input length $\{0, 1\}^{n(\lambda)}$ equipped with a puncturing algorithm GenPuncture , satisfying generalized puncturing security (see Definition 6.2). Let $\mathcal{U}_{2n(\lambda)}(1^\lambda)$ be the uniform distribution on $\{0, 1\}^{2n(\lambda)}$ such that $p_{\text{triv}} = p_{\text{triv}}(\lambda)$ with respect to $\mathcal{D}_{\mathcal{K}}$ and $\mathcal{U}_{2n(\lambda)}(1^\lambda)$ is a negligible function of λ .

Theorem 11.3. *Given any unclonable puncturable obfuscation $\text{UPO} = (\text{UPO}.\text{Obf}, \text{UPO}.\text{Eval})$ for $(\mathfrak{C}, \text{GenPuncture})$ that satisfies $\mathcal{U}_{2n(\lambda)}$ -independent-UPO security (as defined Definition 7.3), the copy-protection construction given in Construction 10.1 for $\mathcal{F}$, satisfies $(\mathcal{D}_{\mathcal{F}}, \mathcal{U}_{2n(\lambda)})$ -one-out-of-two-anti-piracy (see Definition 8.2) for $(\mathcal{D}_{\mathcal{F}}, \mathcal{F})$ implemented by $(\mathcal{D}_{\mathcal{K}}, \mathfrak{C})$.*

Proof. The correctness of the copy-protection scheme is immediate from the correctness of UPO. Let $\text{UPO}' = (\text{UPO}'.\text{Obf}, \text{UPO}'.\text{Eval})$ be defined as $\text{UPO}'.\text{Obf}(1^\lambda, C) = \text{UPO}(1^\lambda, i\mathcal{O}(1^\lambda, C))$ and $\text{UPO}'.\text{Eval} = \text{UPO}.\text{Eval}$ where $\text{UPO} = (\text{UPO}.\text{Obf}, \text{UPO}.\text{Eval})$ is the unclonable puncturable obfuscation scheme used in Construction 10.1. Note that since UPO satisfies independent-UPO security, then so does UPO' by the composition result for UPO with independent-UPO security, see Corollary 7.5. For security, we will use the same reduction as used in the proof of Theorem 11.1 to the independent-UPO security of UPO' . For completeness, we rewrite the reduction in Figure 21. Recall that for any quadruple of bits, $b_1, b_2, b_3, b_4 \in \{0, 1\}$, let $q_{b_3 b_4}^{b_1 b_2}$ denote the probability that the outputs of $\mathcal{R}_{\mathcal{B}}$ and $\mathcal{R}_{\mathcal{C}}$ are b_1 and b_2 , respectively, i.e., $b_{\mathbf{B}} = b_1$, and $b_{\mathbf{B}} = b_2$, conditioned on $b_{\mathcal{B}} = b_3$ and $b_{\mathcal{C}} = b_4$ in the independent UPO security game.

Moreover, note that by design of the reduction $(\mathcal{R}_{\mathcal{A}}, \mathcal{R}_{\mathcal{B}}, \mathcal{R}_{\mathcal{C}})$, the probability that $\mathcal{B}$ outputs correctly in the copy-protection security game (Figure 10) is the same as $\mathcal{R}_{\mathcal{B}}$ outputs 0 in the independent-UPO security game (Figure 9) conditioned on the challenger's bits satisfy $b_{\mathcal{B}} = 0 = b_{\mathcal{C}}$, i.e.,

$$\Pr \left[(1, \star) \leftarrow \text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda) \right] = \Pr[b_{\mathbf{B}} = 0 \mid b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 0] = q_{00}^{00} + q_{00}^{01}.$$

By a symmetric argument obtained by exchanging the roles of $\mathcal{B}$ and $\mathcal{C}$ in the derivation of the last equation we get,

$$\Pr \left[(\star, 1) \leftarrow \text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda) \right] = \Pr[b_{\mathbf{C}} = 0 \mid b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 0] = q_{00}^{00} + q_{00}^{10}.$$

Hence,

$$\begin{aligned} & \Pr \left[(1, \star) \leftarrow \text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda) \right] + \Pr \left[(\star, 1) \leftarrow \text{CP.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{K}}, \mathcal{D}_{\mathcal{X}^2}}(1^\lambda) \right] \\ &= q_{00}^{00} + q_{00}^{01} + q_{00}^{00} + q_{00}^{10} = 2q_{00}^{00} + q_{00}^{01} + q_{00}^{10}. \end{aligned}$$

Therefore, by the one-out-of-two-copy-protection security (Definition 8.2), it is enough to show that there exists a negligible function $\epsilon(\delta)$ such that,

$$2q_{00}^{00} + q_{00}^{01} + q_{00}^{10} \leq 1 + p_{\text{triv}} + \epsilon(\lambda).$$

Since we are dealing with function classes, such that $p_{\text{triv}} = p_{\text{triv}}(\lambda)$ is a negligible function in λ , to prove the last equation, it is enough to show that,

$$2q_{00}^{00} + q_{00}^{01} + q_{00}^{10} \leq 1 + \epsilon'(\lambda), \tag{22}$$

for some negligible function $\epsilon'(\lambda)$.

Note that by Claim 11.2 proved while proving Theorem 11.1, we get that

$$3q_{00}^{00} + q_{00}^{01} + q_{00}^{10} \leq 1 + 4p_{\text{triv}} + \delta'(\lambda), \tag{23}$$

for some negligible function $\delta'(\cdot)$. Since $p_{\text{triv}} = p_{\text{triv}}(\lambda)$ is negligible in λ , we conclude that,

$$3q_{00}^{00} + q_{00}^{01} + q_{00}^{10} \leq 1 + \delta''(\lambda), \tag{24}$$

where $\delta''(\lambda) := 4p_{\text{triv}}(\lambda) + \delta'(\lambda)$ is a negligible function of λ since $p_{\text{triv}}(\lambda)$ and $\delta'(\lambda)$ are negligible in λ .

Therefore by Equation (24), we get that

$$2q_{00}^{00} + q_{00}^{01} + q_{00}^{10} \leq 3q_{00}^{00} + q_{00}^{01} + q_{00}^{10} \leq 1 + \delta''(\lambda), \quad (25)$$

Since $\delta''(\lambda)$ is negligible in λ , the last equation implies Equation (22), and hence concludes the proof of the theorem. $\square$

$(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$:

- $\mathcal{R}_A$ samples $k \leftarrow \mathcal{D}_K(1^\lambda)$ and sends $k, \perp, \perp$ to Ch.
- Ch samples $b_B, b_C \xleftarrow{\$} \{0, 1\}$.
- Ch samples $x_B, x_C \leftarrow \mathcal{D}_{\mathcal{X}^2}(1^\lambda)$.
- Ch does the following:
 1. If $b_B = b_C = 0$: generate $\rho_k \leftarrow \text{UPO}'.\text{Obf}(1^\lambda, C_k)$.
 2. Else if $b_B = 1, b_C = 0$: generate $G_{k*}^{10} \leftarrow \text{GenPuncture}(k, x_B, \perp)$, and generate $\rho_k \leftarrow \text{UPO}'.\text{Obf}(1^\lambda, G_{k*}^{10})$.
 3. Else if $b_B = 0, b_C = 1$: generate $G_{k*}^{01} \leftarrow \text{GenPuncture}(k, x_C, \perp)$, and generate $\rho_k \leftarrow \text{UPO}'.\text{Obf}(1^\lambda, G_{k*}^{01})$.
 4. Else if $b_B = b_C = 1$: generate $G_{k*}^{11} \leftarrow \text{GenPuncture}(k, x_B, x_C, \perp, \perp)$, and generate $\rho_k \leftarrow \text{UPO}'.\text{Obf}(1^\lambda, G_{k*}^{11})$.
- Ch sends ρ_k to $\mathcal{R}_A$.
- $\mathcal{R}_A$ runs $\sigma_{B,C} \leftarrow \mathcal{A}(\rho_k)$. $\mathcal{R}_A$ outputs (σ_B, k) to $\mathcal{B}$ and (σ_C, k) to $\mathcal{C}$.
- $\mathcal{R}_B$ (respectively, $\mathcal{R}_C$) runs $y_B \leftarrow \mathcal{B}(\sigma_B)$ (respectively, $y_C \leftarrow \mathcal{C}(\sigma_C)$).
- $\mathcal{R}_B$ generates $y'_B \leftarrow C_k(x_B)$ and outputs $b_B = 0$ if $y'_B = y_B$, else $b_B = 1$. Similarly, $\mathcal{R}_C$ generates $y'_C \leftarrow C_k(x_C)$ and outputs $b_C = 0$ if $y'_C = y_C$, else $b_C = 1$.

Figure 21: Reduction from the anti-piracy experiment of CP to independent-UPO security experiment of UPO' .

12 Constructing UPO: A Modular Approach

In this section we will construct a UPO scheme satisfying independent-UPO security with respect to uniform challenge distribution from concrete assumptions, in two steps:

1. First in Section 12.1, we will show a generic construction of a UPO scheme satisfying $\mathcal{D}_{\chi^2}$ -independent-UPO security (see Definition 7.3) from any copy-protection for generalized puncturable signature satisfying $\mathcal{D}_{\chi^2}$ -anti-piracy (see Section 8.1) for any efficiently generatable challenge distribution $\mathcal{D}_{\chi^2}$ on pairs of inputs.
2. Next in Section 12.2.1 we show a construction of copy-protection for generalized puncturable signature satisfying anti-piracy with respect to the uniform challenge distribution on the pairs of inputs. The security proof of the construction is detailed in Section 12.3.1.

12.1 UPO from Copy-Protection for Generalized Puncturable Signatures

In this section, we follow the following result.

Theorem 12.1 (UPO with independent-UPO security from copy-protection for generalized puncturable signature). *Assuming there exists a copy-protection scheme for generalized puncturable signatures, there exists independent-generalized secure UPO for P/poly.*

Proof. Let $\text{CP.GPS} = (\text{KeyGen}, \text{CopyProtect}, \text{Sign}, \text{Verify}, \text{GenPuncture})$ be a copy-protection scheme for generalized puncturable signature, with message size $n(\lambda)$ and signature size $m(\lambda)$. We construct a UPO scheme $\text{UPO} = (\text{UPO.Obf}, \text{UPO.Eval})$ as follows.

1. $\text{UPO.Obf}(1^\lambda, W)$: The obfuscation algorithm, on input any W , does the following:
 - (a) Samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
 - (b) Computes $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$, where C is as defined in Figure 22.
 - (c) Computes the copy-protected state $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$.
 - (d) Outputs $\rho_C = (\hat{C}, \rho_{sk})$.

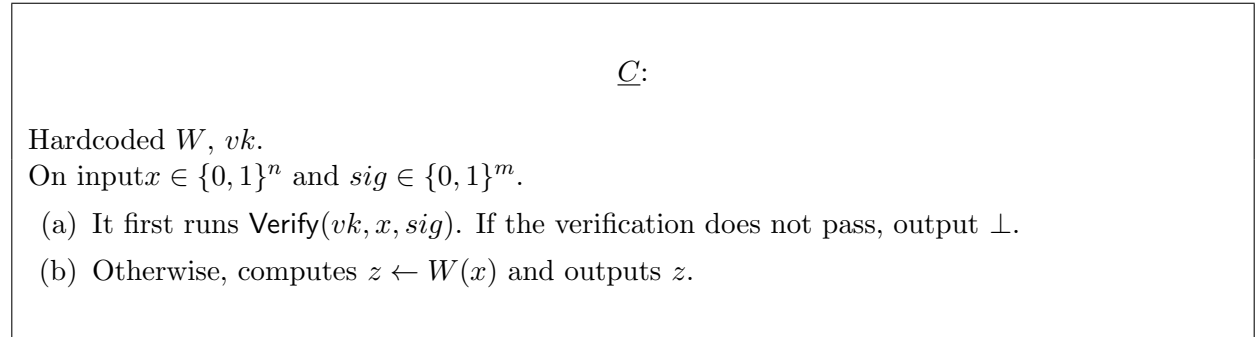


Figure 22: Circuit C in the construction of UPO from copy-protection for generalized puncturable signature.

2. $\text{UPO.Eval}(\rho, x)$: The evaluator algorithm takes an input $x \in \{0, 1\}^n$, and a state ρ , and does the following:
 - (a) Parses ρ as $\rho = (\hat{C}, \rho_{sk})$.
 - (b) Computes $\sigma \leftarrow \text{Sign}(\rho_{sk}, x)$.

- (c) Computes $z \leftarrow \widehat{C}(x, \sigma)$ and outputs z .

The correctness of the above construction is immediate from the correctness of CP.GPS. Next, we prove that $\text{UPO} = (\text{UPO.Obf}, \text{UPO.Eval})$ satisfies $\mathcal{D}_{\mathcal{X}}$ -independent-UPO security for every generalized puncturable circuit class in P/poly .

Fix a generalized keyed circuit class $\mathfrak{C} = \{\mathfrak{C}_\lambda\}_{\lambda \in \mathbb{N}}$ equipped with a puncturing algorithm GenPuncture , in P/poly . For any key k , we will denote the corresponding circuit as W_k . Let $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ be a QPT adversary in the independent-UPO security game

$(\text{IndUPO.Expt}^{(\mathcal{A}, \mathcal{B}, \mathcal{C}), \mathcal{D}_{\mathcal{X}}, \mathfrak{C}}(1^\lambda, b_{\mathcal{B}}, b_{\mathcal{C}}))$.

We do a separate hybrid argument for each of the four cases $b_{\mathcal{B}} = 0 = b_{\mathcal{C}}$, $b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0$, $b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 1$, and $b_{\mathcal{B}} = 1 = b_{\mathcal{C}}$.

$b_{\mathcal{B}} = 0 = b_{\mathcal{C}}$: This is the case when it is unpunctured for both $\mathcal{B}$ and $\mathcal{C}$.

H_0 : This is the original security game.

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
- Ch samples $x_{\mathcal{B}}, x_{\mathcal{C}} \leftarrow \mathcal{D}_{\mathcal{X}}(1^\lambda)$.
- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\widehat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 23.

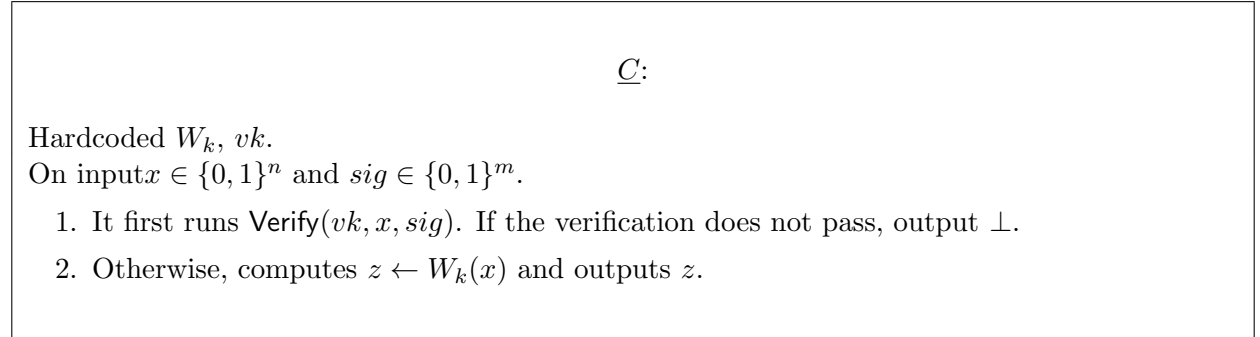


Figure 23: Circuit C in the H_0 for the unpunctured case.

- Ch sends $\rho = (\rho_{sk}, \widehat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{\mathcal{B}, \mathcal{C}}$ for $\mathcal{B}$ and $\mathcal{C}$.
- Ch sends $x_{\mathcal{B}}$ to $\mathcal{B}$ and $x_{\mathcal{C}}$ to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}})$ respectively, and outputs $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

H_1 : This is the original security game.

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.

- Ch samples $x_{\mathcal{B}}, x_{\mathcal{C}} \leftarrow \mathcal{D}_{\mathcal{X}}(1^\lambda)$.
- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 24.

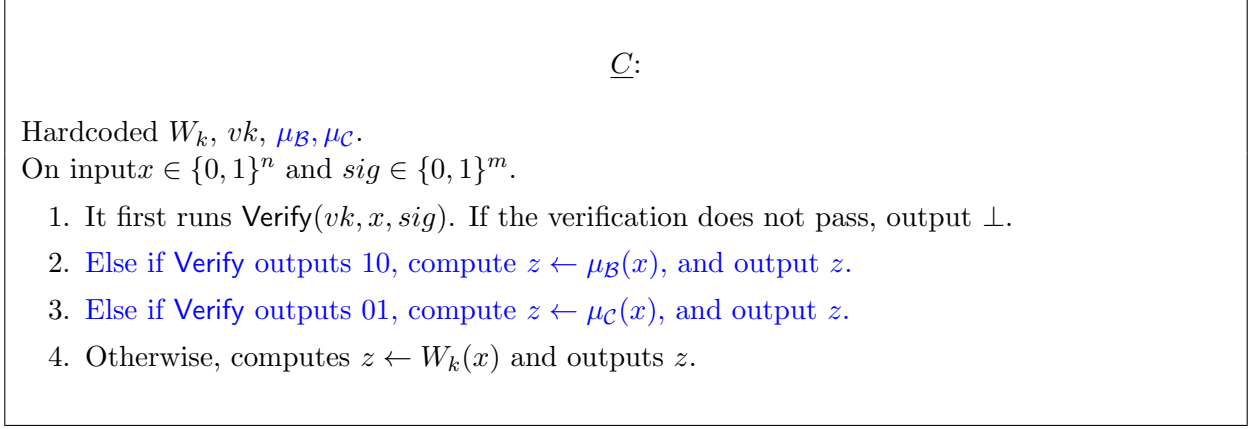


Figure 24: Circuit C in the H_0 for the unpunctured case.

- Ch sends $\rho = (\rho_{sk}, \hat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{\mathcal{B}, \mathcal{C}}$ for $\mathcal{B}$ and $\mathcal{C}$.
- Ch sends $x_{\mathcal{B}}$ to $\mathcal{B}$ and $x_{\mathcal{C}}$ to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}})$ respectively, and outputs $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

The functionality of the circuit C did not change across hybrids H_0 and H_1 , and hence indistinguishability follows.

Next we look at the $b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0$. We mark in red the changes between H_0 of this case and the H_0 of the $b_{\mathcal{B}} = b_{\mathcal{C}} = 0$ case.

$b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0$: This is the case when it is only punctured for $\mathcal{B}$ and unpunctured for $\mathcal{C}$. Consider the following hybrids. The changes across the hybrids are highlighted in blue. We also use the convention that if a (generalized) punctured key $vk_{x \rightarrow y}$ is hardcoded in a circuit, then every occurrence of the key inside the key that use the notation vk should be identified with the punctured key $vk_{x \rightarrow y}$.

H_0 :

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
- Ch samples $x_{\mathcal{B}}, x_{\mathcal{C}} \leftarrow \mathcal{D}_{\mathcal{X}}(1^\lambda, k)$.
- Ch generates $G_{k^*}^{10} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, \mu_{\mathcal{B}})$.
- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 25.

C :

Hardcoded $G_{k^*}^{10}, vk$.

On input $x \in \{0, 1\}^n$ and $sig \in \{0, 1\}^m$.

1. It first runs $\text{Verify}(vk, x, sig)$. If the verification does not pass, output $\perp$.
2. Otherwise, computes $z \leftarrow G_{k^*}^{10}(x)$ and outputs z .

Figure 25: Circuit C in the H_0 for the only punctured for $\mathcal{B}$ case.

- Ch sends $\rho = (\rho_{sk}, \hat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{\mathcal{B}, \mathcal{C}}$ for $\mathcal{B}$ and $\mathcal{C}$.
- Ch sends $x_{\mathcal{B}}$ to $\mathcal{B}$ and $x_{\mathcal{C}}$ to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}})$ respectively, and outputs $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

H_1 :

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
- Ch samples $x_{\mathcal{B}}, x_{\mathcal{C}} \leftarrow \mathcal{D}_{\mathcal{X}}(1^\lambda, k)$.
- Ch generates $G_{k^*}^{10} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, \mu_{\mathcal{B}})$.
- Ch generates $vk_{x_{\mathcal{B}} \rightarrow 10} \leftarrow \text{CP.GPS.GenPuncture}(sk, x_{\mathcal{B}}, 1_{10})$, where 1_{10} is the circuit that on all inputs $x \in \{0, 1\}^n$ outputs 10.
- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 26.
- Ch sends $\rho = (\rho_{sk}, \hat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{\mathcal{B}, \mathcal{C}}$ for $\mathcal{B}$ and $\mathcal{C}$.
- Ch sends $x_{\mathcal{B}}$ to $\mathcal{B}$ and $x_{\mathcal{C}}$ to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}})$ respectively, and outputs $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

The functionality of the circuit C did not change across hybrids H_0 and H_1 by the correctness of GenPuncture and $\text{CP.GPS.GenPuncture}$, and hence indistinguishability follows.

Next, we look at the $b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 1$ case, which is the same as the previous case up to changing the roles of $\mathcal{B}$ and $\mathcal{C}$. We mark in red the changes between H_0 of this case and the H_0 of the $b_{\mathcal{B}} = b_{\mathcal{C}} = 0$ case.

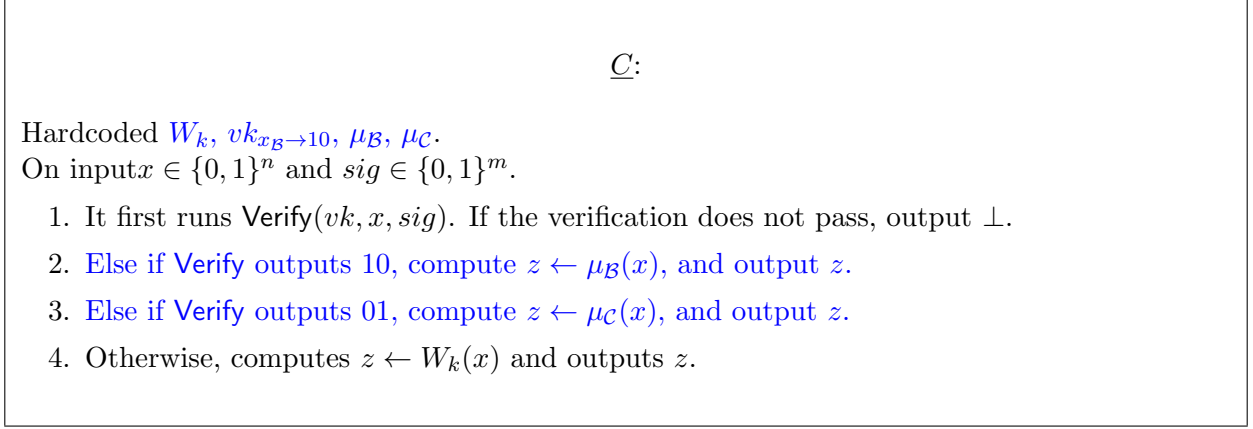


Figure 26: Circuit C in the H_1 for the only punctured for $\mathcal{B}$ case.

$b_B = 0, b_C = 1$: This is the case when it is only punctured for $\mathcal{C}$ and unpunctured for $\mathcal{B}$. Consider the following hybrids. The changes across the hybrids are highlighted in blue. We also use the convention that if a (generalized) punctured key $vk_{x \rightarrow y}$ is hardcoded in a circuit, then every occurrence of the key inside the key that use the notation vk should be identified with the punctured key $vk_{x \rightarrow y}$.

H_0 :

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits μ_B, μ_C .
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
- Ch samples $x_B, x_C \leftarrow \mathcal{D}_X(1^\lambda, k)$.
- Ch generates $G_{k^*}^{01} \leftarrow \text{GenPuncture}(k, x_C, \mu_C)$.
- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 27.

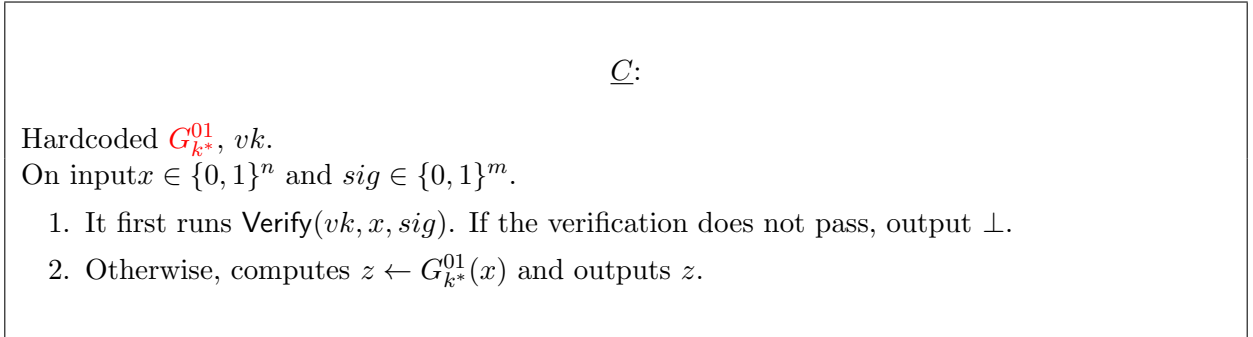


Figure 27: Circuit C in the H_0 for the only punctured for $\mathcal{C}$ case.

- Ch sends $\rho = (\rho_{sk}, \hat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{B,C}$ for $\mathcal{B}$ and $\mathcal{C}$.

- Ch sends $x_{\mathcal{B}}$ to $\mathcal{B}$ and $x_{\mathcal{C}}$ to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}})$ respectively, and outputs $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

H_1 :

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
- Ch samples $x_{\mathcal{B}}, x_{\mathcal{C}} \leftarrow \mathcal{D}_{\mathcal{X}}(1^\lambda, k)$.
- Ch generates $G_{k^*}^{01} \leftarrow \text{GenPuncture}(k, x_{\mathcal{C}}, \mu_{\mathcal{B}})$.
- Ch generates $vk_{x_{\mathcal{C}} \rightarrow 01} \leftarrow \text{CP.GPS.GenPuncture}(sk, x_{\mathcal{C}}, 1_{01})$, where 1_{01} is the circuit that on all inputs $x \in \{0, 1\}^n$ outputs 01.
- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 28.

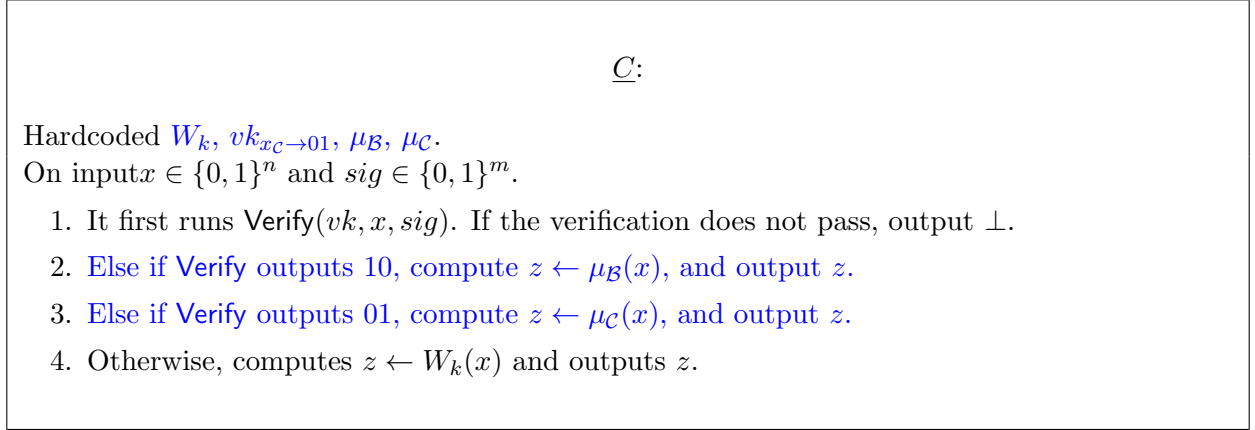


Figure 28: Circuit C in the H_1 for the only punctured for $\mathcal{C}$ case.

- Ch sends $\rho = (\rho_{sk}, \hat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{\mathcal{B}, \mathcal{C}}$ for $\mathcal{B}$ and $\mathcal{C}$.
- Ch sends $x_{\mathcal{B}}$ to $\mathcal{B}$ and $x_{\mathcal{C}}$ to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}})$ respectively, and outputs $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

The functionality of the circuit C did not change across hybrids H_0 and H_1 by the correctness of GenPuncture and $\text{CP.GPS.GenPuncture}$, and hence indistinguishability follows.

Next we look at the $b_{\mathcal{B}} = 1b_{\mathcal{C}} = 1$ case. We mark in red the changes between H_0 of this case and the H_0 of the $b_{\mathcal{B}} = b_{\mathcal{C}} = 0$ case.

$b_{\mathcal{B}} = b_{\mathcal{C}} = 1$: This is the case when it is punctured for both $\mathcal{B}$ and $\mathcal{C}$. Consider the following hybrids. The changes across the hybrids are highlighted in blue. We also use the convention that if a (generalized) punctured key $vk_{u \rightarrow y_u, v \rightarrow y_v}$ is hardcoded in a circuit, then every occurrence of the key inside the key that use the notation vk should be indentified with the punctured key $vk_{u \rightarrow y_u, v \rightarrow y_v}$.

H_0 :

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
- Ch samples $x_{\mathcal{B}}, x_{\mathcal{C}} \leftarrow \mathcal{D}_{\mathcal{X}}(1^\lambda, k)$.
- Ch generates $G_{k^*}^{11} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, x_{\mathcal{C}}, \mu_{\mathcal{B}}, \mu_{\mathcal{C}})$.
- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 29.

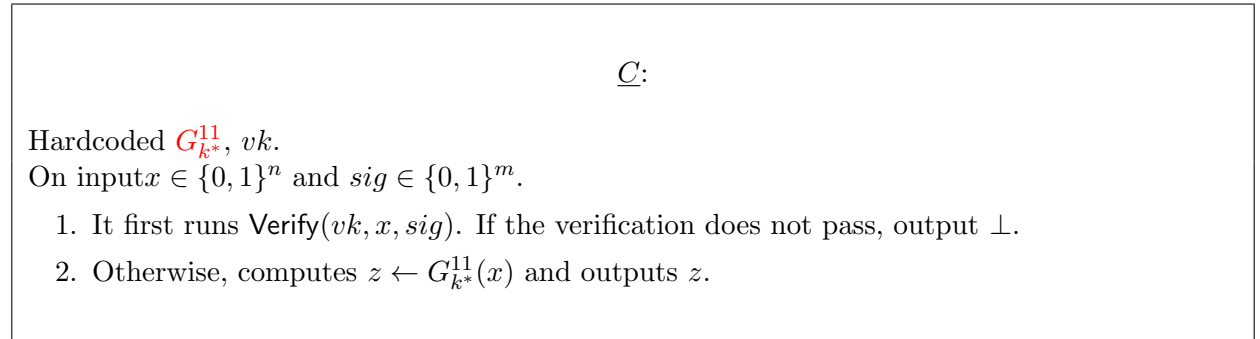


Figure 29: Circuit C in the H_0 for the punctured for both case.

- Ch sends $\rho = (\rho_{sk}, \hat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{\mathcal{B}, \mathcal{C}}$ for $\mathcal{B}$ and $\mathcal{C}$.
- Ch sends $x_{\mathcal{B}}$ to $\mathcal{B}$ and $x_{\mathcal{C}}$ to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}})$ respectively, and outputs $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

H_1 :

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
- Ch samples $x_{\mathcal{B}}, x_{\mathcal{C}} \leftarrow \mathcal{D}_{\mathcal{X}}(1^\lambda, k)$.
- Ch generates $G_{k^*}^{11} \leftarrow \text{GenPuncture}(k, x_{\mathcal{B}}, \mu_{\mathcal{B}})$.
- Ch generates $vk_{x_{\mathcal{B}} \rightarrow 10, x_{\mathcal{C}} \rightarrow 01} \leftarrow \text{CP.GPS.GenPuncture}(sk, x_{\mathcal{B}}, x_{\mathcal{C}}, 1_{10}, 1_{01})$, where 1_{01} and 1_{10} are the circuits that on all inputs $x \in \{0, 1\}^n$ outputs 01 and 10, respectively.

C :

Hardcoded $W_k, vk_{x_B \rightarrow 10, x_C \rightarrow 01}, \mu_B, \mu_C$.
 On input $x \in \{0, 1\}^n$ and $sig \in \{0, 1\}^m$.

1. It first runs $\text{Verify}(vk, x, sig)$. If the verification does not pass, output $\perp$.
2. Else if Verify outputs 10, compute $z \leftarrow \mu_B(x)$, and output z .
3. Else if Verify outputs 01, compute $z \leftarrow \mu_C(x)$, and output z .
4. Otherwise, computes $z \leftarrow W_k(x)$ and outputs z .

Figure 30: Circuit C in the H_1 for the punctured for both case.

- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 30.
- Ch sends $\rho = (\rho_{sk}, \hat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{B,C}$ for $\mathcal{B}$ and $\mathcal{C}$.
- Ch sends x_B to $\mathcal{B}$ and x_C to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input (σ_B, x_B) and (σ_C, x_C) respectively, and outputs b_B and b_C respectively.

The functionality of the circuit C did not change across hybrids H_0 and H_1 by the correctness of GenPuncture and $\text{CP.GPS.GenPuncture}$, and hence indistinguishability follows. Combining the respective H_1 of all the four cases $b_B, b_C \in \{0, 1\}$, we get the following hybrid: H^* :

- $\mathcal{A}$ sends a circuit key k , and another pair of circuits μ_B, μ_C .
- Ch samples $(sk, vk) \leftarrow \text{CP.GPS.KeyGen}(1^\lambda)$.
- Ch samples $x_B, x_C \leftarrow \mathcal{D}_X(1^\lambda, k)$.
- If $b_B = 0 = b_C$, then Ch sets $vk^* = vk$.
- Else if $b_B = 1, b_C = 0$, then Ch sets $vk^* = vk_{x_B \rightarrow 10}$ where $vk_{x_B \rightarrow 10} \leftarrow \text{CP.GPS.GenPuncture}(sk, x_B, 1_{10})$.
- Else if $b_B = 0, b_C = 1$, then Ch sets $vk^* = vk_{x_C \rightarrow 01}$ where $vk_{x_C \rightarrow 01} \leftarrow \text{CP.GPS.GenPuncture}(sk, x_B, 1_{01})$.
- Else if $b_B = b_C = 1$, then Ch sets $vk^* = vk_{x_B \rightarrow 10, x_C \rightarrow 01}$ where $vk_{x_B \rightarrow 10, x_C \rightarrow 01} \leftarrow \text{CP.GPS.GenPuncture}(sk, x_B, x_C, 1_{10}, 1_{01})$.
- Ch generates $\rho_{sk} \leftarrow \text{CopyProtect}(sk)$, and generates the circuit $\hat{C} \leftarrow i\mathcal{O}(1^\lambda, C)$ where C is as defined in Figure 31.

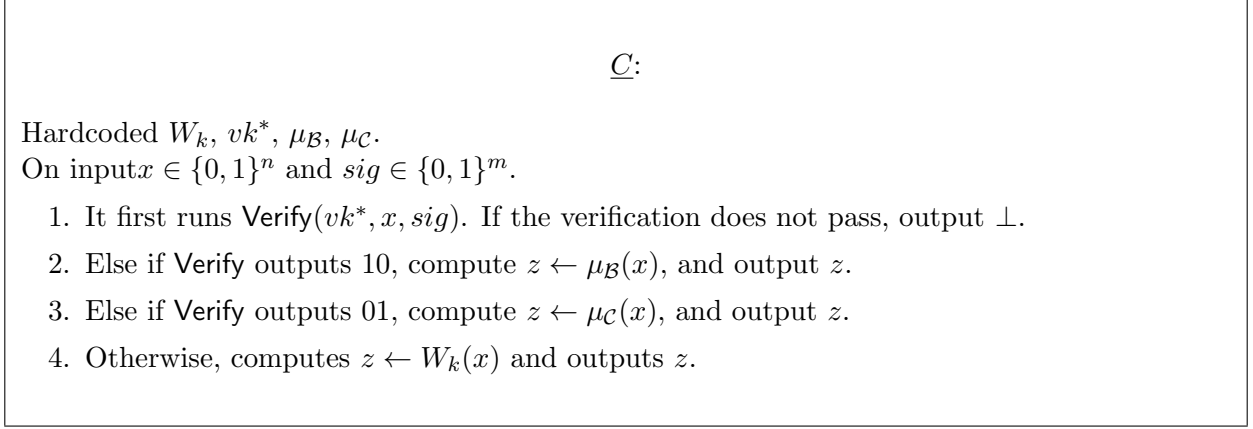


Figure 31: Circuit C in the H^* .

- Ch sends $\rho = (\rho_{sk}, \widehat{C})$ to $\mathcal{A}$, who then produces the state $\sigma_{B,C}$ for $\mathcal{B}$ and $\mathcal{C}$.
- Ch sends x_B to $\mathcal{B}$ and x_C to $\mathcal{C}$.
- $\mathcal{B}$ and $\mathcal{C}$ take as input (σ_B, x_B) and (σ_C, x_C) respectively, and outputs b_B and b_C respectively.

By the separate hybrid arguments for each of the four cases, we conclude that H^* is indistinguishable for the adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ from the original independent-UPO security game with respect to UPO (see Figure 9). Next we show a reduction from H^* to the $\mathcal{D}_X$ -anti-piracy of CP.GPS as defined in Section 8.1, to complete the security reduction to the $\mathcal{D}_X$ -anti-piracy of CP.GPS. Let $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ be an adversary in H^* , we construct a reduction $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ which breaks the $\mathcal{D}_X$ -anti-piracy game given in Figure 11, as follows:

1. $\mathcal{A}$ sends a circuit key k and a pair of circuits μ_B, μ_C to $\mathcal{R}_A$.
2. $\mathcal{R}_A$ sends $1_{01}, 1_{10}$ as the two circuits as the outputs circuits to indicate valid signatures at the punctured point, and gets back a signing program ρ_{sk} , and a verification key vk^* .
3. $\mathcal{R}_A$ uses the circuit at k , i.e., W_k along with vk^* and circuits μ_B, μ_C to construct the circuit C as described in Figure 31, and then generates $\widehat{C} \leftarrow iO(1^\lambda, C)$. $\mathcal{R}_A$ runs $\sigma_{B,C} \leftarrow \mathcal{A}(\rho, \widehat{C})$ and sends σ_B to $\mathcal{R}_B$ and σ_C to $\mathcal{R}_C$.
4. $\mathcal{R}_B$ gets as input x_B from Ch and σ_B from $\mathcal{A}$, and runs $b_B \leftarrow \mathcal{B}(\sigma_B, x_B)$, and outputs b_B .
5. $\mathcal{R}_C$ gets as input x_C from Ch and σ_C from $\mathcal{A}$, and runs $b_C \leftarrow \mathcal{B}(\sigma_C, x_C)$, and outputs b_C .

Clearly, the view of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in the simulated game of $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ is the same as H^* , and the challenge bits b_B, b_C of Ch in H^* corresponds to that of the challenge bits of the challenger Ch in the $\mathcal{D}_X$ -anti-piracy security game (see Figure 11) with respect to CP.GPS, and hence $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ win if and only if $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ win H^* , i.e., $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ break the $\mathcal{D}_X$ -anti-piracy with respect to CP.GPS with the same advantage as the adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in H^* , which completes the proof of the reduction, and hence the security proof. □

12.2 Copy-Protecting Generalized Puncturable Signatures

Our construction of the copy-protection scheme is the same as the one used in [LLQZ22], added with a puncturing algorithm for verification keys, and slightly different instantiations of the key ingredients. The copy-protection of generalized puncturable signature construction in [LLQZ22] is itself based on the copy-protection of PRF construction in [CLLZ21] and the signature scheme in [SW21]. We fix three families of PRF, F_1, F_2, F_3 , such that

1. A puncturable statistically injective PRF F_2 with input length $\ell_2(\lambda)$ and output length $\ell_1(\lambda)$ and failure probability $2^{-\lambda}$, such that $\ell_1(\lambda) \geq 2\ell_2(\lambda) + \lambda$.
2. A puncturable PRF F_3 that has input length ℓ_1 and output length ℓ_2 .
3. F_1 is a PRF that has input length $n(\lambda)$ and output length $m(\lambda)$, satisfying $n(\lambda) \geq m(\lambda) + 2\lambda + 4$.
4. Moreover, F_1 is a puncturable extracting PRF with error $2^{-(\lambda+1)}$ (with respect to computationally bounded algorithms)¹³ for min-entropy $k(\lambda) = n(\lambda) - \ell_0(\lambda) = \ell_1(\lambda) + \ell_2(\lambda)$.
5. Additionally¹⁴, F_1 satisfies that there exists a set $T \subset \{0, 1\}^{m(\lambda)}$ such that $\frac{|S|}{2^{m(\lambda)}}$ is a negligible function of λ , and that for every key k_1 , $\mathcal{I}(F_1(k_1, \cdot)) \subset S \subset \{0, 1\}^{m(\lambda)}$.

As in the case of [CLLZ21, LLQZ22], in our construction, we will parse the input x to PRF $F_1(K_1, \cdot)$ as three substrings $x_0 \| x_1 \| x_2$, where each x_i is of length ℓ_i for every $i \in \{0, 1, 2\}$ $0, 1, 2$ and $n(\lambda) = \ell_0(\lambda) + \ell_1(\lambda) + \ell_2(\lambda)$ and that for every $i \in \{0, 1, 2\}$, $\ell_i(\lambda)$ is a polynomial function of λ . $\ell_2(\lambda) - \ell_0(\lambda)$ must also be sufficiently large, and $n(\lambda) = \ell_1(\lambda) + \ell_2(\lambda)$. For brevity, we will often drop the security parameter λ while mentioning the parameters. For example, we will use m, n instead of $m(\lambda), n(\lambda)$, etc.

Remark 12.2. *The conditions on F_1, F_2, F_3 mentioned above are the same as the ones considered in [CLLZ21, LLQZ22] with an additional condition (Condition 5).*

Ref. [CLLZ21, LLQZ22] show how to build F_1, F_2, F_3 satisfying all but Condition 5, based on post-quantum one-way functions. Hence, in order to show that there is a construction of F_1, F_2, F_3 satisfying all conditions mentioned above, based on post-quantum one-way functions, it is enough to show that any PRF F_1 satisfying Condition 3 and Condition 4 can be lifted to a F_1 that satisfies all the three conditions (Conditions 3, 4, and 5), based on post-quantum one-way functions.

Lemma 12.3 (Lifting lemma for the existence of F_1). *Let PRG be a length-doubling PRG : $\{0, 1\}^{m(\lambda)/2} \rightarrow \{0, 1\}^{m(\lambda)}$. If there exists a PRF F satisfying Conditions 3 and 4, then $\text{PRG}(F_1^{\text{half}}(\cdot, \cdot))$ is a PRF satisfying Conditions 3, 4, and 5, where for tuple of key and input (k, x) $F^{\text{half}}(k, x)$ is the first $m(\lambda)/2$ bits of $F(k, x)$.*

¹³In [CLLZ21, LLQZ22], the authors considered information-theoretic extracting property, whereas we consider computational extracting property, since a) we observe that since we only need to consider QPT adversaries, computational extracting property is enough, b) working with computational extracting property makes it possible and relatively straightforward to also achieve for our purposes, Condition 5 as we do using the lifting lemma for F_1 (see Lemma 12.3).

¹⁴This is the only additional condition that is not required in the construction of [LLQZ22].

The proof is given in Appendix A on Page 98. Furthermore since F_1 is puncturable, WLOG we can equip F_1 with an algorithm **GenPuncture** that takes as input a key k , a pair of inputs (u, v) and circuits $\mu_{y_u, y_v}, \nu_{y_u, y_v}$ ¹⁵, where $y_u \leftarrow F_1(k, u)$ and $y_v \leftarrow F_1(k, v)$, and outputs $k_{u \rightarrow \mu_{y_u, y_v}(u), v \rightarrow \nu_{y_u, y_v}(v)}$, such that

1. **Correctness:** For every $k, u, v, \mu_{y_u, y_v}, \nu_{y_u, y_v}$, it must hold that if $u = v$

$$F_1(k_{u \rightarrow \mu_{y_u, y_v}(u), v \rightarrow \nu_{y_u, y_v}(v)}, x) = \begin{cases} F_1(k, x) & \text{if } x \neq u \\ \mu(u) & \text{if } x = u, \end{cases}$$

and if $u \neq v$,

$$F_1(k_{u \rightarrow \mu_{y_u, y_v}(u), v \rightarrow \nu_{y_u, y_v}(v)}, x) = \begin{cases} F_1(k, x) & \text{if } x \notin \{u, v\} \\ \mu(u) & \text{if } x = u. \\ \nu(v) & \text{if } x = v. \end{cases}$$

2. **Pseudorandomness at the punctured points:** For any fixed pair of points, u, v , and circuits μ, ν ,

$$\begin{aligned} & \{K_{u \rightarrow \mu_{Y_u, Y_v}(u), v \rightarrow \nu_{Y_u, Y_v}(v)}, Y_u, Y_v\}_{Y_u \leftarrow F_1(K, u), Y_v \leftarrow F_1(K, v), K \leftarrow F_1.\text{KeyGen}(1^\lambda)} \\ & \approx_c \{K_{u \rightarrow \mu_{Y_1, Y_2}(u), v \rightarrow \nu_{Y_1, Y_2}(v)}, Y_1, Y_2\}_{Y_1, Y_2 \xleftarrow{\$} \{0, 1\}^{m(\lambda)}, K \leftarrow F_1.\text{KeyGen}(1^\lambda)}. \end{aligned}$$

In simpler words, replacing the function values Y_u, Y_v under the unpunctured key K given in the clear and hardcoded in the circuits μ, ν for the punctured points with uniform random strings, Y_1, Y_2 is computationally indistinguishable.

Remark 12.4. Any generalized puncturable PRF can be equipped with a generalized puncturing algorithm **GenPuncture**, where on input a key k , a pair of inputs (u, v) and circuits $\mu_{y_u, y_v}, \nu_{y_u, y_v}$, first punctures the key k at u, v , and then uses the punctured key $k_{u, v}$ and circuits $\mu_{y_u, y_v}, \nu_{y_u, y_v}$ to construct the circuit $k_{u \rightarrow \mu_{y_u, y_v}(u), v \rightarrow \nu_{y_u, y_v}(v)}$ which has u, v, y_u, y_v hardcoded in it, and on any given input x , checks if $x \notin \{u, v\}$ and if so outputs $F_1(k_{u, v}, \cdot)$ else if $x = u$ it constructs the circuit μ_{y_u, y_v} using the hardcoded strings y_u, y_v , and then outputs $\mu_{y_u, y_v}(u)$ and if $x = v$ it constructs the circuit ν_{y_u, y_v} using the hardcoded strings y_u, y_v , and then outputs $\nu_{y_u, y_v}(v)$. By design, the algorithm satisfies the correctness property mentioned above, and by the pseudorandomness at the punctured point, the algorithm also satisfies pseudorandomness at punctured points, as mentioned above.

We will also require an injective one-way function $OWF : \{0, 1\}^{m(\lambda)} \rightarrow \{0, 1\}^{m'(\lambda)}$. We note that in [CLLZ21, LLQZ22], the authors used an one-way function from m -bits to m -bits that is not necessarily injective. For our purposes, the one-way function OWF needs to be injective, but the output size of OWF could be any polynomial greater than $m(\lambda)$. We note that such a OWF can be constructed from arbitrary post-quantum injective one-way functions. Moreover, as a signature scheme, our construction, just like the one in [LLQZ22], is a special case of the generic construction in [SW21] (i.e., same up to a different instantiation of the underlying PRF), and hence satisfies unforgeability as a signature scheme.

¹⁵The subscripts y_u, y_v indicate that the circuits can have y_u, y_v hardcoded in it.

12.2.1 Copy-protection for generalized puncturable signature construction.

The construction is as follows:

Keygen The $\text{KeyGen}(1^\lambda)$ algorithm does the following:

- Sample random PRF keys K_1, K_2, K_3 for F_1, F_2, F_3 respectively with security parameter λ .
- Constructs the circuit $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$, where VK is the circuit given in Figure 32 which has K_1, K_2, K_3 , and a one-way function f hardcoded in it.

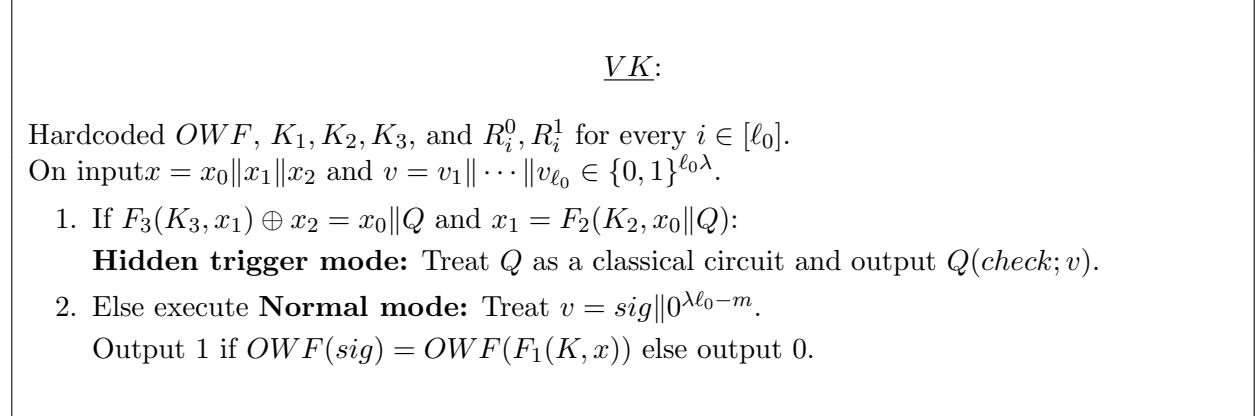


Figure 32: Circuit VK in the construction for copy-protection of generalized puncturable signature.

- Output $sk = (K_1, K_2, K_3)$ as the secret key, and $\widetilde{VK}$ as the public verification key.

Copy-protection algorithm The copy-protection algorithm, on input a signing key $sk = (K_1, K_2, K_3)$, does the following:

- Sample subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$. For every $i \in [\ell_0]$, generate the circuit $R_i^0 \leftarrow i\mathcal{O}(1^\lambda, 1_{A_i + s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1^\lambda, 1_{A_i + s'_i})$, where for any set $S \subset \mathbb{F}_2^n$, 1_S denotes the circuit that checks membership in S .
- Generate the circuit $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$, where CP-Sign is the circuit given in Figure 33.
- Output $\rho = (\{\{|A_{i s_i, s'_i}\rangle\}_i\}, \widetilde{\text{CP-Sign}})$.

Puncturing algorithm The puncturing algorithm takes a secret key $sk = (K_1, K_2, K_3)$, a pair of messages $u, v \in \{0, 1\}^n$, and circuit μ, ν , and outputs $\widetilde{VK}_{u \rightarrow \mu(u), v \rightarrow \nu(v)}$ where $\widetilde{VK}_{u \rightarrow \mu(u), v \rightarrow \nu(v)} \leftarrow i\mathcal{O}(1^\lambda, VK_{u \rightarrow \mu(u), v \rightarrow \nu(v)})$ where if $v = u$, $VK_{u \rightarrow \mu(u), v \rightarrow \nu(v)}$ is the circuit given in Figure 34 and if $v \neq u$, $VK_{u \rightarrow \mu(u), v \rightarrow \nu(v)}$ is the circuit given in Figure 35.

Clearly, functionality wise, for any u, v , irrespective of whether $u = v$ or not, $VK_{u \rightarrow \mu(u), v \rightarrow \nu(v)}$ as defined in Figure 34 or Figure 35 respectively, is the same as VK except for inputs of the form

CP-Sign:

Hardcoded K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(eval; v)$.

2. Else execute **Normal mode:** Output $F_1(K_1, x)$.

Figure 33: Circuit CP-Sign in the construction for copy-protection of generalized puncturable signature.

$VK_{u \rightarrow \mu(u), v \rightarrow \nu(v)}$:

Hardcoded OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, μ, ν and u, v .
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(check; v)$.

2. Else execute **Normal mode:** Treat $v = sig \| 0^{\lambda \ell_0 - m}$.

If $x \neq u$, output 1 if $OWF(sig) = OWF(F_1(K, x))$ else output 0.

Else if $x = u$, output $\mu(x)$ if $OWF(sig) = OWF(F_1(K, x))$ else output 0.

Figure 34: Circuit VK punctured at a point u in the construction for copy-protection of generalized puncturable signature.

(x', σ') such that $x' \in \{u, v\}$. For the correctness of puncturing (see the definition in Section 8.1), note that for any pair of inputs (u, v) if $u \neq v$, the only way puncturing guarantee does not hold for u, v if either u or v satisfies the hidden-triggers check given in Item 1 in Figure 35 (same as the check, in Item 1 in Figure 32). However, for any fixed $\tilde{x}$, and in particular for any fixed $x \in \{u, v\}$, the probability over the choice of K_2 , that x satisfies the hidden-triggers check is negligible by [CLLZ21, Lemma 7.13]. Hence, for any fixed u, v such that $u \neq v$, except for a negligible probability of the choice of K_2 , the correctness of puncturing holds. The same holds in the case when $u = v$, by a similar argument on Item 1 in Figure 34.

$$\underline{VK_{u \rightarrow \mu(u), v \rightarrow \nu(v)}}:$$

Hardcoded OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, μ, ν and u, v .
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.

2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.

If $x \notin \{u, v\}$, output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.

Else if $x = u$, output $\mu(x)$ if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.

Else if $x = v$, output $\nu(x)$ if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.

Figure 35: Circuit VK punctured at points u, v in the construction for copy-protection of generalized puncturable signature.

Signing algorithm The signing algorithm takes as input a copy-protection for signing key $\rho = (\{|A_{i s_i, s'_i}\rangle\}_i, \widetilde{\text{CP-Sign}})$ and a message x , and runs $\widetilde{\text{CP-Sign}}$ in superposition on $(x, \{|A_{i s_i, s'_i}\rangle\}_i)$, and measures the output register and outputs the outcome as the signature.

Lemma 12.5 (Correctness for the construction of copy-protection of puncturable signatures). *The construction of copy-protection for generalized puncturable signatures given in Section 12.2.1 satisfies statistical correctness.*

Proof. The correctness of the above construction follows from the correctness of the construction in [LLQZ22], since the construction is the same as the construction in [LLQZ22] (except that our instantiation of the primary PRF, F_1 satisfies an additional condition, namely Condition 5) which clearly does not affect the correctness of the scheme. $\square$

Theorem 12.6 (Anti-piracy for the construction of copy-protection of puncturable signatures). *Assuming the existence of post-quantum sub-exponentially secure $i\mathcal{O}$ and injective one-way functions, and the quantum hardness of Learning-with-errors problem (LWE), the construction of copy-protection for generalized puncturable signatures given in Section 12.2.1 satisfies $\mathcal{U}_{2n(\lambda)}$ -anti-piracy as defined in Section 8.1, where $\mathcal{U}_{2n(\lambda)}(1^\lambda)$ is the uniform distribution on $\{0, 1\}^{2n(\lambda)}$.*

The proof is given in Section 12.3.1.

Combining Theorem 12.6 with Lemma 12.5, we conclude the following corollary.

Corollary 12.7 (Existence of copy-protection from generalized puncturable signature from concrete assumptions). *Assuming the existence of post-quantum sub-exponentially secure $i\mathcal{O}$ and injective one-way functions, and the quantum hardness of Learning-with-errors problem (LWE), there exists a copy-protection for generalized puncturable signature satisfying $\mathcal{U}_{2n(\lambda)}$ -anti-piracy as defined in Section 8.1, where $\mathcal{U}_{2n(\lambda)}(1^\lambda)$ is the uniform distribution on $\{0, 1\}^{2n(\lambda)}$.*

12.3 Proof of antipiracy for the construction of Copy-protection for Signatures given in Section 12.2.1

Definition 12.8 (Gen-Trigger). *The Gen-Trigger algorithm takes as input a string $x = x_0 \in \{0, 1\}^{\ell_0}$, a string $y \in \{0, 1\}^m$, and keys K_2 and K_3 with respect to the PRF F_2 and F_3 , and circuits R_i^0, R_i^1 for every $i \in [\ell_0]$, respectively and does the following:*

1. Construct the program Q padded to $(\ell_2 - \ell_0)$ given in Figure 36 using $\{R_i^0, R_i^1\}_{i \in [\ell_0]}$.
2. Generate $x'_1 \leftarrow F_2(K_2, x_0 \| Q)$.
3. Generate $x'_2 \leftarrow F_3(K_3, x'_1) \oplus x_0 \| Q$.
4. Output $x' = x_0 \| x'_1 \| x'_2$.

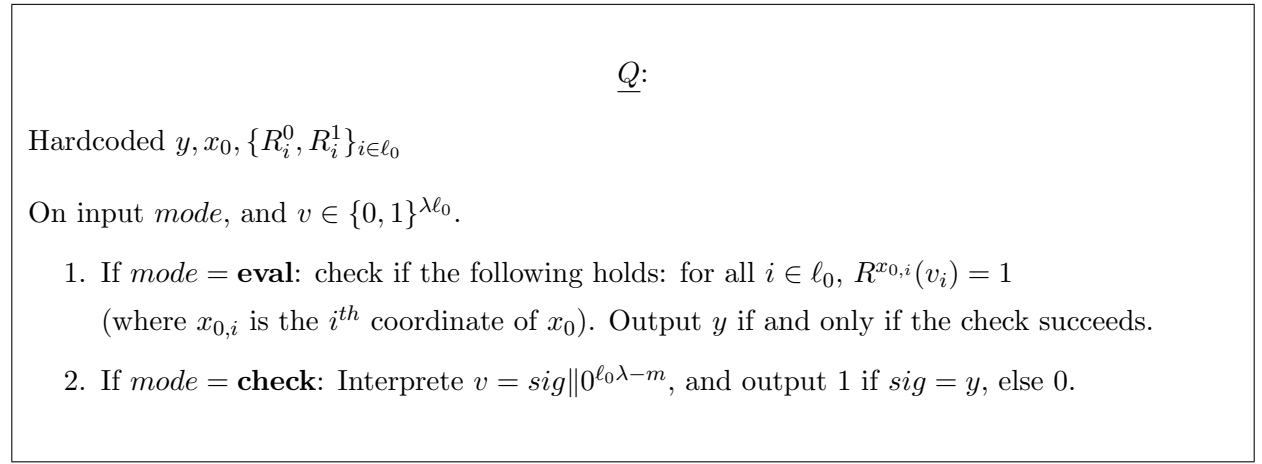


Figure 36: Circuit Q given in Definition 12.8.

Lemma 12.9 (Hidden-trigger lemma with auxiliary information, generalized from [LLQZ22, Lemma C.7]). *Assuming post-quantum $i\mathcal{O}$ and one-way functions, any efficient QPT algorithm $\mathcal{A}$ cannot win the following distinguishing game, except with non-negligible advantage:*

1. **Ch** samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \xleftarrow{\$} \{0, 1\}^n$, and computes $y_u \leftarrow F_1(K_1, u)$ and $y_v \leftarrow F_1(K_1, v)$.
2. **Ch** samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{ \{ |A_{i s_i, s'_i} \rangle \}_i \}$.
3. **Ch** parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$.
Then **Ch** generates $u' \leftarrow \mathbf{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.
Similarly, **Ch** generates $v' \leftarrow \mathbf{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.

4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(1^\lambda, V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1^\lambda, 1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1^\lambda, 1_{A^\perp+s'_i})$, and V is the circuit given in Figure 37, where $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ is a circuit that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(1^\lambda, P)$ where P is the circuit given in Figure 38, where $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ is a circuit that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
5. Ch samples $b \xleftarrow{\$} \{0, 1\}$, and if $b = 0$, sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$, sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

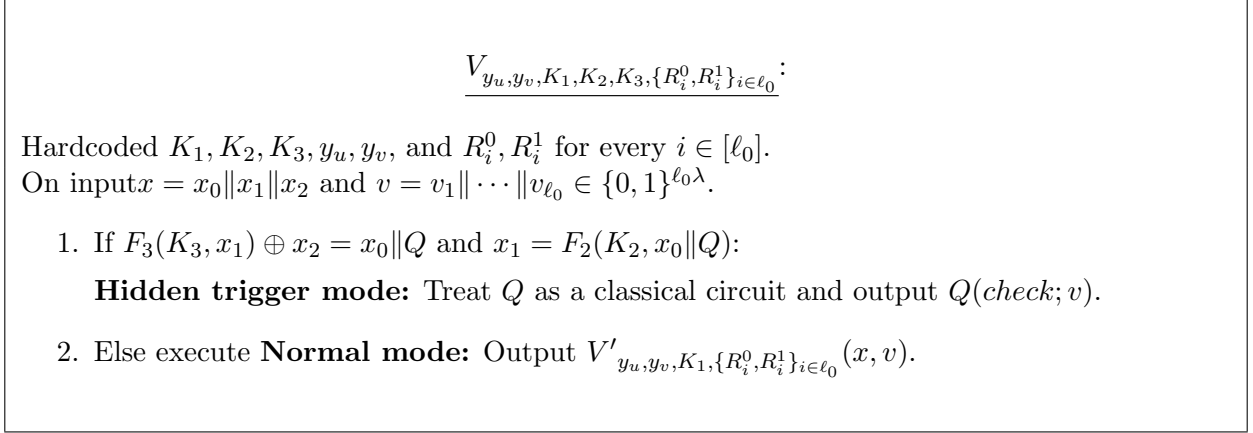


Figure 37: Circuit $V_{y_u, y_v, K_1, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ in Lemma 12.9.

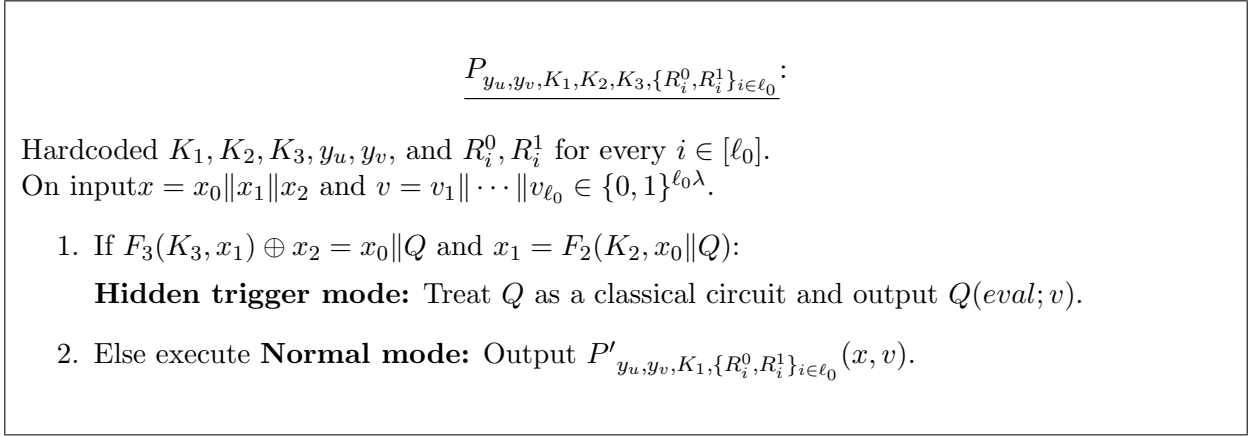


Figure 38: Circuit $P_{y_u, y_v, K_1, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ in Lemma 12.9.

The proof of Lemma 12.9 is given in Appendix B on Page 100.

12.3.1 Proof of Theorem 12.6

The copy-protection security game described in Figure 11 is a guessing game with an adversary $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ that has four cases: the unpunctured case (i.e., $b_{\mathcal{B}} = b_{\mathcal{C}} = 0$), punctured for $\mathcal{B}$ and unpunctured for $\mathcal{C}$ ($b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0$), punctured for $\mathcal{C}$ and unpunctured for $\mathcal{B}$ ($b_{\mathcal{C}} = 1, b_{\mathcal{B}} = 0$), and the both punctured case (i.e., $b_{\mathcal{B}} = b_{\mathcal{C}} = 1$) in Figure 11, and the adversary $\mathcal{B}$ and $\mathcal{C}$ needs to guess b . We will do separate hybrid arguments for each of the cases¹⁶ and then combine the corresponding last hybrids of the two cases to get a unified hybrid game, which we then reduce to the modified CPA style anti-piracy of [CLLZ21] which was proven in [LLQZ22, Proposition C.8].

We start with the unpunctured case.

Unpunctured Case. This is the $b_{\mathcal{B}} = b_{\mathcal{C}} = 0$ case in the copy-protection security game described in Figure 11. We will show the changes across the hybrids in blue.

Consider the following hybrids: H_0 : the original experiment.

The challenger gets as input, the circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.

VK :

Hardcoded OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.
 Output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.

Figure 39: Circuit VK in H_0 of the unpunctured case.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 40 and Figure 39, respectively.
4. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.

¹⁶We will analyze the $b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0$ and $b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 1$ case together since the two cases have symmetric hybrids, i.e., given the hybrid argument of one case, we can switch the names of $\mathcal{B}$ and $\mathcal{C}$ to get the hybrid argument for the other case.

CP-Sign:

Hardcoded K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(eval; v)$.
2. Else execute **Normal mode:** Output $F_1(K_1, x)$.

Figure 40: Circuit CP-Sign in H_0 of the unpunctured case.

5. Ch then samples $x_B^0, x_C^0 \xleftarrow{\$} \{0, 1\}^n$, and sends x_B^0 to $\mathcal{B}$ and x_C^0 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^0) and (σ_C, x_C^0) respectively, and outputs bits b_B and b_C respectively.

H_1 : we generate $\widetilde{VK}$ differently.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{|A_{is_i, s'_i}\rangle\}_i$.
3. Ch samples $z_1^B, z_1^C \xleftarrow{\$} \{0, 1\}^{2m}$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 40 and Figure 41, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
6. Ch then samples $x_B^0, x_C^0 \xleftarrow{\$} \{0, 1\}^n$, and sends x_B^0 to $\mathcal{B}$ and x_C^0 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^0) and (σ_C, x_C^0) respectively, and outputs bits b_B and b_C respectively.

Since the image set of PRG is a negligible fraction of $\{0, 1\}^{2m}$, with overwhelming probability over z_1^B, z_1^C , the functionality did not change; hence indistinguishability of the hybrids holds due to the $i\mathcal{O}$ security.

H_2 : we generate z_1^B, z_1^C differently.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{|A_{is_i, s'_i}\rangle\}_i$.

$\underline{VK}$:

Hardcoded $z_1^{\mathcal{B}}, z_1^{\mathcal{C}}, \text{PRG}, \text{OWF}, K_1, K_2, K_3$, and R_i^0, R_i^1 for every $i \in [\ell_0]$.
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.
 - (a) Generate $z \leftarrow \text{PRG}(\text{sig})$.
 - (b) If $z \notin \{z_1^{\mathcal{B}}, z_1^{\mathcal{C}}\}$, output 1 if $\text{OWF}(\text{sig}) = \text{OWF}(F_1(K, x))$ and 0 otherwise.
 - (c) Else if $z = z_1^{\mathcal{B}}$, output $\mu_{\mathcal{B}}(x)$ if $\text{OWF}(\text{sig}) = \text{OWF}(F_1(K, x))$ and 0 otherwise.
 - (d) Else if $z = z_1^{\mathcal{C}}$, output $\mu_{\mathcal{C}}(x)$ if $\text{OWF}(\text{sig}) = \text{OWF}(F_1(K, x))$ and 0 otherwise.

Figure 41: Circuit $\underline{VK}$ in H_1 of the unpunctured case.

3. Ch generates $z_1^{\mathcal{B}} \leftarrow \text{PRG}(y_{\mathcal{B}}^1), z_1^{\mathcal{C}} \leftarrow \text{PRG}(y_{\mathcal{C}}^1)$ where $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1 \leftarrow \{0, 1\}^m$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 40 and Figure 41, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch then samples $x_{\mathcal{B}}^0, x_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^n$, and sends $x_{\mathcal{B}}^0$ to $\mathcal{B}$ and $x_{\mathcal{C}}^0$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}}^0)$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}}^0)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

By the pseudorandomness of PRG , the indistinguishability of the hybrids holds.

H_3 : we generate $\widetilde{VK}$ differently.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1 \leftarrow \{0, 1\}^m$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 40 and Figure 42, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.

6. Ch then samples $x_{\mathcal{B}}^0, x_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^n$, and sends $x_{\mathcal{B}}^0$ to $\mathcal{B}$ and $x_{\mathcal{C}}^0$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}}^0)$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}}^0)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

VK:

Hardcoded $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1$, OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.

On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.

2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.

- (a) If $\text{sig} \notin \{y_{\mathcal{B}}^1, y_{\mathcal{C}}^1\}$, output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ and 0 otherwise.
- (b) Else if $\text{sig} = y_{\mathcal{B}}^1$, output $\mu_{\mathcal{B}}(x)$ if $OWF(\text{sig}) = OWF(F_1(K, x))$ and 0 otherwise.
- (c) Else if $\text{sig} = y_{\mathcal{C}}^1$, output $\mu_{\mathcal{C}}(x)$ if $OWF(\text{sig}) = OWF(F_1(K, x))$ and 0 otherwise.

Figure 42: Circuit VK in H_3 of the unpunctured case.

Since PRG is injective, the functionality did not change, and hence indistinguishability holds due to the $i\mathcal{O}$ security.

H_4 : The challenger generates $x_{\mathcal{B}}^0, x_{\mathcal{C}}^0$ differently.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1 \leftarrow \{0, 1\}^m$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 40 and Figure 42, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch samples $x_{\mathcal{B}}^0, x_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^n$.
7. Ch computes $y_{\mathcal{B}}^0 \leftarrow F_1(K_1, x_{\mathcal{B}}^0)$ and similarly computes $y_{\mathcal{C}}^0$, and then runs the **Gen-Trigger** algorithm (as defined in Definition 12.8) on the first ℓ_0 part of $x_{\mathcal{B}}^0$, and the first ℓ_0 part of $x_{\mathcal{C}}^0$ respectively, and the corresponding $y_{\mathcal{B}}^0$ and $y_{\mathcal{C}}^0$ to get a triggered input $x'_{\mathcal{B}}^0, x'_{\mathcal{C}}^0$ respectively, and sends $x'_{\mathcal{B}}^0, x'_{\mathcal{C}}^0$ to $\mathcal{B}$ and $\mathcal{C}$, respectively.

8. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x'_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x'_{\mathcal{C}})$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

The indistinguishability holds due to Lemma 12.9, which is the generalized form of the hidden-trigger lemma with auxiliary information [LLQZ22, Lemma C.7] where CP-Sign, VK plays the role of circuits P and V , and $x_{\mathcal{B}}^1$ and $x_{\mathcal{C}}^0$ plays the role of u and v respectively in Lemma 12.9.

H_5 : We generate $y_{\mathcal{B}}^0, y_{\mathcal{C}}^0$ differently.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{[A_{i s_i, s'_i}]\}_i$.
3. Ch samples $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1 \leftarrow \{0, 1\}^m$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 40 and Figure 42, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch samples $x_{\mathcal{B}}^0, x_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^n$.
7. Ch samples $y_{\mathcal{B}}^0, y_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^m$, and then runs the Gen-Trigger algorithm (as defined in Definition 12.8) on the first ℓ_0 part of $x_{\mathcal{B}}^0$, and the first ℓ_0 part of $x_{\mathcal{C}}^0$ respectively, and the corresponding $y_{\mathcal{B}}^0$ and $y_{\mathcal{C}}^0$ to get a triggered input $x'_{\mathcal{B}}^0, x'_{\mathcal{C}}^0$ respectively, and sends $x'_{\mathcal{B}}^0, x'_{\mathcal{C}}^0$ to $\mathcal{B}$ and $\mathcal{C}$, respectively.
8. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x'_{\mathcal{B}})$ and $(\sigma_{\mathcal{C}}, x'_{\mathcal{C}})$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

Note that in H_4 , the last $n - \ell_0$ bits of the string $x_{\mathcal{B}}^0$ and $x_{\mathcal{C}}^0$ were not revealed to $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$. Therefore, since $x_{\mathcal{B}}^0, x_{\mathcal{C}}^0$ were chosen uniformly at random from $\{0, 1\}^n$, there are at least $n - \ell_0$ bits of min-entropy left in the random strings $x_{\mathcal{B}}^0, x_{\mathcal{C}}^0$ from the viewpoint of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$. Since by design, $n - \ell_0 \geq k$ (see Condition 4), we conclude by the (computational) extracting property of F_1 , that $y_{\mathcal{B}}^0, y_{\mathcal{C}}^0$ in H_4 are computationally indistinguishable from uniformly and independently random strings sampled from $\{0, 1\}^m$. Therefore, H_4 and H_5 are indistinguishable for $(\mathcal{A}, \mathcal{B}, \mathcal{C})$.

Next we move to punctured for either only $\mathcal{B}$ or only $\mathcal{C}$ and unpunctured for the other, i.e., $b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0$ or $b_{\mathcal{C}} = 1, b_{\mathcal{B}} = 0$.

Punctured for only $\mathcal{B}$ or only $\mathcal{C}$ Case. First, we consider the case where $b_{\mathcal{B}} = 1$ and $b_{\mathcal{C}} = 0$. We will show the changes across the hybrids in blue. For the sake of brevity, we use the convention that if we hardcode a PRF key K_1 punctured at a point u (programmed to output the value y) in a circuit represented as $K_1 \setminus \{u \rightarrow y\}$ ¹⁷, then by every instruction inside the circuit that involves the key K_1 , we mean that the instruction is actually using the punctured key $K_1 \setminus \{u \rightarrow y\}$.

Consider the following hybrids: H_0 : the original experiment. The challenger gets as input, the circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.

VK:

Hardcoded OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, $\mu_{\mathcal{B}}$ and $x_{\mathcal{B}}^1$.
On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.

2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.

If $x \neq x_{\mathcal{B}}^1$, output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0. Else if $x = x_{\mathcal{B}}^1$, output $\mu_{\mathcal{B}}(x)$ if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.

Figure 43: Circuit VK in H_0 of the only punctured for $\mathcal{B}$ case.

CP-Sign:

Hardcoded K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.
On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{eval}; v)$.

2. Else execute **Normal mode:** Output $F_1(K_1, x)$.

Figure 44: Circuit $CP\text{-}Sign$ in H_0 of the only punctured for $\mathcal{B}$ case.

For the ease of comparison, the main differences of H_0 with the H_0 in the unpunctured case are highlighted in red.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples $x_{\mathcal{B}}^1 \xleftarrow{\$} \{0, 1\}^n$.
4. Ch then constructs the circuits $\widetilde{CP\text{-}Sign}$, and $\widetilde{VK}$ where $\widetilde{CP\text{-}Sign} \leftarrow i\mathcal{O}(1^\lambda, CP\text{-}Sign)$ and

¹⁷Here by the notation $u \rightarrow y$, we mean $u \rightarrow \mu(u)$ where μ is the constant circuit that always outputs y .

$\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 44 and Figure 43, respectively.

5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch samples $x_C^0 \xleftarrow{\$} \{0, 1\}^n$ and sends x_B^1 to $\mathcal{B}$ and x_C^0 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_B^1)$ and $(\sigma_{\mathcal{C}}, x_C^0)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

H_1 :

VK :

Hardcoded OWF , $K_1 \setminus \{x_B^1 \rightarrow y_B^1\}$, K_2 , K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, $\mu_{\mathcal{B}}$ and x_B^1 .
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.
 If $x \neq x_B^1$, output 1 if $OWF(\text{sig}) = OWF(F_1(K_1, x))$ else output 0.
 Else if $x = x_B^1$, output $\mu_{\mathcal{B}}(x)$ if $OWF(\text{sig}) = OWF(F_1(K_1, x))$ else output 0.

Figure 45: Circuit VK in H_1 of the only punctured for $\mathcal{B}$ case.

CP-Sign:

Hardcoded $K_1 \setminus \{x_B^1 \rightarrow y_B^1\}$, K_2 , K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{eval}; v)$.
2. Else execute **Normal mode:**
 Output $F_1(K_1, x)$.

Figure 46: Circuit CP-Sign in H_1 of the only punctured for $\mathcal{B}$ case.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.

2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{|A_{i s_i, s'_i}\rangle\}_i$.
3. Ch samples $x_B^1 \xleftarrow{\$} \{0, 1\}^n$. Ch generates $y_B^1 \leftarrow F_1(K, x_B^1)$. Ch also generates $K_1 \setminus \{x_B^1 \rightarrow y_B^1\} \leftarrow \text{GenPuncture}(K_1, \{x_B^1, x_B^1\}, \{y_B^1, y_B^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 46 and Figure 45, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
6. Ch samples $x_C^0 \xleftarrow{\$} \{0, 1\}^n$ and sends x_B^1 to $\mathcal{B}$ and x_C^0 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^1) and (σ_C, x_C^0) respectively, and outputs bits b_B and b_C respectively.

By the correctness of GenPuncture algorithm (see Remark 12.4), $F_1(K_1, \cdot)$ and $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1\}, \cdot)$ have the same functionality. Hence, the functionalities of VK did not change across H_0 and H_1 , so indistinguishability of the hybrids follows by the security of $i\mathcal{O}$.

H_2 : We sample the strings y_B^1, y_C^1 used in both $\widetilde{\text{CP-Sign}}$ and $\widetilde{VK}_{x_B^1, x_C^1}$ as $y_B^1, y_C^1 \xleftarrow{\$} \{0, 1\}^m$.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{|A_{i s_i, s'_i}\rangle\}_i$.
3. Ch samples x_B^1 . Ch samples $y_B^1 \xleftarrow{\$} \{0, 1\}^m$.
Ch also generates $K_1 \setminus \{x_B^1 \rightarrow y_B^1\} \leftarrow \text{GenPuncture}(K_1, \{x_B^1, x_B^1\}, \{y_B^1, y_B^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 46 and Figure 45, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
6. Ch samples $x_C^0 \xleftarrow{\$} \{0, 1\}^n$ and sends x_B^1 to $\mathcal{B}$ and x_C^0 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^1) and (σ_C, x_C^0) respectively, and outputs bits b_B and b_C respectively.

The indistinguishability of the hybrids follows by the pseudorandomness at the punctured point for GenPuncture with respect to F_1 and key K_1 punctured at x_B^1 with the circuit μ_B for the punctured point x_B^1 (see Remark 12.4).

H_3 : In $\widetilde{VK}$, we replace the check $x = x_B^1$ with the check $\sigma = y_B^1$.

1. $\mathcal{A}$ sends μ_B, μ_C .

VK:

Hardcoded OWF , $K_1 \setminus \{x_B^1 \rightarrow y_B^1\}$, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, μ_B and y_B^1 .
On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.
 If $\text{sig} \neq y_B^1$, output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.
 Else if $\sigma = y_B^1$.
 output $\mu_B(x)$ if $OWF(\text{sig}) = OWF(F_1(K_1, x))$ else output 0.

Figure 47: Circuit VK in H_3 of the only punctured for B case.

2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples x_B^1 . Ch samples $y_B^1 \xleftarrow{\$} \{0, 1\}^m$.
 Ch also generates $K_1 \setminus \{x_B^1 \rightarrow y_B^1\} \leftarrow \text{GenPuncture}(K_1, \{x_B^1, x_B^1\}, \{y_B^1, y_B^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 46 and Figure 47, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B, \mathcal{C}}$.
6. Ch samples $x_C^0 \xleftarrow{\$} \{0, 1\}^n$ and sends x_B^1 to $\mathcal{B}$ and x_C^0 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^1) and (σ_C, x_C^0) respectively, and outputs bits b_B and b_C respectively.

The indistinguishability of the hybrids follows by the fact that image of F_1 under any unpunctured or unpunctured key is a negligible fraction of $\{0, 1\}^m$ (by Condition 5 of the construction), and hence with overwhelming probability over the choice of y_B^1 in H_2 , the event *Good* that y_B^1 lie outside the image of $F_1(K_1, \cdot)$ restricted to the input set $\{0, 1\}^n \setminus \{x_B^1\}$, occurs. Hence conditioned on *Good*, by the correctness of **GenPuncture** algorithm, the only pre-image of y_B^1 under $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1\}, \cdot)$ is x_B^1 . Hence, the only string (x, sig) such that $x = x_B^1$ and $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1\}, x) = \text{sig}$ is (x_B^1, y_B^1) . Therefore, the set of (x, sig) such that $x = x_B^1$ and $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1\}, x) = \text{sig}$ is the set (x_B^1, y_B^1) which is the same as the set of (x, sig) such that $\text{sig} = y_B^1$ and $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1\}, x) = \text{sig}$. By the injectivity of OWF , this implies that the set of (x, sig) such that $x = x_B^1$ and $OWF(F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1\}, x)) = OWF(\text{sig})$ is the set (x_B^1, y_B^1) which

is the same as the set of (x, sig) such that $sig = y_B^1$ and $OWF(F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1\}, x)) = OWF(sig)$. Hence, conditioned on the event *Good* that holds except with negligible probability over the choice of y_B^1 , the functionality of VK did not change across H_2 and H_3 .

H_4 : We sample the strings y_B^1, y_C^1 as $y_B^1 \leftarrow F_1(K_1, x_B^1), y_C^1 \leftarrow F_1(K_1, x_C^1)$.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples x_B^1 . Ch generates $y_B^1 \leftarrow F_1(K_1, x_B^1)$.
 Ch also generates $K_1 \setminus \{x_B^1 \rightarrow y_B^1\} \leftarrow \text{GenPuncture}(K_1, \{x_B^1, x_B^1\}, \{y_B^1, y_B^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 46 and Figure 47, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
6. Ch samples $x_C^0 \xleftarrow{\$} \{0, 1\}^n$ and sends x_B^1 to $\mathcal{B}$ and x_C^0 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^1) and (σ_C, x_C^0) respectively, and outputs bits b_B and b_C respectively.

The indistinguishability between H_4 and H_3 holds by the same argument as used in the proof of the indistinguishability of H_1 and H_2 .

H_5 : We replace the key $K_{1 x_B^1 \rightarrow y_B^1}$ with the key K_1 in both CP-Sign and VK .

VK :

Hardcoded OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, μ_B and y_B^1 .
On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
2. Else execute **Normal mode:** Treat $v = sig \| 0^{\lambda \ell_0 - m}$.
If $sig \neq y_B^1$, output 1 if $OWF(sig) = OWF(F_1(K, x))$ else output 0.
Else if $\sigma = y_B^1$, output $\mu_B(x)$ if $OWF(sig) = OWF(F_1(K_1, x))$ else output 0.

Figure 48: Circuit VK in H_5 of the only punctured for $\mathcal{B}$ case.

1. $\mathcal{A}$ sends μ_B, μ_C .

CP-Sign:

Hardcoded K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.

On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(eval; v)$.

2. Else execute **Normal mode:**

Output $F_1(K_1, x)$.

Figure 49: Circuit CP-Sign in H_5 of the only punctured for $\mathcal{B}$ case.

2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{is_i, s'_i}\rangle\}_i\}$.
3. Ch samples x_B^1 . Ch generates $y_B^1 \leftarrow F_1(K_1, x_B^1)$.
Ch also generates $K_1 \setminus \{x_B^1 \rightarrow y_B^1\} \leftarrow \text{GenPuncture}(K_1, \{x_B^1, x_B^1\}, \{y_B^1, y_B^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow iO(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow iO(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 49 and Figure 48, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch samples $x_C^0 \xleftarrow{\$} \{0, 1\}^n$ and sends x_B^1 to $\mathcal{B}$ and x_C^0 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_B^1)$ and $(\sigma_{\mathcal{C}}, x_C^0)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

The indistinguishability between H_4 and H_5 holds by the same argument as used in the proof of the indistinguishability of H_0 and H_1 .

H_6 : we generate $\widetilde{VK}$ differently.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{is_i, s'_i}\rangle\}_i\}$.
3. Ch samples x_B^1 . Ch generates $y_B^1 \leftarrow F_1(K_1, x_B^1)$.
4. Ch samples $z_1^{\mathcal{C}} \xleftarrow{\$} \{0, 1\}^{2m}$.

5. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 49 and Figure 50, respectively.
6. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
7. Ch samples $x_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^n$ and sends $x_{\mathcal{B}}^1$ to $\mathcal{B}$ and $x_{\mathcal{C}}^0$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}}^0)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

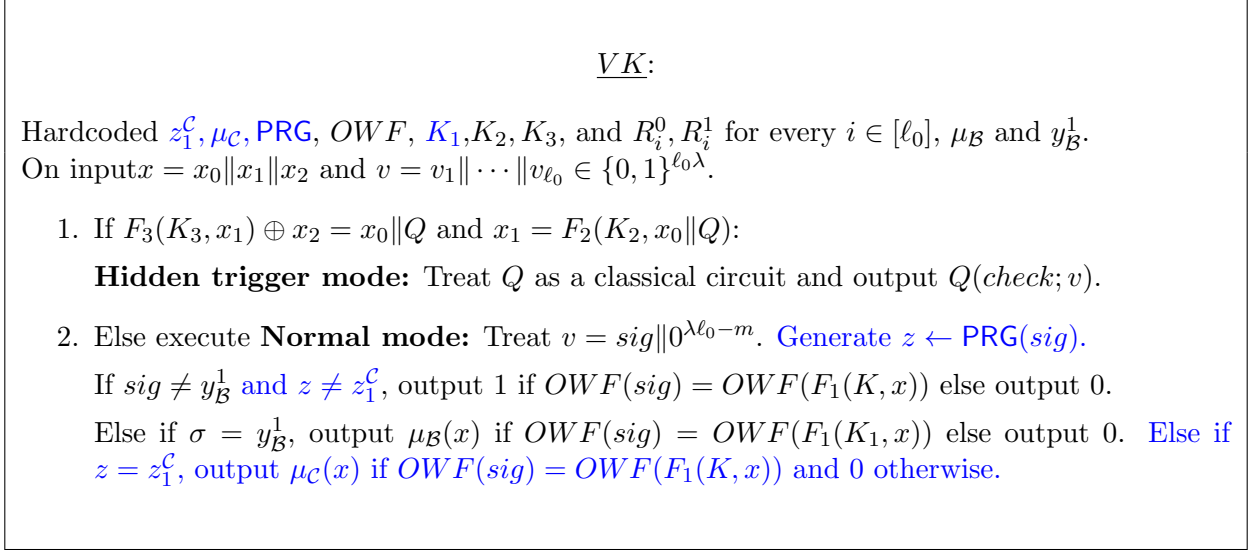


Figure 50: Circuit VK in H_6 of the only punctured for $\mathcal{B}$ case.

Since the image set of PRG is a negligible fraction of $\{0, 1\}^{2m}$, with overwhelming probability over $z_1^{\mathcal{C}}$, the functionality did not change; hence indistinguishability of the hybrids holds due to the $i\mathcal{O}$ security.

H_7 : we generate $z_1^{\mathcal{C}}$ differently.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{|A_{i s_i, s'_i}\rangle_i\}$.
3. Ch samples $x_{\mathcal{B}}^1$. Ch generates $y_{\mathcal{B}}^1 \leftarrow F_1(K_1, x_{\mathcal{B}}^1)$.
4. Ch generates $z_1^{\mathcal{C}} \leftarrow \text{PRG}(y_{\mathcal{C}}^1)$, where $y_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^m$.
5. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 49 and Figure 50, respectively.

6. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
7. Ch samples $x_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^n$ and sends $x_{\mathcal{B}}^1$ to $\mathcal{B}$ and $x_{\mathcal{C}}^0$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}}^0)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

By the pseudorandomness of PRG, the indistinguishability of the hybrids holds.
 H_8 : we generate $\widetilde{VK}$ differently.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i_{s_i, s'_i}}\rangle\}_i\}$.
3. Ch samples $x_{\mathcal{B}}^1$. Ch generates $y_{\mathcal{B}}^1 \leftarrow F_1(K_1, x_{\mathcal{B}}^1)$.
4. Ch samples $y_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^m$.
5. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 49 and Figure 51, respectively.
6. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
7. Ch samples $x_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^n$ and sends $x_{\mathcal{B}}^1$ to $\mathcal{B}$ and $x_{\mathcal{C}}^0$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}}^0)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

VK:

Hardcoded $y_{\mathcal{C}}^1, \mu_{\mathcal{C}}$, OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, $\mu_{\mathcal{B}}$ and $y_{\mathcal{B}}^1$.
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$. Generate $z \leftarrow \text{PRG}(\text{sig})$.
 If $\text{sig} \neq y_{\mathcal{B}}^1$ and $\text{sig} \neq y_{\mathcal{C}}^1$, output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.
 Else if $\sigma = y_{\mathcal{B}}^1$, output $\mu_{\mathcal{B}}(x)$ if $OWF(\text{sig}) = OWF(F_1(K_1, x))$ else output 0.
 Else if $\text{sig} = y_{\mathcal{C}}^1$, output $\mu_{\mathcal{C}}(x)$ if $OWF(\text{sig}) = OWF(F_1(K, x))$ and 0 otherwise.

Figure 51: Circuit VK in H_8 of the only punctured for $\mathcal{B}$ case.

Since PRG is injective, the functionality did not change, and hence indistinguishability holds due to the $i\mathcal{O}$ security.

H₉: The challenger generates x_B^1, x_C^0 differently.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples x_B^1 . Ch generates $y_B^1 \leftarrow F_1(K_1, x_B^1)$.
4. Ch samples $y_C^1 \xleftarrow{\$} \{0, 1\}^m$.
5. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 49 and Figure 51, respectively.
6. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
7. Ch samples $x_C^0 \xleftarrow{\$} \{0, 1\}^n$.
8. Ch computes $y_C^0 \leftarrow F_1(K_1, x_C^0)$, and then runs the Gen-Trigger algorithm (as defined in Definition 12.8) on the first ℓ_0 part of x_B^1 , and the first ℓ_0 part of x_C^0 respectively, and the corresponding y_B^1 and y_C^0 to get a triggered input $x_B'^1, x_C'^0$ respectively, and sends $x_B'^1, x_C'^0$ to $\mathcal{B}$ and $\mathcal{C}$ respectively.
9. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_B, x_B'^1)$ and $(\sigma_C, x_C'^0)$ respectively, and outputs bits b_B and b_C respectively.

The indistinguishability between H₈ and H₉ holds due to Lemma 12.9, which is the generalized form of the hidden-trigger lemma with auxiliary information [LLQZ22, Lemma C.7] where CP-Sign and VK plays the role of circuits P and V , and x_B^1 and x_C^0 plays the role of u and v respectively in Lemma 12.9.

H₁₀: We generate y_B^0, y_C^0 differently.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples x_B^1 . Ch samples $y_B^1 \xleftarrow{\$} \{0, 1\}^m$.
4. Ch samples $y_C^1 \xleftarrow{\$} \{0, 1\}^m$.
5. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 49 and Figure 51, respectively.

6. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
7. Ch samples $x_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^n$.
8. Ch samples $y_{\mathcal{C}}^0 \xleftarrow{\$} \{0, 1\}^m$, and then runs the Gen-Trigger algorithm (as defined in Definition 12.8) on the first ℓ_0 part of $x_{\mathcal{B}}^1$, and the first ℓ_0 part of $x_{\mathcal{C}}^0$ respectively, and the corresponding $y_{\mathcal{B}}^1$ and $y_{\mathcal{C}}^0$ to get a triggered input $x'_{\mathcal{B}}^1, x'_{\mathcal{C}}^0$ respectively, and sends $x'_{\mathcal{B}}^1, x'_{\mathcal{C}}^0$ to $\mathcal{B}$ and $\mathcal{C}$ respectively.
9. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x'_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x'_{\mathcal{C}}^0)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

Note that in H_9 , the last $n - \ell_0$ bits of the string $x_{\mathcal{B}}^1$ and $x_{\mathcal{C}}^0$ were not revealed to $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$. Therefore, since $x_{\mathcal{B}}^1, x_{\mathcal{C}}^0$ were chosen uniformly at random from $\{0, 1\}^n$, there are at least $n - \ell_0$ bits of min-entropy left in the random strings $x_{\mathcal{B}}^1, x_{\mathcal{C}}^0$ from the viewpoint of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$. Since by design, $n - \ell_0 \geq k$ (see Condition 4), we conclude by the (computational) extracting property of F_1 that $y_{\mathcal{B}}^1, y_{\mathcal{C}}^0$ in H_4 are computationally indistinguishable from uniformly and independently random strings sampled from $\{0, 1\}^m$. Therefore, H_9 and H_{10} are indistinguishable for $(\mathcal{A}, \mathcal{B}, \mathcal{C})$.

Next we move to punctured for either only $\mathcal{C}$ and unpunctured for $\mathcal{B}$ the other, i.e., $b_{\mathcal{C}} = 1, b_{\mathcal{B}} = 0$.

Next, we consider the case where $b_{\mathcal{B}} = 0$ and $b_{\mathcal{C}} = 1$. Consider the following hybrids:

H_0 : the original experiment.

The challenger gets as input, the circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.

VK :

Hardcoded OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, $\mu_{\mathcal{C}}$ and $x_{\mathcal{C}}^1$.
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.
 If $x \neq x_{\mathcal{C}}^1$, output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.
 Else if $x = x_{\mathcal{C}}^1$, output $\mu_{\mathcal{B}}(x)$ if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.

Figure 52: Circuit VK in H_0 of the only punctured for $\mathcal{C}$ case.

For the ease of comparison, the main differences of H_0 with the H_0 in the unpunctured case are highlighted in red.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.

CP-Sign:

Hardcoded K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(eval; v)$.

2. Else execute **Normal mode:** Output $F_1(K_1, x)$.

Figure 53: Circuit CP-Sign in H_0 of the only punctured for $\mathcal{C}$ case.

2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples $x_C^1 \xleftarrow{\$} \{0, 1\}^n$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 53 and Figure 52, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch samples $x_C^0 \xleftarrow{\$} \{0, 1\}^n$ and sends x_C^1 to $\mathcal{C}$ and x_B^0 to $\mathcal{B}$. $\mathcal{C}$ and $\mathcal{B}$ takes as input (σ_C, x_C^1) and (σ_B, x_B^0) respectively, and outputs bits b_C and b_B respectively.

Since the role of $\mathcal{B}, \mathcal{C}$ are symmetric in the copy-protection security game described in Figure 11, the case $b_C = 1, b_B = 0$ is the same as $b_B = 1, b_C = 0$, up to renaming of $\mathcal{B}$ and $\mathcal{C}$. Hence, by a symmetric version of the same hybrid argument as in the case of $b_B = 1, b_C = 0$, we conclude that H_0 is indistinguishable from the following hybrid.

H_1 :

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples x_C^1 . Ch samples $y_C^1 \xleftarrow{\$} \{0, 1\}^m$.
4. Ch samples $y_B^1 \xleftarrow{\$} \{0, 1\}^m$.

VK :

Hardcoded y_B^1, μ_B OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, μ_B and y_B^1 .
 On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$. Generate $z \leftarrow \text{PRG}(\text{sig})$.
 If $\text{sig} \neq y_C^1$ and $\text{sig} \neq y_B^1$, output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.
 Else if $\sigma = y_C^1$, output $\mu_C(x)$ if $OWF(\text{sig}) = OWF(F_1(K_1, x))$ else output 0.
 Else if $\text{sig} = y_B^1$, output $\mu_B(x)$ if $OWF(\text{sig}) = OWF(F_1(K, x))$ and 0 otherwise.

Figure 54: Circuit VK in H_1 of the only punctured for $\mathcal{C}$ case.

5. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 53 and Figure 54, respectively.
6. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B, \mathcal{C}}$.
7. Ch samples $x_B^0 \xleftarrow{\$} \{0, 1\}^n$.
8. Ch samples $y_B^0 \xleftarrow{\$} \{0, 1\}^m$, and then runs the **Gen-Trigger** algorithm (as defined in Definition 12.8) on the first ℓ_0 part of x_C^1 , and the first ℓ_0 part of x_B^0 respectively, and the corresponding y_C^1 and y_B^0 to get a triggered input $x_C'^1, x_B'^0$ respectively, and sends $x_C'^1, x_B'^0$ to $\mathcal{C}$ and $\mathcal{B}$ respectively.
9. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_B, x_B'^0)$ and $(\sigma_C, x_C'^1)$ respectively, and outputs bits b_B and b_C respectively.

Next, we move to the both-punctured case.

Both punctured Case. This is the $b_B = b_C = 1$ case in the copy-protection security game described in Figure 11. We will show the changes across the hybrids in blue. For the sake of brevity, we use the convention that if we hardcode a PRF key K_1 punctured at points u, v to output y_u, y_v respectively, in a circuit represented as $K_1 \setminus \{u \rightarrow y_u, v \rightarrow y_v\}$, then by every instruction inside the circuit that involves the key K_1 , we mean that the instruction is actually using the punctured key $K_1 \setminus \{u \rightarrow y_u, v \rightarrow y_v\}$.

Consider the following hybrids: H_0 : the original experiment.
 The challenger gets as input, the circuits μ_B, μ_C .

VK:

Hardcoded OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, μ_B, μ_C and x_B^1, x_C^1 .
On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(check; v)$.
2. Else execute **Normal mode:** Treat $v = sig \| 0^{\lambda \ell_0 - m}$.
 If $x \notin \{x_B^1, x_C^1\}$, output 1 if $OWF(sig) = OWF(F_1(K, x))$ else output 0.
 Else if $x = x_B^1$, output $\mu_B(x)$ if $OWF(sig) = OWF(F_1(K, x))$ else output 0.
 Else if $x = x_C^1$, output $\mu_C(x)$ if $OWF(sig) = OWF(F_1(K, x))$ else output 0.

Figure 55: Circuit VK in H_0 of the both punctured case.

CP-Sign:

Hardcoded K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.
On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(eval; v)$.
2. Else execute **Normal mode:** Output $F_1(K_1, x)$.

Figure 56: Circuit CP-Sign in H_0 of the both punctured case.

For the ease of comparison, the main differences of H_0 with the H_0 in the unpunctured case are highlighted in red.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples $x_B^1, x_C^1 \xleftarrow{\$} \{0, 1\}^n$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and

$\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 56 and Figure 55, respectively.

5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch sends $x_{\mathcal{B}}^1$ to $\mathcal{B}$ and $x_{\mathcal{C}}^1$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}}^1)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

H_1 :

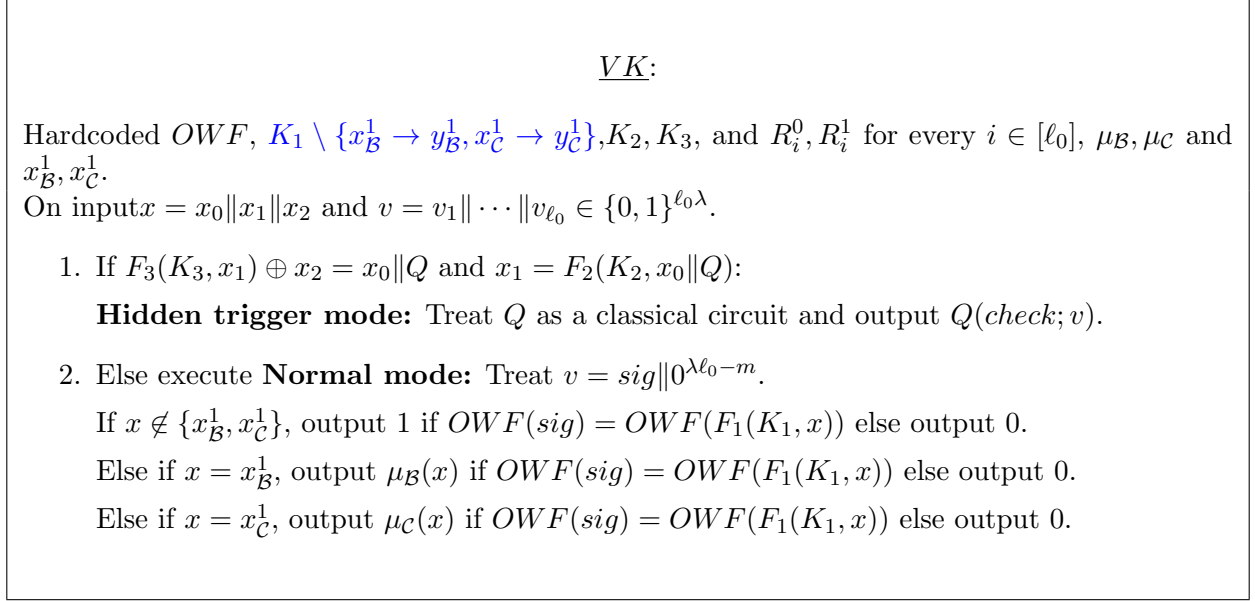


Figure 57: Circuit VK in H_1 of the both punctured case.

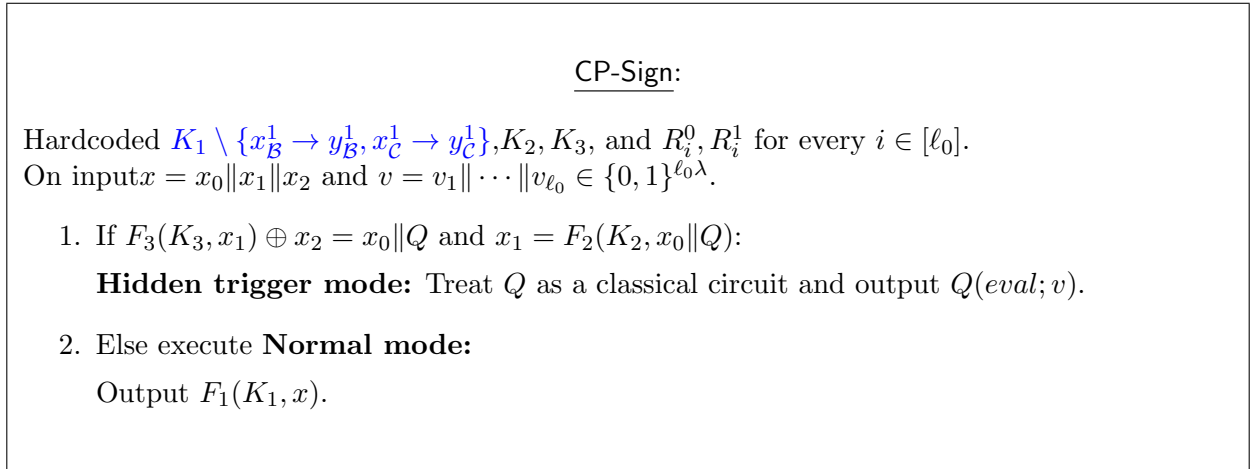


Figure 58: Circuit CP-Sign in H_1 of the both punctured case.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{ |A_{i s_i, s'_i} \rangle \}_i$.
3. Ch samples $x_{\mathcal{B}}^1, x_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^n$. Ch generates $y_{\mathcal{B}}^1 \leftarrow F_1(K, x_{\mathcal{B}}^1)$, $y_{\mathcal{C}}^1 \leftarrow F_1(K, x_{\mathcal{C}}^1)$. Ch also generates $K_1 \setminus \{x_{\mathcal{B}}^1 \rightarrow y_{\mathcal{B}}^1, x_{\mathcal{C}}^1 \rightarrow y_{\mathcal{C}}^1\} \leftarrow \text{GenPuncture}(K_1, \{x_{\mathcal{B}}^1, x_{\mathcal{C}}^1\}, \{y_{\mathcal{B}}^1, y_{\mathcal{C}}^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 58 and Figure 57, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch sends $x_{\mathcal{B}}^1$ to $\mathcal{B}$ and $x_{\mathcal{C}}^1$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}}^1)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

By the correctness of GenPuncture algorithm (see Remark 12.4), $F_1(K_1, \cdot)$ and $F_1(K_1 \setminus \{x_{\mathcal{B}}^1 \rightarrow y_{\mathcal{B}}^1, x_{\mathcal{C}}^1 \rightarrow y_{\mathcal{C}}^1\}, \cdot)$ have the same functionality. Hence, the functionalities of VK did not change across H_0 and H_1 , so indistinguishability of the hybrids follows by the security of $i\mathcal{O}$.

H_2 : We sample the strings $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1$ used in both $\widetilde{\text{CP-Sign}}$ and $\widetilde{VK}_{x_{\mathcal{B}}^1, x_{\mathcal{C}}^1}$ as $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^m$.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{ |A_{i s_i, s'_i} \rangle \}_i$.
3. Ch samples $x_{\mathcal{B}}^1, x_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^n$. Ch samples $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^m$. Ch also generates $K_1 \setminus \{x_{\mathcal{B}}^1 \rightarrow y_{\mathcal{B}}^1, x_{\mathcal{C}}^1 \rightarrow y_{\mathcal{C}}^1\} \leftarrow \text{GenPuncture}(K_1, \{x_{\mathcal{B}}^1, x_{\mathcal{C}}^1\}, \{y_{\mathcal{B}}^1, y_{\mathcal{C}}^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 58 and Figure 57, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch sends $x_{\mathcal{B}}^1$ to $\mathcal{B}$ and $x_{\mathcal{C}}^1$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x_{\mathcal{C}}^1)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

The indistinguishability of the hybrids follows by the pseudorandomness at the punctured point for GenPuncture with respect to F_1 and key K_1 punctured at $x_{\mathcal{B}}^1, x_{\mathcal{C}}^1$ with respect to the circuits $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$ respectively for the punctured points $x_{\mathcal{B}}^1, x_{\mathcal{C}}^1$ (see Remark 12.4).

H_3 : In $\widetilde{VK}$, we replace the check $x \in \{x_{\mathcal{B}}^1, x_{\mathcal{C}}^1\}$ with the check $\sigma \in \{y_{\mathcal{B}}^1, y_{\mathcal{C}}^1\}$.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.

VK:

Hardcoded OWF , $K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}$, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, μ_B, μ_C and y_B^1, y_C^1 .

On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:

Hidden trigger mode: Treat Q as a classical circuit and output $Q(\text{check}; v)$.

2. Else execute **Normal mode:** Treat $v = \text{sig} \| 0^{\lambda \ell_0 - m}$.

If $\text{sig} \notin \{y_B^1, y_C^1\}$, output 1 if $OWF(\text{sig}) = OWF(F_1(K, x))$ else output 0.

Else if $\sigma = y_B^1$, output $\mu_B(x)$ if $OWF(\text{sig}) = OWF(F_1(K_1, x))$ else output 0.

Else if $\sigma = y_C^1$, output $\mu_C(x)$ if $OWF(\text{sig}) = OWF(F_1(K_1, x))$ else output 0.

Figure 59: Circuit VK in H_3 of the both punctured case.

2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{ |A_{i s_i, s'_i} \rangle \}_i$.
3. Ch samples $x_B^1, x_C^1 \xleftarrow{\$} \{0, 1\}^n$. Ch samples $y_B^1, y_C^1 \xleftarrow{\$} \{0, 1\}^m$. Ch also generates $K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\} \leftarrow \text{GenPuncture}(K_1, \{x_B^1, x_C^1\}, \{y_B^1, y_C^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 58 and Figure 59, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
6. Ch sends x_B^1 to $\mathcal{B}$ and x_C^1 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^1) and (σ_C, x_C^1) respectively, and outputs bits b_B and b_C respectively.

The indistinguishability of the hybrids follows by the fact that image of F_1 under any unpunctured or unpunctured key is a negligible fraction of $\{0, 1\}^m$ (by Condition 5 of the construction), and hence with overwhelming probability over the choice of y_B^1, y_C^1 in H_2 , the event *Good* that y_B^1, y_C^1 are distinct and lie outside the image of $F_1(K_1, \cdot)$ restricted to the input set $\{0, 1\}^n \setminus \{x_B^1, x_C^1\}$ occurs. Hence conditioned on *Good*, by the correctness of **GenPuncture** algorithm, the only pre-image of y_B^1, y_C^1 under $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}, \cdot)$ are x_B^1, x_C^1 respectively. Hence, the only string (x, sig) such that $x = x_B^1$ and $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}, x) = \text{sig}$ is x_B^1, y_B^1 . Therefore, the set of (x, sig) such that $x = x_B^1$ and $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}, x) = \text{sig}$ is the set (x_B^1, y_B^1) which is the same as the set of (x, sig) such that $\text{sig} = y_B^1$ and $F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}, x) = \text{sig}$. By the injectivity of OWF , this implies that the set of (x, sig) such that $x = x_B^1$ and $OWF(F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}, x)) = \text{sig}$ is the set (x_B^1, y_B^1) .

$y_C^1\}, x)) = \text{OWF}(\text{sig})$ is the set (x_B^1, y_B^1) which is the same as the set of (x, sig) such that $\text{sig} = y_B^1$ and $\text{OWF}(F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}, x)) = \text{OWF}(\text{sig})$. By a symmetric argument for $\mathcal{C}$, it holds that the set of (x, sig) such that $x = x_C^1$ and $\text{OWF}(F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}, x)) = \text{OWF}(\text{sig})$ is x_C^1, y_C^1 is the set (x_C^1, y_C^1) same as the set of (x, sig) such that $\text{sig} = y_C^1$ and $\text{OWF}(F_1(K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\}, x)) = \text{OWF}(\text{sig})$. Hence, conditioned on the event *Good* that holds except with negligible probability over the choice of y_B^1, y_C^1 , the functionality of VK did not change across H_2 and H_3 .

H_4 : We sample the strings y_B^1, y_C^1 as $y_B^1 \leftarrow F_1(K_1, x_B^1), y_C^1 \leftarrow F_1(K_1, x_C^1)$.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{|A_{i s_i, s'_i}\rangle_i\}$.
3. Ch samples $x_B^1, x_C^1 \xleftarrow{\$} \{0, 1\}^n$. Ch generates $y_B^1 \leftarrow F_1(K_1, x_B^1), y_C^1 \leftarrow F_1(K_1, x_C^1)$. Ch also generates $K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\} \leftarrow \text{GenPuncture}(K_1, \{x_B^1, x_C^1\}, \{y_B^1, y_C^1\})$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 58 and Figure 59, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
6. Ch sends x_B^1 to $\mathcal{B}$ and x_C^1 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^1) and (σ_C, x_C^1) respectively, and outputs bits b_B and b_C respectively.

The indistinguishability between H_4 and H_3 holds by the same argument as used in the proof of the indistinguishability of H_1 and H_2 .

H_5 : We replace the key $K_1_{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1}$ with the key K_1 in both CP-Sign and VK .

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{|A_{i s_i, s'_i}\rangle_i\}$.
3. Ch samples $x_B^1, x_C^1 \xleftarrow{\$} \{0, 1\}^n$. Ch generates $y_B^1 \leftarrow F_1(K_1, x_B^1), y_C^1 \leftarrow F_1(K_1, x_C^1)$. ~~Ch also generates $K_1 \setminus \{x_B^1 \rightarrow y_B^1, x_C^1 \rightarrow y_C^1\} \leftarrow \text{GenPuncture}(K_1, \{x_B^1, x_C^1\}, \{y_B^1, y_C^1\})$.~~
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 61 and Figure 60, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
6. Ch sends x_B^1 to $\mathcal{B}$ and x_C^1 to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x_B^1) and (σ_C, x_C^1) respectively, and outputs bits b_B and b_C respectively.

VK:

Hardcoded OWF , K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$, μ_B, μ_C and y_B^1, y_C^1 .
On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(check; v)$.
2. Else execute **Normal mode:** Treat $v = sig \| 0^{\lambda \ell_0 - m}$.
 If $sig \notin \{y_B^1, y_C^1\}$, output 1 if $OWF(sig) = OWF(F_1(K, x))$ else output 0.
 Else if $\sigma = y_B^1$, output $\mu_B(x)$ if $OWF(sig) = OWF(F_1(K_1, x))$ else output 0.
 Else if $\sigma = y_C^1$, output $\mu_C(x)$ if $OWF(sig) = OWF(F_1(K_1, x))$ else output 0.

Figure 60: Circuit VK in H_5 of the both punctured case.

CP-Sign:

Hardcoded K_1, K_2, K_3 , and R_i^0, R_i^1 for every $i \in [\ell_0]$.
On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
Hidden trigger mode: Treat Q as a classical circuit and output $Q(eval; v)$.
2. Else execute **Normal mode:**
 Output $F_1(K_1, x)$.

Figure 61: Circuit $CP\text{-}Sign$ in H_5 of the both punctured case.

The indistinguishability between H_4 and H_5 holds by the same argument as used in the proof of the indistinguishability of H_0 and H_1 .

H_6 : We replace x_B^1, x_C^1 that $\mathcal{B}$ and $\mathcal{C}$ receives in the challenge phase with $x_B'^0, x_C'^0$ as follows.

1. $\mathcal{A}$ sends μ_B, μ_C .
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch samples $x_B^1, x_C^1 \xleftarrow{\$} \{0, 1\}^n$. Ch generates $y_B^1 \leftarrow F_1(K_1, x_B^1), y_C^1 \leftarrow F_1(K_1, x_C^1)$.

4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 61 and Figure 60, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch runs the Gen-Trigger algorithm (as defined in Definition 12.8) on the first ℓ_0 part of $x_{\mathcal{B}}^1$, and the first ℓ_0 part of $x_{\mathcal{C}}^1$ respectively, and the corresponding $y_{\mathcal{B}}^1$ and $y_{\mathcal{C}}^1$ to get a triggered input $x'_{\mathcal{B}}^1, x'_{\mathcal{C}}^1$ respectively, and sends $x'_{\mathcal{B}}^1, x'_{\mathcal{C}}^1$.
7. Ch sends $x'_{\mathcal{B}}^1$ to $\mathcal{B}$ and $x'_{\mathcal{C}}^1$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x'_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x'_{\mathcal{C}}^1)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

The indistinguishability holds due to Lemma 12.9, which is the generalized form of the hidden-trigger lemma with auxiliary information [LLQZ22, Lemma C.7] with CP-Sign and VK plays the role of circuits P and V in Lemma 12.9.

H₇: we generate $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1$ differently.

1. $\mathcal{A}$ sends $\mu_{\mathcal{B}}, \mu_{\mathcal{C}}$.
2. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{ \{ |A_{i s_i, s'_i} \rangle \}_i \}$.
3. Ch samples $x_{\mathcal{B}}^1, x_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^n$. Ch samples $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1 \xleftarrow{\$} \{0, 1\}^m$.
4. Ch then constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 61 and Figure 60, respectively.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{\mathcal{B}, \mathcal{C}}$.
6. Ch runs the Gen-Trigger algorithm (as defined in Definition 12.8) on the first ℓ_0 part of $x_{\mathcal{B}}^1$, and the first ℓ_0 part of $x_{\mathcal{C}}^1$ respectively, and the corresponding $y_{\mathcal{B}}^1$ and $y_{\mathcal{C}}^1$ to get a triggered input $x'_{\mathcal{B}}^1, x'_{\mathcal{C}}^1$ respectively, and sends $x'_{\mathcal{B}}^1, x'_{\mathcal{C}}^1$.
7. Ch sends $x'_{\mathcal{B}}^1$ to $\mathcal{B}$ and $x'_{\mathcal{C}}^1$ to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input $(\sigma_{\mathcal{B}}, x'_{\mathcal{B}}^1)$ and $(\sigma_{\mathcal{C}}, x'_{\mathcal{C}}^1)$ respectively, and outputs bits $b_{\mathcal{B}}$ and $b_{\mathcal{C}}$ respectively.

Note that in H₄, the last $n - \ell_0$ bits of the string $x_{\mathcal{B}}^1$ and $x_{\mathcal{C}}^1$ were not revealed to $\mathcal{A}$, $\mathcal{B}$, and $\mathcal{C}$. Therefore, since $x_{\mathcal{B}}^1, x_{\mathcal{C}}^1$ were chosen uniformly at random from $\{0, 1\}^n$, there are at least $n - \ell_0$ bits of min-entropy left in the random strings $x_{\mathcal{B}}^1, x_{\mathcal{C}}^1$ from the viewpoint of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$. Since by design, $n - \ell_0 \geq k$ (see Condition 4), we conclude by the (computational) extracting property of F_1 that $y_{\mathcal{B}}^1, y_{\mathcal{C}}^1$ in H₄ are computationally indistinguishable from uniformly and independently random strings sampled from $\{0, 1\}^m$. Therefore, H₆ and H₇ are indistinguishable for $(\mathcal{A}, \mathcal{B}, \mathcal{C})$.

Next we combine H₅ of the unpunctured case ($b_{\mathcal{B}} = b_{\mathcal{C}} = 0$ case), H₁₀ of the only punctured for $\mathcal{B}$ case ($b_{\mathcal{B}} = 1, b_{\mathcal{C}} = 0$), H₁ of the only punctured for $\mathcal{C}$ case ($b_{\mathcal{B}} = 0, b_{\mathcal{C}} = 1$), and H₇ of the punctured case ($b_{\mathcal{B}} = b_{\mathcal{C}} = 1$ case) together in the form of the following hybrid:

H*:

1. The challenger Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\{ |A_{i s_i, s'_i} \rangle \}_i$.
2. Ch samples $b_B, b_C \xleftarrow{\$} \{0, 1\}$.
3. Ch samples $x_B^1, x_C^1 \xleftarrow{\$} \{0, 1\}^n$. Ch samples $y_B^1, y_C^1 \xleftarrow{\$} \{0, 1\}^m$.
4. Ch constructs the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 61 and Figure 60, respectively¹⁸.
5. Ch sends $(\sigma, \widetilde{VK}, \widetilde{\text{CP-Sign}})$ to $\mathcal{A}$ who then outputs a state $\sigma_{B,C}$.
6. Ch then samples $x_B^0, x_C^0 \xleftarrow{\$} \{0, 1\}^n$, and $y_B^1, y_C^1 \xleftarrow{\$} \{0, 1\}^m$.
7. Ch runs the Gen-Trigger algorithm (as defined in Definition 12.8) on the first ℓ_0 part of $x_B^{b_B}$, and the first ℓ_0 part of $x_C^{b_C}$ respectively, and the corresponding $y_B^{b_B}$ and $y_C^{b_C}$ to get a triggered input x'_B, x'_C respectively, and sends x'_B, x'_C .
8. Ch sends x'_B to $\mathcal{B}$ and x'_C to $\mathcal{C}$. $\mathcal{B}$ and $\mathcal{C}$ takes as input (σ_B, x'_B) and (σ_C, x'_C) respectively, and outputs bits b_B and b_C respectively.

By the separate hybrid arguments for the four cases, i.e., all possible values for $b_B, b_C \in \{0, 1\}$, we conclude that the success probability of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in H^* is negligibly (in security parameter λ) close to the success probability of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in the original copy-protection experiment for signatures (CP-SignExpt) given in Section 8.1.

Next, we reduce H^* to the modified CPA anti-piracy of the single decryptor encryption [LLQZ22, Definition 3.4] for the single decryptor encryption in [CLLZ21] as follows. We construct the following reduction $(\mathcal{R}_A, \mathcal{R}_B, \mathcal{R}_C)$ against the modified CPA anti-piracy of the single decryptor encryption [LLQZ22, Definition 3.4] for the single decryptor encryption in [CLLZ21], that uses $(\mathcal{A}, \mathcal{B}, \mathcal{C})$.

1. $\mathcal{R}_A$ samples $y_B^0, y_B^1, y_C^0, y_C^1 \xleftarrow{\$} \{0, 1\}^{2m}$, and sends it to the challenger.
2. $\mathcal{R}_A$ receives the public encryption key pk which is the set of circuits $\{R_i^0, R_i^1\}_{i \in [\ell_0]}$, and the quantum state $\{ |A_{i s_i, s'_i} \rangle \}_i$ from the challenger.
3. $\mathcal{R}_A$ samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and uses them along with $\{R_i^0, R_i^1\}_{i \in [\ell_0]}$, and y_B^1, y_C^1 , and circuits μ_B, μ_C (where μ_B, μ_C is fixed parameter in H^*) to construct the circuits $\widetilde{\text{CP-Sign}}$, and $\widetilde{VK}$ where $\widetilde{\text{CP-Sign}} \leftarrow i\mathcal{O}(1^\lambda, \text{CP-Sign})$ and $\widetilde{VK} \leftarrow i\mathcal{O}(1^\lambda, VK)$ where the circuits CP-Sign and VK are as defined in Figure 61 and Figure 60, respectively.

¹⁸The signing and verification circuits in H^* , CP-Sign and VK are independent of b_B, b_C since the CP-Sign circuit in the final hybrid of all the four cases corresponding to the different values for b_B, b_C , i.e., Figures 40, 49, 53 and 61 are the same and similarly, the VK circuit in the final hybrid of all the four cases, namely, Figures 42, 51, 54 and 61 are the same.

4. $\mathcal{R}_A$ runs $\sigma_{\mathcal{B},\mathcal{C}} \leftarrow \mathcal{A}(\rho, \widetilde{VK})$ where $\rho = (\{\{|A_{i s_i, s'_i}\rangle\}_i\}, \widetilde{\text{CP-Sign}})$, and outputs $\sigma_{\mathcal{B},\mathcal{C}}$, along with a copy of $y_{\mathcal{B}}^0, y_{\mathcal{B}}^1$ for $\mathcal{R}_{\mathcal{B}}$ and $y_{\mathcal{C}}^0, y_{\mathcal{C}}^1$ for $\mathcal{R}_{\mathcal{C}}$.
5. $\mathcal{R}_{\mathcal{B}}$ and $\mathcal{R}_{\mathcal{C}}$ receives challenge ciphers $(r_{b_{\mathcal{B}}}^{\mathcal{B}}, Q_{\mathcal{B}})$ and $(r_{b_{\mathcal{C}}}^{\mathcal{C}}, Q_{\mathcal{C}})$ from the challenger, respectively.
6. $\mathcal{R}_{\mathcal{B}}$ generates the input $x'^{b_{\mathcal{B}}}_{\mathcal{B}}$ as follows:

- (a) Generate $x'^{b_{\mathcal{B}},1}_{\mathcal{B}} \leftarrow F_2(K_2, r_0^{\mathcal{B}} \| Q_{\mathcal{B}})$.
- (b) Generate $x'^{b_{\mathcal{B}},2}_{\mathcal{B}} \leftarrow F_3(K_3, x'^{b_{\mathcal{B}},1}_{\mathcal{B}}) \oplus r_0^{\mathcal{B}} \| Q_{\mathcal{B}}$.
- (c) Output $x'^{b_{\mathcal{B}}}_{\mathcal{B}} = r_0^{\mathcal{B}} \| x'^{b_{\mathcal{B}},1}_{\mathcal{B}} \| x'^{b_{\mathcal{B}},2}_{\mathcal{B}}$.

and similarly, $\mathcal{R}_{\mathcal{C}}$ generates the input $x'^{b_{\mathcal{C}}}_{\mathcal{C}}$ as follows:

- (a) Generate $x'^{b_{\mathcal{C}},1}_{\mathcal{C}} \leftarrow F_2(K_2, r_0^{\mathcal{C}} \| Q_{\mathcal{C}})$.
- (b) Generate $x'^{b_{\mathcal{C}},2}_{\mathcal{C}} \leftarrow F_3(K_3, x'^{b_{\mathcal{C}},1}_{\mathcal{C}}) \oplus r_0^{\mathcal{C}} \| Q_{\mathcal{C}}$.
- (c) Output $x'^{b_{\mathcal{C}}}_{\mathcal{C}} = r_0^{\mathcal{C}} \| x'^{b_{\mathcal{C}},1}_{\mathcal{C}} \| x'^{b_{\mathcal{C}},2}_{\mathcal{C}}$.

7. $\mathcal{R}_{\mathcal{B}}$ runs $b_{\mathcal{B}} \leftarrow \mathcal{B}(\sigma_{\mathcal{B}}, x'^{b_{\mathcal{B}}}_{\mathcal{B}})$ and outputs the outcome $b_{\mathcal{B}}$. Similarly, $\mathcal{R}_{\mathcal{C}}$ runs $b_{\mathcal{C}} \leftarrow \mathcal{C}(\sigma_{\mathcal{C}}, x'^{b_{\mathcal{C}}}_{\mathcal{C}})$ and outputs the outcome $b_{\mathcal{C}}$.

Clearly, the view of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in the simulated game of $(\mathcal{R}_A, \mathcal{R}_{\mathcal{B}}, \mathcal{R}_{\mathcal{C}})$ is the same as $\mathbf{H}^*$ such that for any $b_1, b_2 \in \{0, 1\}$, the $b_{\mathcal{B}}, b_{\mathcal{C}} = b_1, b_2$ corresponds to the cipher for $\mathcal{B}$ (and respectively, $\mathcal{C}$) being the encryption of $y_{\mathcal{B}}^{b_1}$ (respectively, $y_{\mathcal{C}}^{b_2}$). Moreover, since $\mathcal{R}_{\mathcal{B}}$ and $\mathcal{R}_{\mathcal{C}}$ output the guesses of $\mathcal{B}$ and $\mathcal{C}$, we conclude that $(\mathcal{R}_A, \mathcal{R}_{\mathcal{B}}, \mathcal{R}_{\mathcal{C}})$ wins the modified CPA game, i.e., guesses of $\mathcal{R}_{\mathcal{B}}$ and $\mathcal{R}_{\mathcal{C}}$ are correct if and only if the guesses of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ of $\mathcal{B}$ and $\mathcal{C}$ are correct, i.e., $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ wins in the simulated $\mathbf{H}^*$, which completes the correctness of the reduction.

Since it was shown in [LLQZ22, Proposition C.8], that the single decryptor encryption [LLQZ22, Definition 3.4] for the single decryptor encryption in [CLLZ21] satisfied the modified anti-piracy security, we conclude that the advantage of $(\mathcal{A}, \mathcal{B}, \mathcal{C})$ in $\mathbf{H}^*$ is negligible, which concludes the security proof for our construction.

References

- [Aar09] Scott Aaronson. “Quantum copy-protection and quantum money”. In: *2009 24th Annual IEEE Conference on Computational Complexity*. IEEE. 2009, pp. 229–242 (cit. on p. 4).
- [AB24] Prabhanjan Ananth and Amit Behera. “A modular approach to unclonable cryptography”. In: *Annual International Cryptology Conference*. Springer. 2024, pp. 3–37 (cit. on pp. 4, 5, 8–11, 14, 15, 22, 23, 26–28, 30, 47).
- [AK22] Prabhanjan Ananth and Fatih Kaleoglu. “A note on copy-protection from random oracles”. In: *arXiv preprint arXiv:2208.12884* (2022) (cit. on p. 4).
- [AKL⁺22] Prabhanjan Ananth, Fatih Kaleoglu, Xingjian Li, Qipeng Liu, and Mark Zhandry. “On the feasibility of unclonable encryption, and more”. In: *Annual International Cryptology Conference*. Springer. 2022, pp. 212–241 (cit. on p. 4).

- [AL21] Prabhanjan Ananth and Rolando L La Placa. “Secure software leasing”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2021, pp. 501–530 (cit. on p. 4).
- [ALL⁺21] Scott Aaronson, Jiahui Liu, Qipeng Liu, Mark Zhandry, and Ruizhe Zhang. “New approaches for quantum copy-protection”. In: *Annual International Cryptology Conference*. Springer. 2021, pp. 526–555 (cit. on p. 4).
- [BBV24] James Bartusek, Zvika Brakerski, and Vinod Vaikuntanathan. “Quantum State Obfuscation from Classical Oracles”. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. 2024, pp. 1009–1017 (cit. on p. 5).
- [BGI⁺01] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. “On the (Im)possibility of Obfuscating Programs”. In: *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*. Ed. by Joe Kilian. Vol. 2139. Lecture Notes in Computer Science. Springer, 2001, pp. 1–18. DOI: 10.1007/3-540-44647-8_1. URL: https://doi.org/10.1007/3-540-44647-8%5C_1 (cit. on p. 28).
- [BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. “Functional signatures and pseudo-random functions”. In: *International workshop on public key cryptography*. Springer. 2014, pp. 501–519 (cit. on p. 4).
- [BGK⁺24] James Bartusek, Vipul Goyal, Dakshita Khurana, Giulio Malavolta, Justin Raizes, and Bhaskar Roberts. “Software with certified deletion”. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer. 2024, pp. 85–111 (cit. on p. 4).
- [BW13] Dan Boneh and Brent Waters. “Constrained pseudorandom functions and their applications”. In: *Advances in Cryptology-ASIACRYPT 2013: 19th International Conference on the Theory and Application of Cryptology and Information Security, Bengaluru, India, December 1-5, 2013, Proceedings, Part II 19*. Springer. 2013, pp. 280–300 (cit. on p. 4).
- [ÇG23] Alper Çakan and Vipul Goyal. “Unclonable cryptography with unbounded collusions”. In: *Cryptology ePrint Archive* (2023) (cit. on p. 4).
- [CG24] Andrea Coladangelo and Sam Gunn. “How to use quantum indistinguishability obfuscation”. In: *Proceedings of the 56th Annual ACM Symposium on Theory of Computing*. 2024, pp. 1003–1008 (cit. on pp. 4, 5, 10).
- [CHJV14] Ran Canetti, Justin Holmgren, Abhishek Jain, and Vinod Vaikuntanathan. “Indistinguishability obfuscation of iterated circuits and RAM programs”. In: *Cryptology ePrint Archive* (2014) (cit. on p. 4).
- [CHV23] Céline Chevalier, Paul Hermouet, and Quoc-Huy Vu. “Semi-quantum copy-protection and more”. In: *Theory of Cryptography Conference*. Springer. 2023, pp. 155–182 (cit. on p. 4).

- [CLLZ21] Andrea Coladangelo, Jiahui Liu, Qipeng Liu, and Mark Zhandry. “Hidden Cosets and Applications to Unclonable Cryptography”. In: *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I*. Ed. by Tal Malkin and Chris Peikert. Vol. 12825. Lecture Notes in Computer Science. Springer, 2021, pp. 556–584. DOI: 10.1007/978-3-030-84242-0_20. URL: https://doi.org/10.1007/978-3-030-84242-0%5C_20 (cit. on pp. 4, 62, 63, 65, 69, 93, 94, 103).
- [CMP24] Andrea Coladangelo, Christian Majenz, and Alexander Poremba. “Quantum copy-protection of compute-and-compare programs in the quantum random oracle model”. In: *Quantum* 8 (2024), p. 1330 (cit. on p. 4).
- [KLW15] Venkata Koppula, Allison Bishop Lewko, and Brent Waters. “Indistinguishability obfuscation for turing machines with unbounded memory”. In: *Proceedings of the forty-seventh annual ACM symposium on Theory of Computing*. 2015, pp. 419–428 (cit. on p. 4).
- [KN22] Fuyuki Kitagawa and Ryo Nishimaki. “Functional encryption with secure key leasing”. In: *International Conference on the Theory and Application of Cryptology and Information Security*. Springer. 2022, pp. 569–598 (cit. on p. 4).
- [KN23] Fuyuki Kitagawa and Ryo Nishimaki. “One-Out-of-Many Unclonable Cryptography: Definitions, Constructions, and More”. In: *Theory of Cryptography - 21st International Conference, TCC 2023, Taipei, Taiwan, November 29 - December 2, 2023, Proceedings, Part IV*. Ed. by Guy N. Rothblum and Hoeteck Wee. Vol. 14372. Lecture Notes in Computer Science. Springer, 2023, pp. 246–275. DOI: 10.1007/978-3-031-48624-1_10. URL: https://doi.org/10.1007/978-3-031-48624-1%5C_10 (cit. on pp. 4, 5, 8, 9, 16, 32).
- [KNY21] Fuyuki Kitagawa, Ryo Nishimaki, and Takashi Yamakawa. “Secure software leasing from standard assumptions”. In: *Theory of Cryptography Conference*. Springer. 2021, pp. 31–61 (cit. on p. 4).
- [KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. “Delegatable pseudorandom functions and applications”. In: *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 2013, pp. 669–684 (cit. on p. 4).
- [LLQZ22] Jiahui Liu, Qipeng Liu, Luowen Qian, and Mark Zhandry. “Collusion Resistant Copy-Protection for Watermarkable Functionalities”. In: *Theory of Cryptography - 20th International Conference, TCC 2022, Chicago, IL, USA, November 7-10, 2022, Proceedings, Part I*. Ed. by Eike Kiltz and Vinod Vaikuntanathan. Vol. 13747. Lecture Notes in Computer Science. Springer, 2022, pp. 294–323. DOI: 10.1007/978-3-031-22318-1_11. URL: https://doi.org/10.1007/978-3-031-22318-1%5C_11 (cit. on pp. 4, 22, 23, 32, 62, 63, 66, 67, 69, 73, 82, 92–94).
- [MM24] Arthur Mehta and Anne Müller. “Unclonable Functional Encryption”. In: *arXiv preprint arXiv:2410.06029* (2024) (cit. on p. 4).
- [SW21] Amit Sahai and Brent Waters. “How to Use Indistinguishability Obfuscation: Deniable Encryption, and More”. In: *SIAM J. Comput.* 50.3 (2021), pp. 857–908. DOI: 10.1137/15M1030108. URL: <https://doi.org/10.1137/15M1030108> (cit. on pp. 4, 23, 62, 63).

- [Zha21] Mark Zhandry. “How to Construct Quantum Random Functions”. In: *J. ACM* 68.5 (2021), 33:1–33:43. DOI: 10.1145/3450745. URL: <https://doi.org/10.1145/3450745> (cit. on p. 100).

A Proof of lemma 12.3

Let F be any extracting puncturable PRF, mapping $\{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$, with extracting error $s(\lambda)$ for min-entropy at least $k(\lambda)$, and PRG be any length-doubling pseudorandom generator, mapping $\{0, 1\}^{m(\lambda)/2} \rightarrow \{0, 1\}^{m(\lambda)}$. It is enough to show that $F' := \text{PRG}(F_1^{\text{half}}(\cdot, \cdot))$ with the same $\mathcal{K}$ (and KeyGen algorithm) is an extractable PRF, mapping $\{0, 1\}^{n(\lambda)} \rightarrow \{0, 1\}^{m(\lambda)}$, with extracting error $s(\lambda)$. Clearly, $F' : \{0, 1\}^n \rightarrow \{0, 1\}^m$. Moreover, for every key k , the image of F' given as $\mathcal{I}(F')$, is contained in the image of PRG, $\mathcal{I}(\text{PRG})$, which has size at most $2^{m/2}$ since $\text{PRG} : \{0, 1\}^{m(\lambda)/2} \rightarrow \{0, 1\}^{m(\lambda)}$. Hence, $\frac{|\mathcal{I}(\text{PRG})|}{2^m} \leq \frac{2^{m/2}}{2^m}$ which is negligible in λ . Hence setting T to be $\mathcal{I}(\text{PRG})$, we conclude that F' satisfies Item 5.

Next, we show that F_1 is puncturable. We use the same puncturing algorithm and the same evaluation algorithm for punctured keys, as those of F_1 .

By the pseudorandomness at the punctured point for F , for any polynomially set of inputs $S \subset \{0, 1\}^n$, and for every $x \in S$,

$$\{K \setminus S, F(K, x)\}_{K \leftarrow F.\text{KeyGen}(1^\lambda)} \approx_c \{K \setminus S, Y\}_{Y \leftarrow \mathbb{S}_{\{0, 1\}^m, K \leftarrow F.\text{KeyGen}(1^\lambda)}}.$$

Hence, by definition of F^{half} , i.e., only any key-input pair (k, x) , $F^{\text{half}}(k, x)$ is just the first $m/2$ bits of $F(k, x)$, we conclude that,

$$\{K \setminus S, F^{\text{half}}(K, x)\}_{K \leftarrow F.\text{KeyGen}(1^\lambda)} \approx_c \{K \setminus S, Y\}_{Y \leftarrow \mathbb{S}_{\{0, 1\}^{m/2}, K \leftarrow F.\text{KeyGen}(1^\lambda)}}.$$

Therefore, by the pseudorandomness of PRG,

$$\{K \setminus S, \text{PRG}(F^{\text{half}}(K, x))\}_{K \leftarrow F.\text{KeyGen}(1^\lambda)} \approx_c \{K \setminus S, Y\}_{Y \leftarrow \mathbb{S}_{\{0, 1\}^m, K \leftarrow F.\text{KeyGen}(1^\lambda)}},$$

which is the same as

$$\{K \setminus S, F'(K, x)\}_{K \leftarrow F.\text{KeyGen}(1^\lambda)} \approx_c \{K \setminus S, Y\}_{Y \leftarrow \mathbb{S}_{\{0, 1\}^m, K \leftarrow F.\text{KeyGen}(1^\lambda)}}.$$

Hence, we conclude that F' is puncturable.

Next, we show that F' is an extracting puncturable PRF with extracting error (with respect to computationally bounded algorithms) $s(\lambda)$ for min-entropy at least $k(\lambda)$. Let X be any arbitrary random variable supported in $\{0, 1\}^n$ with min-entropy at least $k = k(\lambda)$. We need to show that

$$\{K, F'(K, X)\}_{K \leftarrow F.\text{KeyGen}(1^\lambda)} \approx \{K, Y\}_{Y \leftarrow \mathbb{S}_{\{0, 1\}^m, K \leftarrow F.\text{KeyGen}(1^\lambda)}},$$

for any computationally bounded distinguisher D , up to a distinguishing error of $s = s(\lambda)$. Since F is an extracting puncturable with extracting error $s(\lambda)$ for min-entropy at least $k(\lambda)$, it holds that

$$\{K, F(K, X)\}_{K \leftarrow F.\text{KeyGen}(1^\lambda)} \approx \{K, Y\}_{Y \leftarrow \mathbb{S}_{\{0, 1\}^m, K \leftarrow F.\text{KeyGen}(1^\lambda)}},$$

up to a statistical distinguishing error (or statistical distance) $s = s(\lambda)$. Hence, by definition of F^{half} ,

$$\{K, F^{\text{half}}(K, X)\}_{K \leftarrow F.\text{KeyGen}(1^\lambda)} \approx \{K, Y\}_{Y \leftarrow \mathbb{S}_{\{0, 1\}^{m/2}, K \leftarrow F.\text{KeyGen}(1^\lambda)}}.$$

up to a statistical distinguishing error (or statistical distance) $s = s(\lambda)$. Therefore, by the pseudorandomness of PRG,

$$\{K, F'(K, X) = \text{PRG}(F^{\text{half}}(K, X))\}_{K \leftarrow F.\text{KeyGen}(1^\lambda)} \approx_c \{K, Y\}_{Y \xleftarrow{\$} \{0,1\}^m, K \leftarrow F.\text{KeyGen}(1^\lambda)},$$

for any computationally bounded distinguisher D , up to a distinguishing error of $s = s(\lambda)$, which completes the proof of extracting puncturable PRF property.

Finally, we show that F' is indeed a PRF. Fix any computationally bounded adversary $\mathcal{A}$. We will show the pseudorandomness of F' using the following hybrid arguments.

H_0 : We indicate the changes in the consecutive hybrids in [blue](#). This is the original pseudorandomness game for F' .

1. Ch samples a key $K \leftarrow F.\text{KeyGen}(1^\lambda)$, a uniformly random function $G \xleftarrow{\$} \{0,1\}^{n\{0,1\}^m}$ and a bit $b \xleftarrow{\$} \{0,1\}$.
2. If $b = 0$ Ch gives the adversary $\mathcal{A}$ oracle access to $F'(K, \cdot) = \text{PRG}(F^{\text{half}}(K, \cdot))$, and if $b = 1$, gives the adversary $\mathcal{A}$ oracle access to G .
3. $\mathcal{A}$ outputs a bit b' .
4. $\mathcal{A}$ wins if $b = b'$.

H_1 :

1. ~~Ch samples a key $K \leftarrow F.\text{KeyGen}(1^\lambda)$~~ Ch samples a uniformly random function $H \xleftarrow{\$} \{0,1\}^{n\{0,1\}^{m/2}}$, a random function $G \xleftarrow{\$} \{0,1\}^{n\{0,1\}^m}$ and a bit $b \xleftarrow{\$} \{0,1\}$.
2. If $b = 0$ Ch gives the adversary $\mathcal{A}$ oracle access to [PRG](#)($H(K, \cdot)$), and if $b = 1$, gives the adversary $\mathcal{A}$ oracle access to G .
3. $\mathcal{A}$ outputs a bit b' .
4. $\mathcal{A}$ wins if $b = b'$.

The indistinguishability follows by the pseudorandomness of the PRF F .

H_2 :

1. Ch samples a uniformly random function $H' \xleftarrow{\$} \{0,1\}^{n\{0,1\}^m}$, a random function $G \xleftarrow{\$} \{0,1\}^{n\{0,1\}^m}$ and a bit $b \xleftarrow{\$} \{0,1\}$.
2. If $b = 0$ Ch gives the adversary $\mathcal{A}$ oracle access to [H'](#), and if $b = 1$, gives the adversary $\mathcal{A}$ oracle access to G .
3. $\mathcal{A}$ outputs a bit b' .
4. $\mathcal{A}$ wins if $b = b'$.

Before showing indistinguishability between H_1 and H_2 , note that for both $b = 0$ and $b = 1$, the view of $\mathcal{A}$ is the same and hence $\mathcal{A}$ cannot win H_2 with probability more than $\frac{1}{2}$, which completes the proof of pseudorandomness of F' assuming the indistinguishability between hybrids H_1 and H_2 .

Next we show that H_1 and H_2 are indistinguishable to complete the proof. Let $\mathcal{X} = \{0, 1\}^n$. Let $D_1(1^\lambda)$ denote the distribution on $\{0, 1\}^m$. $\{\text{PRG}(Y)\}_{Y \leftarrow \$_{\{0, 1\}^{m/2}}}$, and $D_2(1^\lambda)$ denote the distribution $\{Z\}_{Z \leftarrow \$_{\{0, 1\}^m}}$. For any $i \in [2]$, let $D_i^{\mathcal{X}}(1^\lambda)$ denote an oracle distribution on functions from $\{0, 1\}^{n(\lambda)}$ to $\{0, 1\}^{m(\lambda)}$, i.e., for each input $x \in \{0, 1\}$, $D_1^{\mathcal{X}}(1^\lambda)$ samples a random point $Y_x \in \{0, 1\}^{m/2}$ and then outputs $Z_x \leftarrow \text{PRG}(Y_x)$, and let $D_2^{\mathcal{X}}$ outputs a uniformly random sample $Z_x \leftarrow \$_{\{0, 1\}^m}$ sampled uniformly for each $x \in \{0, 1\}^n$. Clearly, for $i \in [2]$, O_i corresponds to the oracle distribution, oracle drawn from which is given to $\mathcal{A}$ as a quantum oracle access in the $b = 0$ case of H_i . Since D_1 and D_2 are efficiently sampleable, by [Zha21, Theorem 1.1], in order to show oracle indistinguishability between D_1 and D_2 (as defined in [Zha21, Definition 2.5]), i.e., indistinguishability between the oracle distributions $D_1^{\mathcal{X}}$ and $D_2^{\mathcal{X}}$, it is enough to show that (sample) indistinguishability holds between D_1 and D_2 .

To argue indistinguishability between D_1 and D_2 , note that by the pseudorandomness of PRG, for any $x \in \{0, 1\}^n$,

$$\{Z_x\}_{Z \leftarrow O_1(x)} = \{Z_x\}_{Z_x \leftarrow \text{PRG}(Y_x), Y_x \leftarrow \$_{\{0, 1\}^{m/2}}} \approx_c \{Z_x\}_{Z_x \leftarrow \$_{\{0, 1\}^m}} = \{Z\}_{Z_x \leftarrow O_2(x)},$$

which is the same as saying D_1 and D_2 are indistinguishable as defined [Zha21, Definition 2.4].

B Proof of lemma 12.9

H_0 :

This is the original game where the input is sampled either uniformly at random or sampled as a hidden trigger input, written in a rearranged manner. Also we drop the subscripts from the circuits V and P for better readability.

1. Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \leftarrow \$_{\{0, 1\}^n}$, and computes $y_u \leftarrow F_1(K_1, u)$ and $y_v \leftarrow F_1(K_1, v)$.
2. Ch samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{ \{ |A_{i s_i, s'_i} \rangle \}_i \}$.
3. Ch parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$. Then Ch generates $u' \leftarrow \text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.

Similarly, Ch generates $v' \leftarrow \text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.

4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1_{A^\perp+s'_i})$, and V is the circuit given in Figure 62, for some circuit $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(P)$ where P is the circuit given in Figure 63, for some circuit $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ which has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.

5. Ch samples $b \xleftarrow{\$} \{0, 1\}$, and if $b = 0$ sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$ sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

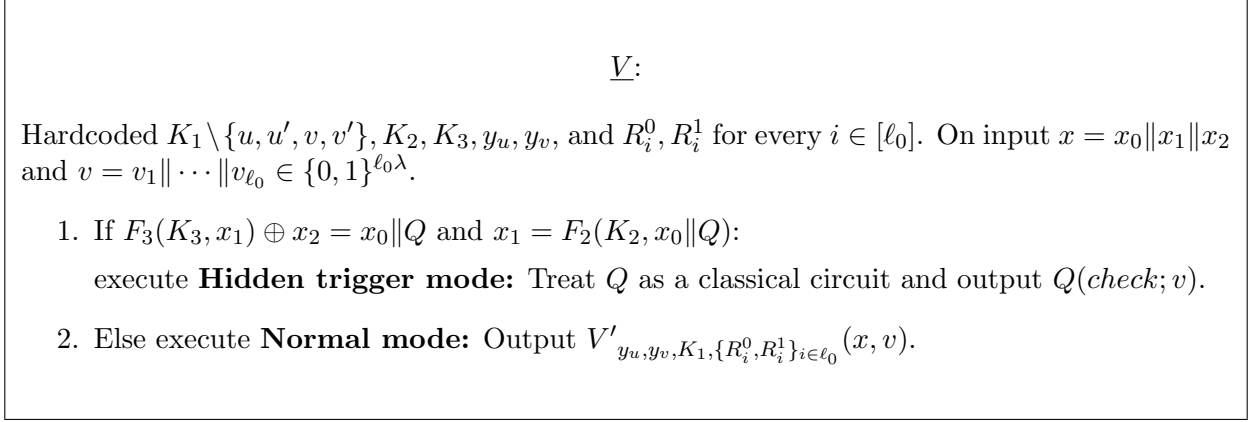


Figure 62: Circuit V in H_0 .

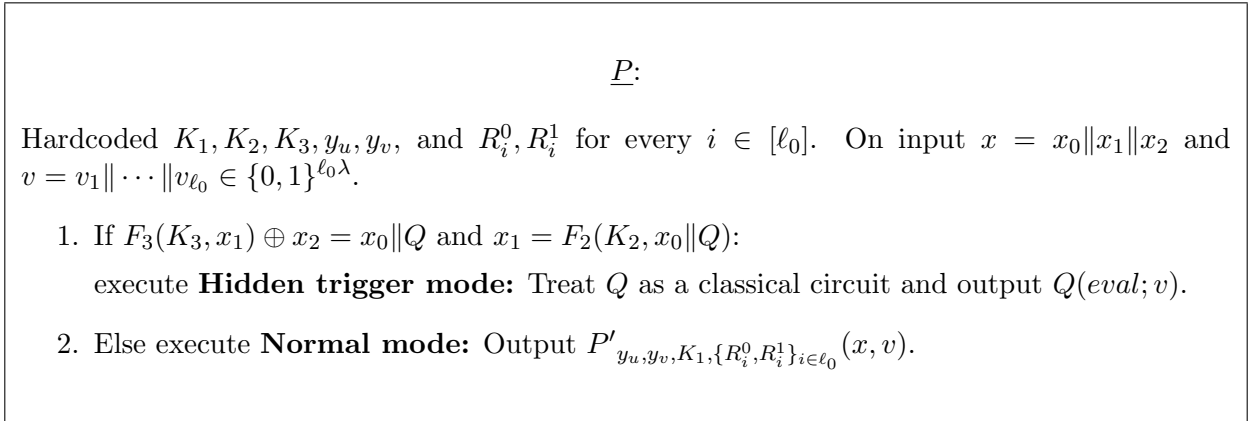


Figure 63: Circuit P in H_0 .

H_1 :

We change the circuits P and V by replacing the hard-coded key K_1 with $K_1 \setminus \{u, u', v, v'\}$, and then adding a check to handle the case when $x \in \{u, u', v, v'\}$. Clearly, the functionality of P and V did not change across H_0 and H_1 , and therefore the indistinguishability of the hybrids hold by the $i\mathcal{O}$ security.

1. Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \xleftarrow{\$} \{0, 1\}^n$, and computes $y_u \leftarrow F_1(K_1, u)$ and $y_v \leftarrow F_1(K_1, v)$.
2. Ch samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{\{|A_{i s_i, s'_i}\rangle\}_i\}$.

3. Ch parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$. Then Ch generates $u' \leftarrow \text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$. Let Q_u denote the circuit generated during the execution of $\text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.
Similarly, Ch generates $v' \leftarrow \text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$. Let Q_v denote the circuit generated during the execution of $\text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.
4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1_{A^\perp+s'_i})$, and V is the circuit given in Figure 64, for some circuit $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(P)$ where P is the circuit given in Figure 65, for some circuit $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ which has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
5. Ch samples $b \xleftarrow{\$} \{0, 1\}$, and if $b = 0$ sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$ sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

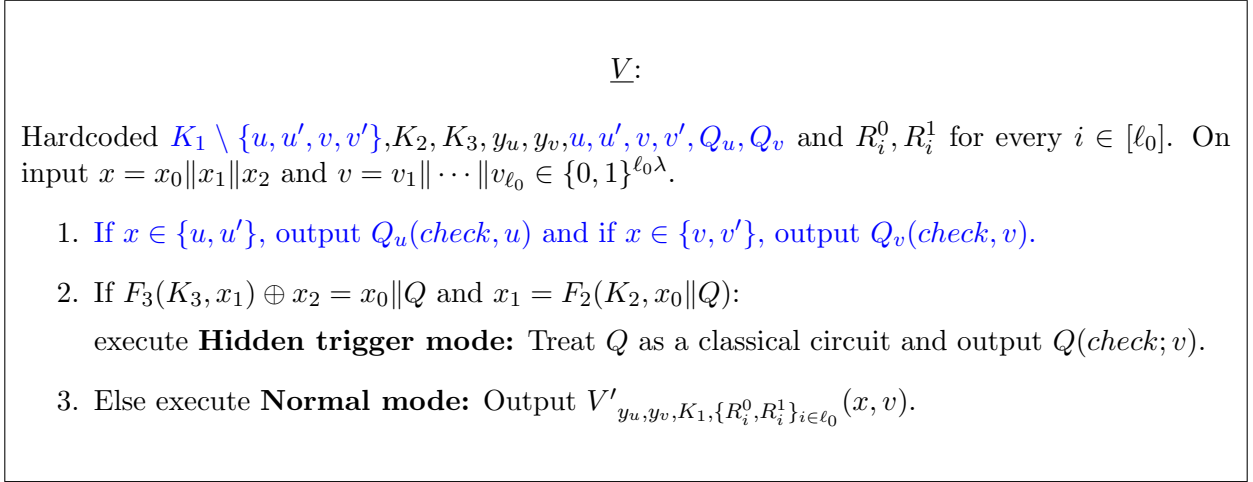


Figure 64: Circuit V in H_1 .

From H_1 onwards, whenever we use the K_1 inside the description of P and V we actually mean using the key $K_1 \setminus \{u, u', v, v'\}$.

H_2 : Next, we replace y_u and y_v with a uniformly random string using the puncturing security of F_1 .

1. Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \xleftarrow{\$} \{0, 1\}^n$, and computes $y_u \xleftarrow{\$} \{0, 1\}^m$ and $y_v \xleftarrow{\$} \{0, 1\}^m$.
2. Ch samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{\{|A_{i s_i, s'_i}\rangle\}_i\}$.
3. Ch parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$. Then Ch generates $u' \leftarrow \text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$. Let Q_u denote the circuit generated during the execution of $\text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.

P :

Hardcoded $K_1 \setminus \{u, u', v, v'\}, K_2, K_3, y_u, y_v, u, u', v, v', Q_u, Q_v$ and R_i^0, R_i^1 for every $i \in [\ell_0]$. On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $x \in \{u, u'\}$, output $Q_u(\text{eval}, u)$ and if $x \in \{v, v'\}$, output $Q_v(\text{eval}, v)$.
2. If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
execute **Hidden trigger mode**: Treat Q as a classical circuit and output $Q(\text{eval}; v)$.
3. Else execute **Normal mode**: Output $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}(x, v)$.

Figure 65: Circuit P in H_1 .

Similarly, Ch generates $v' \leftarrow \text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$. Let Q_v denote the circuit generated during the execution of $\text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.

4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1_{A^\perp+s'_i})$, and V is the circuit given in Figure 64, for some circuit $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(P)$ where P is the circuit given in Figure 65, for some circuit $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ which has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
5. Ch samples $b \xleftarrow{\$} \{0, 1\}$, and if $b = 0$ sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$ sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

H_3 : Next, we change the circuits P and V as follows. We additionally skip the hidden trigger check if $x_1 \in \{u_1, u'_1, v_1, v'_1\}$, in both P and V , and since we are never evaluating F_3 on $\{u_1, u'_1, v_1, v'_1\}$, we replce the key K_3 with the punctured key $K_3 \setminus \{u_1, u'_1, v_1, v'_1\}$. By [CLLZ21, Lemma 7.14] except with negligible probability over K_2 , the event E holds where E is the random event that

1. For every string $x_1 \in \{0, 1\}^{\ell_1}$, there is at most one value of (x_0, x_2) such that (x_0, x_1, x_2) satisfies the hidden-trigger check, given in Item 2 in Figure 64.
2. There are at most 2^{ℓ_2} distinct strings $x \in \{0, 1\}^n$ such that x passes the hidden trigger check.

Hence, conditioned on E , u' and v' are the only strings of the form $w_0 \| u_1 \| w_2$ and $w_0 \| v_1 \| w_2$ respectively that pass the hidden-trigger check. Hence conditioned on E , first line where we check if $x \in \{u, u', v, v'\}$ takes care of every input $x \in \{0, 1\}^n$ such that $x_1 = u'_1$ or $x_1 = v'_1$ that passes the hidden-trigger check. Therefore, conditioned on E which holds except with negligible probability, skipping the hidden trigger check when $x_1 \in \{u'_1, v'_1\}$ does not change the functionality across H_2 to H_3 . Next by [CLLZ21, Lemma 7.14] except with negligible probability over K_2 , the event F holds where F is the random event that for every string $x_1 \in \{0, 1\}^{\ell_1}$, there is at most one value of (x_0, x_2) such that (x_0, x_1, x_2) satisfies the hidden-trigger check. On the other hand, conditioned

on E there are at most 2^{ℓ_2} values of x , and hence there are at most 2^{ℓ_2} values of $x_1 \in \{0,1\}^{\ell_1}$ such that there is a $x = x_0 \| x_1 \| x_2 \in \{0,1\}^n$ passes the hidden trigger check. Since u_1, v_1 were sampled uniformly at random from $\{0,1\}^{\ell_1}$, conditioned on E which holds except with negligible probability, the probability that there exists a string x with $x_1 = u_1$ such that x passes Hidden trigger check is at most $\frac{2^{\ell_2}}{2^{\ell_1}} = \frac{1}{2^{\ell_1 - \ell_2}}$, which is negligible in λ for our choice of parameters. Hence, overall we conclude that except with negligible probability over the choice of K_2 , the functionality of both P and V did not change across H_2 and H_3 , and hence the indistinguishability between the hybrids hold by $i\mathcal{O}$ security.

1. Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \xleftarrow{\$} \{0,1\}^n$, and computes $y_u \xleftarrow{\$} \{0,1\}^m$ and $y_v \xleftarrow{\$} \{0,1\}^m$.
2. Ch samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{ \{ |A_{is_i, s'_i}\rangle \}_i \}$.
3. Ch parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$. Then Ch generates $u' \leftarrow \text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$. Let Q_u denote the circuit generated during the execution of $\text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.

Similarly, Ch generates $v' \leftarrow \text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$. Let Q_v denote the circuit generated during the execution of $\text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.

4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1_{A^\perp+s'_i})$, and V is the circuit [given in Figure 66](#), for some circuit $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(P)$ where P is the circuit [given in Figure 67](#), for some circuit $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ which has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
5. Ch samples $b \xleftarrow{\$} \{0,1\}$, and if $b = 0$ sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$ sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

V:

Hardcoded $K_1 \setminus \{u, u', v, v'\}$, $K_2, K_3 \setminus \{u_1, u'_1, v_1, v'_1\}$, $y_u, y_v, u, u', v, v', Q_u, Q_v$ and R_i^0, R_i^1 for every $i \in [\ell_0]$. On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $x \in \{u, u'\}$, output $Q_u(\text{check}, u)$ and if $x \in \{v, v'\}$, output $Q_v(\text{check}, v)$.
2. If $x_1 \in \{u_1, u'_1, v_1, v'_1\}$, skip the following check:
 If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
 execute **Hidden trigger mode**: Treat Q as a classical circuit and output $Q(\text{check}; v)$.
3. Else execute **Normal mode**: Output $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}(x, v)$.

Figure 66: Circuit V in H_3 .

P:

Hardcoded $K_1 \setminus \{u, u', v, v'\}$, $K_2, K_3 \setminus \{u_1, u'_1, v_1, v'_1\}$, $y_u, y_v, u, u', v, v', Q_u, Q_v$ and R_i^0, R_i^1 for every $i \in [\ell_0]$. On input $x = x_0 \| x_1 \| x_2$ and $v = v_1 \| \dots \| v_{\ell_0} \in \{0, 1\}^{\ell_0 \lambda}$.

1. If $x \in \{u, u'\}$, output $Q_u(\text{eval}, u)$ and if $x \in \{v, v'\}$, output $Q_v(\text{eval}, v)$.
2. If $x_1 \in \{u_1, u'_1, v_1, v'_1\}$, skip the following check:
 If $F_3(K_3, x_1) \oplus x_2 = x_0 \| Q$ and $x_1 = F_2(K_2, x_0 \| Q)$:
 execute **Hidden trigger mode**: Treat Q as a classical circuit and output $Q(\text{eval}; v)$.
3. Else execute **Normal mode**: Output $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}(x, v)$.

Figure 67: Circuit P in H_3 .

From H_3 onwards, whenever we use the K_3 inside the description of P and V we actually mean using the key $K_3 \setminus \{u_1, u'_1, v, v'_1\}$.

H_4 : Next we change P and V as follows: In both P and V , during the hidden triggers check, we execute the check if $x_1 = F_2(K_2, x_0 \| Q)$, only if $x_0 \| Q \notin \{u_0 \| Q_u, v_0 \| Q_v\}$. Note that by definition of **Gen-Trigger** procedure (see Definition 12.8), if $x_0 \| Q \in \{u_0 \| Q_u, v_0 \| Q_v\}$, then $x_1 = F_2(K_2) \in \{u'_1, v'_1\}$, and therefore, the hidden-trigger check (the check given in Item 2 in Figure 64) must have been skipped. Therefore, adding this extra check that $x_0 \| Q \notin \{u_0 \| Q_u, v_0 \| Q_v\}$ does not change the functionality of P and V across H_3 and H_4 .

1. Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \xleftarrow{\$} \{0, 1\}^n$, and computes $y_u \xleftarrow{\$} \{0, 1\}^m$ and $y_v \xleftarrow{\$} \{0, 1\}^m$.
2. Ch samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{\{|A_{i_{s_i, s'_i}}\rangle\}_i\}$.
3. Ch parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$. Then Ch generates $u' \leftarrow \text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$. Let Q_u denote the circuit generated during the execution of $\text{Gen-Trigger}(u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.
Similarly, Ch generates $v' \leftarrow \text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$. Let Q_v denote the circuit generated during the execution of $\text{Gen-Trigger}(v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0})$.
4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1_{A^\perp+s'_i})$, and V is the circuit [given in Figure 68](#), for some circuit $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(P)$ where P is the circuit [given in Figure 69](#), for some circuit $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ which has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
5. Ch samples $b \xleftarrow{\$} \{0, 1\}$, and if $b = 0$ sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$ sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

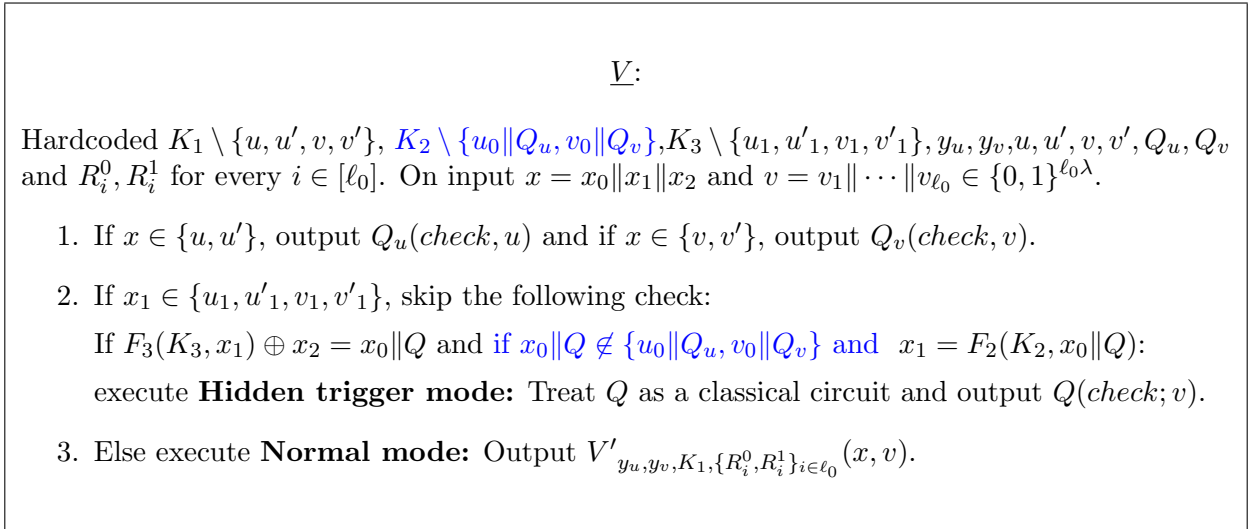


Figure 68: Circuit V in H_4 .

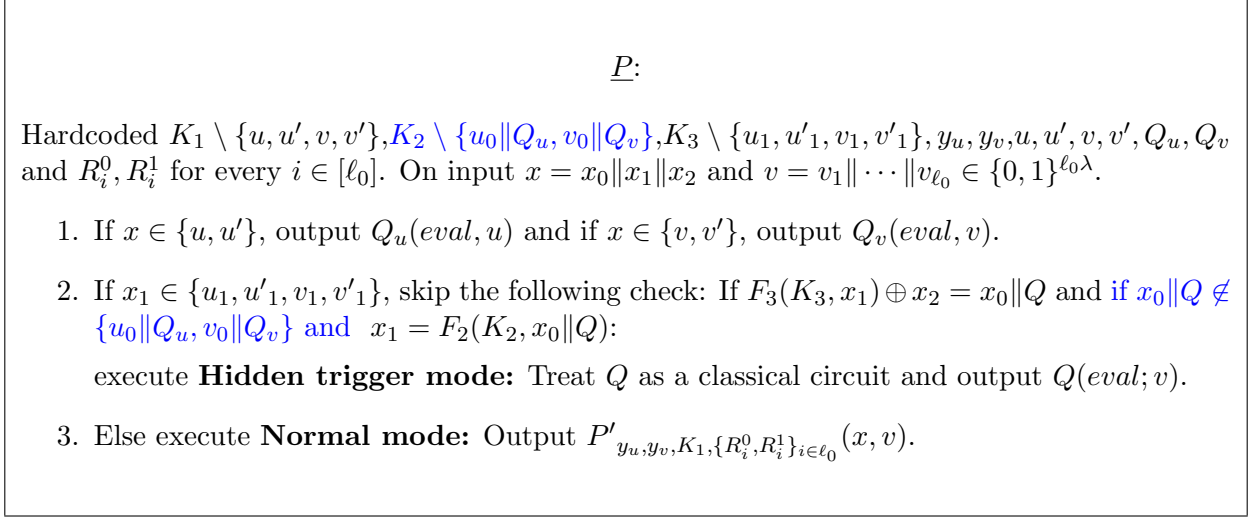


Figure 69: Circuit P in H_4 .

From H_4 onwards, whenever we use the K_2 inside the description of P and V we actually mean using the key $K_2 \setminus \{u_0 \| Q_u, v_0 \| Q_v\}$.

H_5 : We change how u'_1, v'_1 is sampled. Instead of generating u_1, v_1 as per **Gen-Trigger** algorithm (see Definition 12.8) we instead sample u_1, v_1 uniformly at random. The indistinguishability between the hybrids holds due to the puncturing security of F_3 at the points $u_0 \| Q_u$ and $v_0 \| Q_v$.

1. Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \xleftarrow{\$} \{0, 1\}^n$, and computes $y_u \xleftarrow{\$} \{0, 1\}^m$ and $y_v \xleftarrow{\$} \{0, 1\}^m$.
2. Ch samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{\{|A_{i s_i, s'_i}\}_{i \in \ell_0}\}$.
3. Ch parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$. Then Ch generates u' as follows:
 - (a) Let Q_u denote the circuit of the form given in Figure 36 generated by **Gen-Trigger**($u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}$) that has y_u , and $\{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, let Q_v denote the circuit of the form given in Figure 36 generated by **Gen-Trigger**($v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0}$) that has y_v , and $\{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
 - (b) $u'_1, v'_1 \xleftarrow{\$} \{0, 1\}^{\ell_1}$.
 - (c) Generate $u'_2 \leftarrow F_3(K_3, u'_1) \oplus u_0 \| Q_u$, $v'_2 \leftarrow F_3(K_3, v'_1) \oplus v_0 \| Q_v$.
 - (d) Let $u' = u_0 \| u'_1 \| u'_2$, and $v' = v_0 \| v'_1 \| v'_2$.
4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1_{A^\perp+s'_i})$, and V is the circuit given in Figure 68, for some circuit $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(P)$ where P is the circuit given in Figure 69, for some circuit $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ which has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.

5. Ch samples $b \xleftarrow{\$} \{0, 1\}$, and if $b = 0$ sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$ sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

H₆: We change how u'_2, v'_2 is sampled. Instead of generating u_1, v_1 as per Gen-Trigger algorithm (see Definition 12.8) we instead sample u_1, v_1 uniformly at random. The indistinguishability between the hybrids holds due to the puncturing security of F_2 at u'_1 and v'_1 .

1. Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \xleftarrow{\$} \{0, 1\}^n$, and computes $y_u \xleftarrow{\$} \{0, 1\}^m$ and $y_v \xleftarrow{\$} \{0, 1\}^m$.
2. Ch samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{\{|A_{is_i, s'_i}\rangle\}_i\}$.
3. Ch parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$. Then Ch generates u' as follows:
 - (a) Let Q_u denote the circuit of the form given in Figure 36 generated by Gen-Trigger($u_0, y_u, K_2, K_3, \{R_i^0, R_i^1\}$) that has y_u , and $\{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, let Q_v denote the circuit of the form given in Figure 36 generated by Gen-Trigger($v_0, y_v, K_2, K_3, \{R_i^0, R_i^1\}_{i \in \ell_0}$) that has y_v , and $\{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
 - (b) $u'_1, v'_1 \xleftarrow{\$} \{0, 1\}^{\ell_1}$.
 - (c) $u'_2, v'_2 \xleftarrow{\$} \{0, 1\}^{\ell_2}$.
 - (d) Let $u' = u_0 \| u'_1 \| u'_2$, and $v' = v_0 \| v'_1 \| v'_2$.
4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1_{A^\perp+s'_i})$, and V is the circuit given in Figure 68, for some circuit $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(P)$ where P is the circuit given in Figure 69, for some circuit $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ which has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
5. Ch samples $b \xleftarrow{\$} \{0, 1\}$, and if $b = 0$ sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$ sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

H₇: This is the same as the previous hybrid except that we do not generate the circuits Q_u and Q_v in the experiment, as the messages that the challenger sends to the adversary $\mathcal{A}$ in H₆ are independent of Q_u and Q_v and can be generated without the need of Q_u and Q_v . The view of $\mathcal{A}$ did not change across H₆ and H₇, and hence, indistinguishability between the hybrids holds.

1. Ch samples PRF keys K_i for F_i with security parameter λ for every $i \in \{1, 2, 3\}$, and $u, v \xleftarrow{\$} \{0, 1\}^n$, and computes $y_u \xleftarrow{\$} \{0, 1\}^m$ and $y_v \xleftarrow{\$} \{0, 1\}^m$.
2. Ch samples subspaces $\{A_i\}_{i \in \ell_0}$ and coset off-sets $\{s_i, s'_i\}_{i \in \ell_0}$ and constructs the state $\sigma = \{\{|A_{is_i, s'_i}\rangle\}_i\}$.
3. Ch parses the strings $u = (u_0 \| u_1 \| u_2)$ and $v = (v_0 \| v_1 \| v_2)$. Then Ch generates u' as follows:
 - (a) $u'_1, v'_1 \xleftarrow{\$} \{0, 1\}^{\ell_1}$.

- (b) $u'_2, v'_2 \xleftarrow{\$} \{0, 1\}^{\ell_2}$.
- (c) Let $u' = u_0 \| u'_1 \| u'_2$, and $v' = v_0 \| v'_1 \| v'_2$.
4. Ch prepares a circuit $\tilde{V} \leftarrow i\mathcal{O}(V)$, where for every $i \in [\ell_0]$, R_i^0, R_i^1 are generated as $R_i^0 \leftarrow i\mathcal{O}(1_{A+s_i})$ and $R_i^1 \leftarrow i\mathcal{O}(1_{A^\perp+s'_i})$, and V is the circuit given in Figure 68, for some circuit $V'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ that has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it. Similarly, it also prepares the circuit $\tilde{P} \leftarrow i\mathcal{O}(P)$ where P is the circuit given in Figure 69, for some circuit $P'_{y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}}$ which has only $y_u, y_v, K_1, \{R_i^0, R_i^1\}_{i \in \ell_0}$ hardcoded in it.
 5. Ch samples $b \xleftarrow{\$} \{0, 1\}$, and if $b = 0$ sends $(\sigma, \tilde{P}, \tilde{V}, u, v, y_u, y_v)$ and if $b = 1$ sends $(\sigma, \tilde{P}, \tilde{V}, u', v', y_u, y_v)$ to $\mathcal{A}$ who then outputs a bit b' , and wins the game if $b = b'$.

Note that in H_7 , the view of the adversary $\mathcal{A}$ is the same for both $b = 0$ and $b = 1$, and hence $\mathcal{A}$ cannot win H_7 with probability more than $\frac{1}{2}$. Hence by the indistinguishability between consecutive hybrids, we conclude that $\mathcal{A}$ cannot win the original security game H_0 except with negligible advantage.